

APOSTILA COMPLETA

Tkinter & CustomTkinter

Tudo explicado, com telas desenhadas

Do zero ao avançado • 200 páginas

`minha_app.py`

Bem-vindo(a) à sua jornada GUI!

Começar

Sumário

Parte I — Tkinter

1. Introdução à GUI com Python	p. 5
2. Sua primeira janela Tk	p. 9
3. Widgets básicos: Label, Button, Entry	p. 13
4. Gerenciadores de layout: pack, grid, place	p. 21
5. Eventos, comandos e binds	p. 31
6. Variáveis Tk (StringVar, IntVar, DoubleVar, BooleanVar)	p. 37
7. Checkbutton, Radiobutton, Scale e Spinbox	p. 41
8. Listbox, Combobox e Treeview (ttk)	p. 49
9. Text e Canvas — desenho e edição	p. 57
10. Menus, Toolbar e StatusBar	p. 65
11. Diálogos: messagebox, filedialog, colorchooser	p. 71
12. Toplevel, modais e múltiplas janelas	p. 77
13. Frames, LabelFrame e PanedWindow	p. 83
14. Estilos com ttk.Style e temas	p. 89
15. Imagens com PhotoImage e Pillow	p. 95
16. Threads, after() e tarefas longas	p. 101
17. Projeto: Bloco de Notas	p. 107
18. Projeto: Calculadora	p. 113
19. Projeto: To-Do List com SQLite	p. 119

Parte II — CustomTkinter

20. Apresentando o CustomTkinter <code>minha_app.py</code>	p. 127
21. Instalação e primeira CTK()	p. 131
22. Aparência: light, dark e system; color themes	p. 137
23. Widgets CTK: Button, Label, Entry, Frame	p. 143
24. CTKSwitch, CTKCheckBox, CTKRadioButton	p. 151
25. CTKSlider, CTKProgressBar, CTKSegmentedButton	p. 157
26. CTKOptionMenu, CTKComboBox e CTKTextbox	p. 163
27. CTKTabview, CTKScrollableFrame	p. 169
28. CTKImage, CTKFont e ícones	p. 175
29. Layouts responsivos com grid	p. 181
30. Projeto: Dashboard moderno	p. 187
31. Projeto: Login + Cadastro	p. 193
32. Empacotando seu app (PyInstaller)	p. 197
Apêndice A — Referência rápida	p. 199

APOSTILA COMPLETA

Tkinter & CustomTkinter

Tudo explicado, com telas desenhadas

Múltiplas janelas • 200 páginas

Começar

Material didático em português • Python 3.x

Parte I — Tkinter

Nesta primeira parte estudaremos o **Tkinter**, a biblioteca padrão de criação de interfaces gráficas do Python. Você aprenderá desde a estrutura mínima de uma janela até projetos completos, passando por widgets, layouts, eventos, estilos (*ttk*), imagens, threads e diálogos.

Cada capítulo traz: **explicação conceitual**, **código comentado**, **tela desenhada** mostrando como o resultado aparece e **exercícios** propostos.

Nota: Pré-requisitos: Python 3.10+ instalado e conhecimentos básicos de funções, classes e listas. Não é necessário experiência prévia com GUI.

Capítulo 1 — Introdução à GUI com Python

1.1 O que é uma GUI?

GUI é a sigla para *Graphical User Interface* — interface gráfica do usuário. Em vez de digitar comandos em um terminal, o usuário interage com **janelas**, **botões**, **campos de texto** e **menus**. Toda aplicação visual que você usa no dia a dia é uma GUI.

Python oferece várias bibliotecas para criar GUIs: **Tkinter** (padrão), **PyQt**, **Kivy**, **wxPython**, entre outras. Nesta apostila focaremos em Tkinter por já vir embutido na linguagem e, depois, no **CustomTkinter**, uma camada moderna sobre ele.

Os elementos visuais (botões, rótulos, listas) são chamados de **widgets**. Cada widget é um objeto Python com propriedades (cor, texto, fonte) e métodos (mostrar, esconder, etc.).

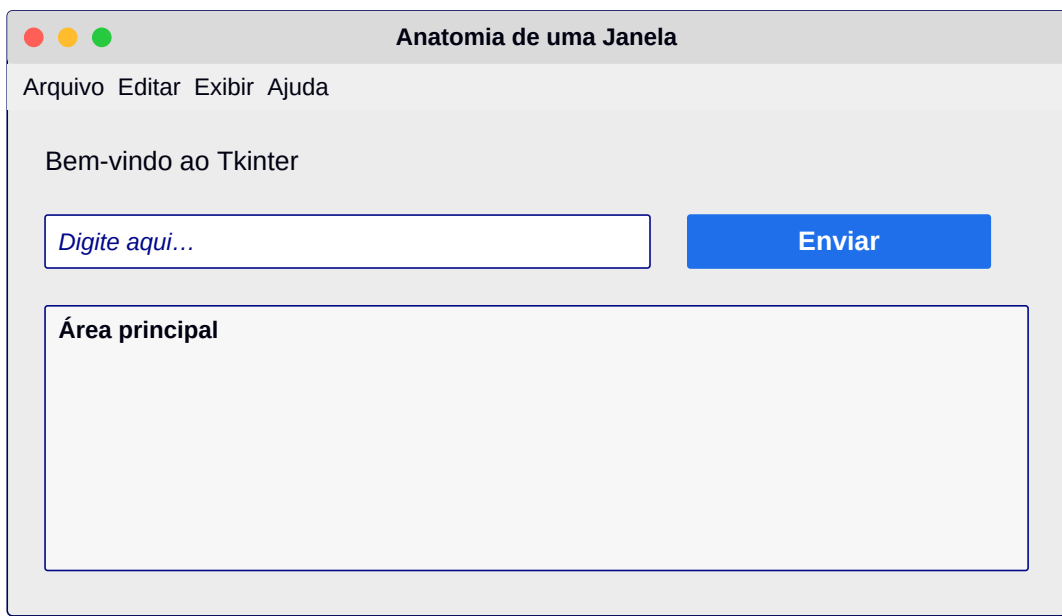


Figura 1.1 — Anatomia básica: barra de menu, rótulos, campo, botão e área principal.

1.2 Tkinter em uma linha

O Tkinter é um *wrapper* sobre a biblioteca C/Tcl chamada **Tk**. Quando você escreve `import tkinter` está, por baixo dos panos, carregando o Tk.

```
import tkinter as tk
janela = tk.Tk()
janela.title('Olá, Mundo!')
janela.mainloop()
```

1.3 O que vamos construir

Ao longo desta apostila implementaremos diversos projetos práticos: bloco de notas, calculadora, lista de tarefas com banco de dados, dashboards modernos e telas de login. Tudo

com explicação passo a passo e mockup visual.

Capítulo 2 — Sua primeira janela Tk

2.1 Estrutura mínima

Toda aplicação Tkinter segue o mesmo esqueleto: importar, criar a janela raiz, configurar, adicionar widgets e chamar `mainloop()`, que mantém a janela aberta processando eventos.

```
import tkinter as tk

app = tk.Tk()                # 1) cria a janela raiz
app.title('Minha Primeira GUI')
app.geometry('400x250')      # largura x altura em pixels
app.resizable(True, True)    # pode redimensionar?

# 2) widgets vão aqui (próximos capítulos)

app.mainloop()               # 3) loop principal de eventos
```

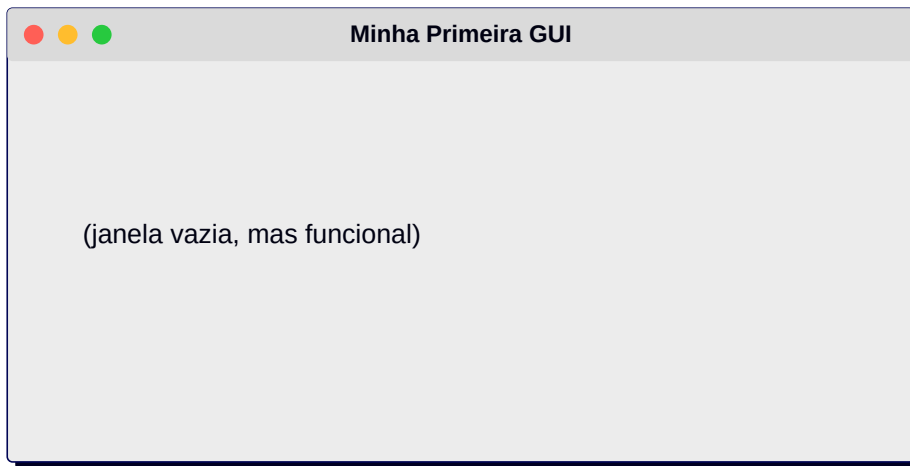


Figura 2.1 — Resultado do código acima.

2.2 Métodos da janela raiz

A janela `Tk()` oferece vários métodos úteis. Os mais comuns:

```
app.title('Texto')           # define o título
app.geometry('600x400+200+100') # tamanho + posição (x,y) na tela
app.minsize(300, 200)        # tamanho mínimo
app.maxsize(800, 600)        # tamanho máximo
app.resizable(False, True)    # bloqueia largura, libera altura
app.configure(bg='#202225')  # cor de fundo
app.iconbitmap('icone.ico')   # ícone (Windows)
app.attributes('-topmost', True) # sempre no topo
app.attributes('-fullscreen', True) # tela cheia
```

2.3 Encerrando a aplicação

Para fechar via código use `app.destroy()`. Para interceptar o clique no botão X da janela e perguntar antes de sair, use o protocolo `WM_DELETE_WINDOW`:

```
from tkinter import messagebox

def sair():
```

```
    if messagebox.askyesno('Sair', 'Deseja realmente sair?):  
        app.destroy()  
  
app.protocol('WM_DELETE_WINDOW', sair)
```

Nota: Sempre chame `mainloop()` apenas uma vez, e sempre no final do script.

Capítulo 3 — Widgets básicos: Label, Button, Entry

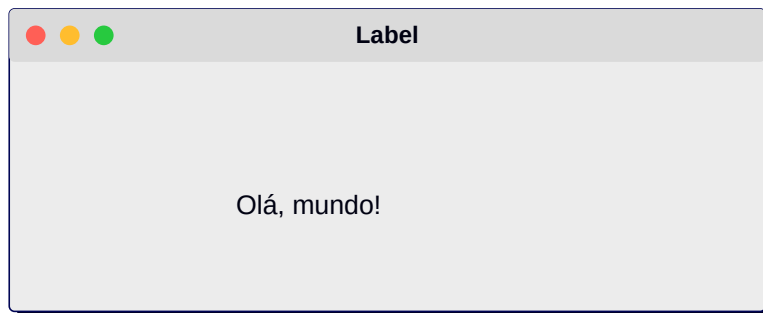
3.1 Label — exibindo texto e imagens

O **Label** é o widget mais simples: mostra texto estático (ou uma imagem). Você define o texto, a fonte, a cor e adiciona à janela com um gerenciador de layout.

```
import tkinter as tk
app = tk.Tk(); app.geometry('320x140')

lb = tk.Label(app, text='Olá, mundo!',
              font=('Segoe UI', 18, 'bold'),
              fg='#0B3D91', bg='#F4F6F8',
              padx=20, pady=10)
lb.pack(pady=30)

app.mainloop()
```

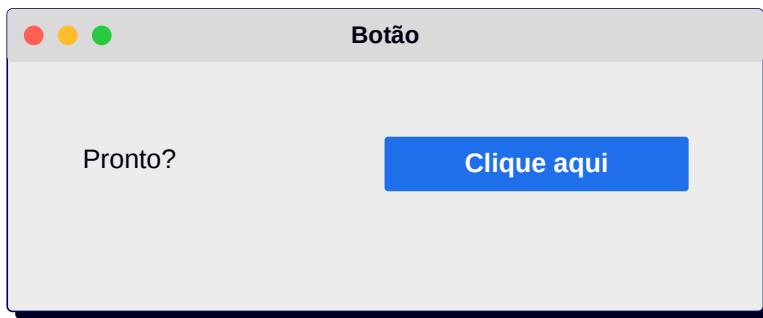


3.2 Button — disparando ações

Um **Button** executa uma função quando clicado. Essa função é passada no parâmetro `command` (sem parênteses!).

```
def clicou():
    print('Botão clicado!')

btn = tk.Button(app, text='Clique aqui',
                command=clicou,
                bg='#1F6FEB', fg='white',
                activebackground='#0B5ED7',
                font=('Segoe UI', 11, 'bold'),
                relief='flat', padx=20, pady=8, cursor='hand2')
btn.pack(pady=10)
```



3.3 Entry — entrada de texto

O **Entry** é um campo de uma linha. Para ler o valor digitado use `.get()`; para preencher, `.insert(0, 'texto')`; para limpar, `.delete(0, 'end')`.

```
nome = tk.Entry(app, font=('Segoe UI', 12), width=25)
nome.pack(pady=6)
nome.insert(0, 'Digite seu nome...')
```

```
def saudacao():
    print('Olá,', nome.get())
```

```
tk.Button(app, text='OK', command=saudacao).pack()
```



3.4 Resumo prático

Padrão geral: *criar widget* → *configurar opções* → *posicionar com pack/grid/place*. Você sempre passa a **janela ou frame pai** como primeiro argumento.

Nota: Prefira fontes legíveis como *Segoe UI*, *Calibri*, *Arial* ou *Helvetica*. Evite cores chapadas e duras — utilize tons suaves de cinza e azul para uma UI agradável.

Capítulo 4 — Gerenciadores de layout: pack, grid, place

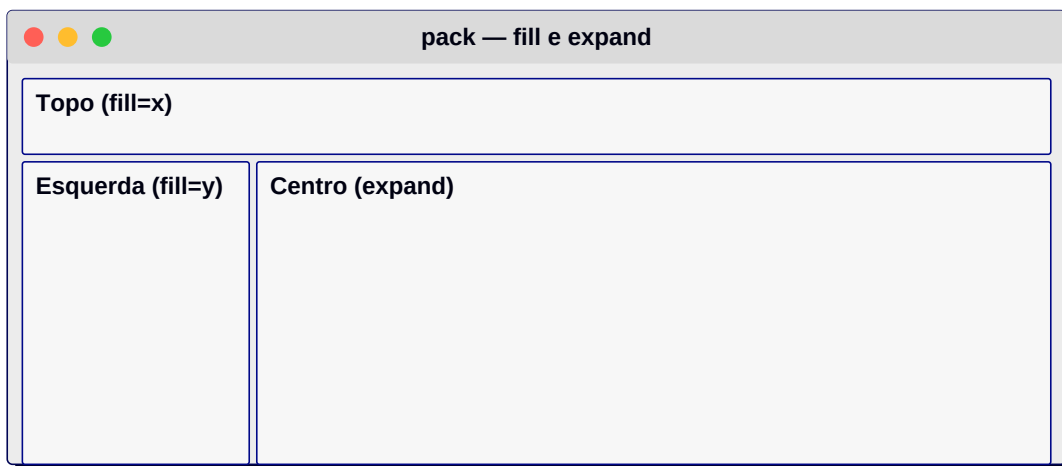
4.1 Visão geral

Tkinter oferece **três** gerenciadores de layout. Cada janela ou frame deve usar **apenas um** deles para evitar conflitos. São:

- **pack** — empilha widgets em uma direção (top, bottom, left, right).
- **grid** — posiciona em linhas e colunas (recomendado para formulários).
- **place** — coordenadas absolutas em pixels (uso pontual).

4.2 pack()

```
tk.Label(app, text='Topo', bg='#FCA').pack(side='top', fill='x')
tk.Label(app, text='Esquerda', bg='#AFC').pack(side='left', fill='y')
tk.Label(app, text='Centro', bg='#ACF').pack(expand=True, fill='both')
```



4.3 grid()

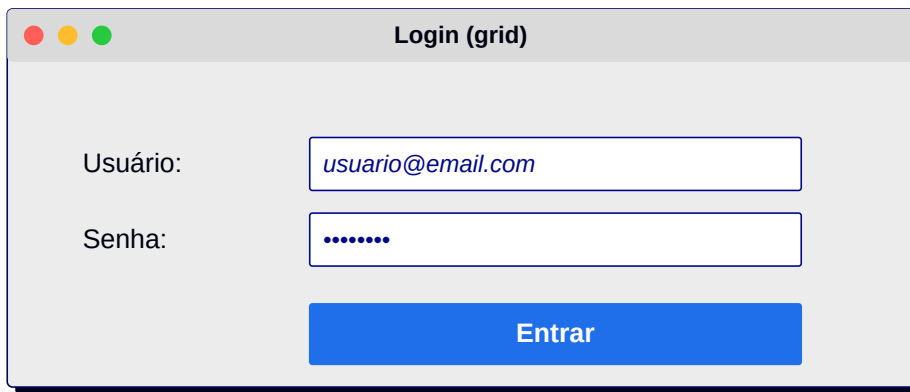
Pense em uma planilha. Cada widget vai em uma row e column; pode ocupar várias com colspan/rowspan.

```
tk.Label(app, text='Usuário:').grid(row=0, column=0, padx=8, pady=8, sticky='e')
tk.Entry(app, width=30).grid(row=0, column=1, padx=8, pady=8, sticky='we')

tk.Label(app, text='Senha:').grid(row=1, column=0, padx=8, pady=8, sticky='e')
tk.Entry(app, show='*', width=30).grid(row=1, column=1, padx=8, pady=8, sticky='we')

tk.Button(app, text='Entrar').grid(row=2, column=0, colspan=2, pady=10, sticky='we')

app.columnconfigure(1, weight=1) # expande coluna 1 ao redimensionar
```



4.4 place()

Coloca o widget em coordenadas exatas. Útil para **elementos sobrepostos**, imagens de fundo ou layouts livres.

```
img = tk.Label(app, bg='#0B3D91')
img.place(x=0, y=0, relwidth=1, relheight=1) # ocupa tudo

tk.Label(app, text='Sobre a imagem', bg='white').place(x=20, y=20)
```

4.5 Boas práticas

- Para janelas inteiras, use grid em frames internos.
- pack brilha para barras superiores/inferiores.
- Reserve place para casos especiais — ele não é responsivo por padrão.
- Use padx/pady para respirar; UIs apertadas cansam o olho.

Capítulo 5 — Eventos, comandos e binds

5.1 command vs bind

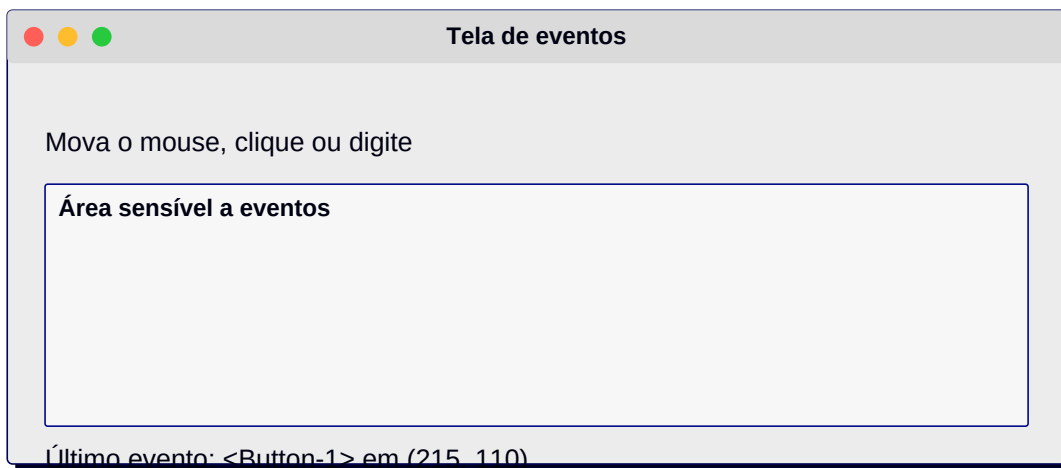
Botões aceitam o atalho `command=fn`. Para outros eventos (teclado, mouse, foco) usamos `widget.bind('<Evento>', funcao)`.

```
def ao_pressionar(evt):
    print('Tecla:', evt.keysym, '| char:', evt.char)

entry.bind('<KeyRelease>', ao_pressionar)
entry.bind('<Return>', lambda e: print('Enter!'))
app.bind('<Escape>', lambda e: app.destroy())
```

5.2 Eventos mais usados

`<Button-1>` clique esquerdo, `<Button-3>` direito, `<Double-Button-1>` duplo clique, `<Motion>` mouse movendo, `<Enter>/<Leave>` entrada/saída do mouse, `<FocusIn>/<FocusOut>`, `<Configure>` redimensionamento.



5.3 Passando argumentos via lambda

```
for i in range(5):
    tk.Button(app, text=f'Item {i}',
              command=lambda x=i: print('Cliquei', x)).pack()
```

Nota: O truque `x=i` no lambda evita o famoso bug de *late binding* em loops.

Capítulo 6 — Variáveis Tk (StringVar, IntVar, etc.)

6.1 Por que existem?

Variáveis Tk são objetos que **refletem** o valor de um widget. Quando você altera a variável, o widget atualiza; quando o widget muda, a variável também. São ideais para dois widgets compartilharem dados.

```
nome = tk.StringVar(value='Anônimo')

tk.Entry(app, textvariable=nome).pack()
tk.Label(app, textvariable=nome, font=('Segoe UI', 14, 'bold')).pack()

def upper():
    nome.set(nome.get().upper())
tk.Button(app, text='MAIÚSCULO', command=upper).pack()
```

6.2 IntVar, DoubleVar, BooleanVar

Mesmos métodos (get, set, trace_add) mas com tipos diferentes. Use com Checkbutton (BooleanVar), Scale (DoubleVar), Spinbox (IntVar), etc.

```
ligado = tk.BooleanVar(value=False)
tk.Checkbutton(app, text='Notificações', variable=ligado).pack()

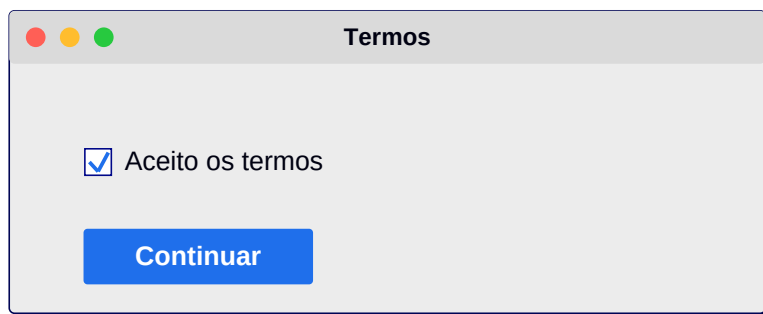
def mudou(*_):
    print('Agora está:', ligado.get())
ligado.trace_add('write', mudou)
```

Capítulo 7 — Checkbutton, Radiobutton, Scale e Spinbox

7.1 Checkbutton

Marca/desmarca uma opção; ligado a uma BooleanVar.

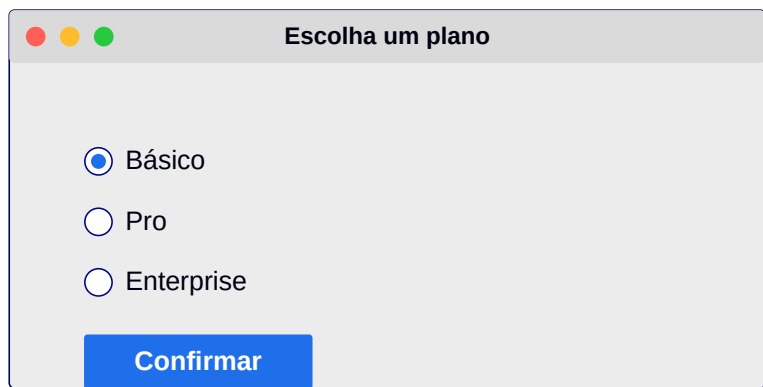
```
aceito = tk.BooleanVar()
tk.Checkbutton(app, text='Aceito os termos', variable=aceito,
               onvalue=True, offvalue=False).pack()
```



7.2 Radiobutton

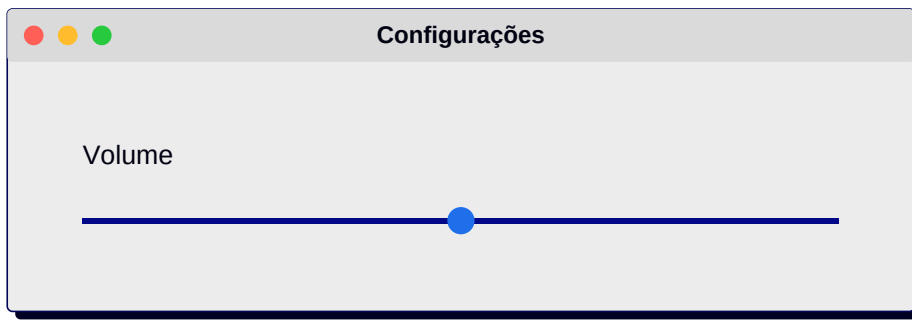
Permite escolher **uma** entre várias opções. Todos compartilham a mesma variável.

```
plano = tk.StringVar(value='basico')
for txt, val in [('Básico', 'basico'), ('Pro', 'pro'), ('Enterprise', 'ent')]:
    tk.Radiobutton(app, text=txt, value=val, variable=plano).pack(anchor='w')
```



7.3 Scale

```
vol = tk.IntVar(value=50)
tk.Scale(app, from_=0, to=100, orient='horizontal',
        variable=vol, label='Volume').pack(fill='x', padx=20)
```



7.4 Spinbox

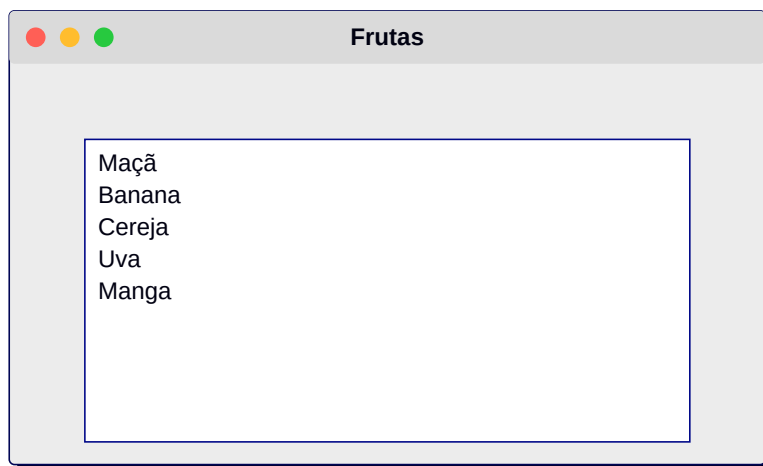
```
qtd = tk.IntVar(value=1)
tk.Spinbox(app, from_=1, to=99, textvariable=qtd, width=5).pack()
```


Capítulo 8 — Listbox, Combobox e Treeview (ttk)

8.1 Listbox

```
lb = tk.Listbox(app, height=6, selectmode='single')
for item in ['Maçã', 'Banana', 'Cereja', 'Uva']:
    lb.insert('end', item)
lb.pack(fill='both', expand=True)

def selecionou(_):
    print(lb.get(lb.curselection()))
lb.bind('<<ListboxSelect>>', selecionou)
```



8.2 Combobox (ttk)

```
from tkinter import ttk
cb = ttk.Combobox(app, values=['Brasil', 'Portugal', 'Angola'])
cb.set('Brasil')
cb.pack()
```

8.3 Treeview — tabelas

```
tv = ttk.Treeview(app, columns=('nome', 'idade'), show='headings')
tv.heading('nome', text='Nome'); tv.heading('idade', text='Idade')
tv.column('nome', width=200); tv.column('idade', width=80, anchor='center')
for n, i in [('Ana', 22), ('Bruno', 30), ('Carla', 27)]:
    tv.insert('', 'end', values=(n, i))
tv.pack(fill='both', expand=True)
```

Pessoas	
Nome	Idade
Ana	22
Bruno	30
Carla	27

Capítulo 9 — Text e Canvas — desenho e edição

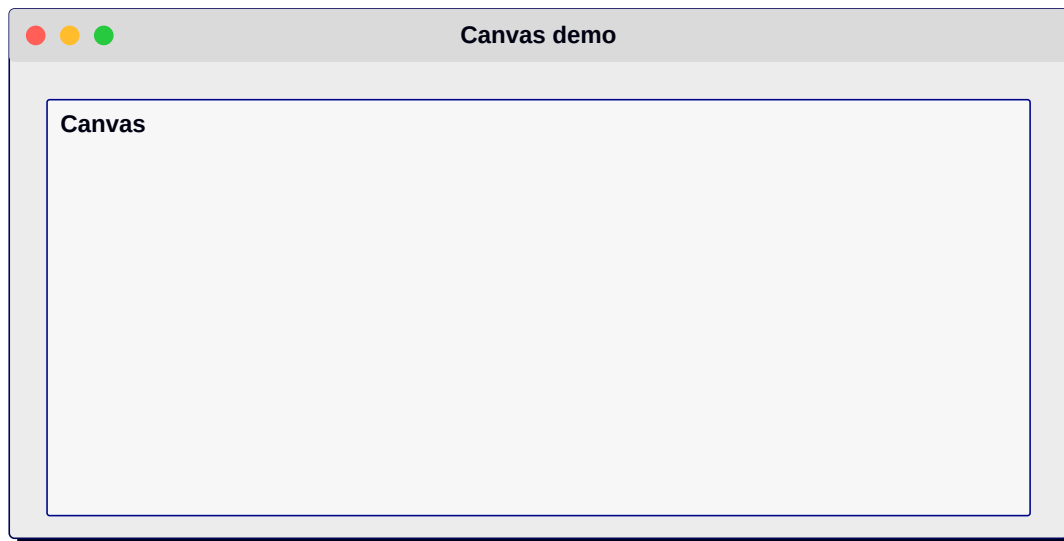
9.1 Text — múltiplas linhas

```
txt = tk.Text(app, wrap='word', height=10, font=('Consolas', 11))
txt.pack(fill='both', expand=True)
txt.insert('1.0', 'Olá, este é um editor multilinha.')
conteudo = txt.get('1.0', 'end-1c')
```

9.2 Canvas — gráficos vetoriais

O Canvas desenha linhas, retângulos, círculos, polígonos, textos e imagens. Cada objeto tem um **ID** retornado para manipulação.

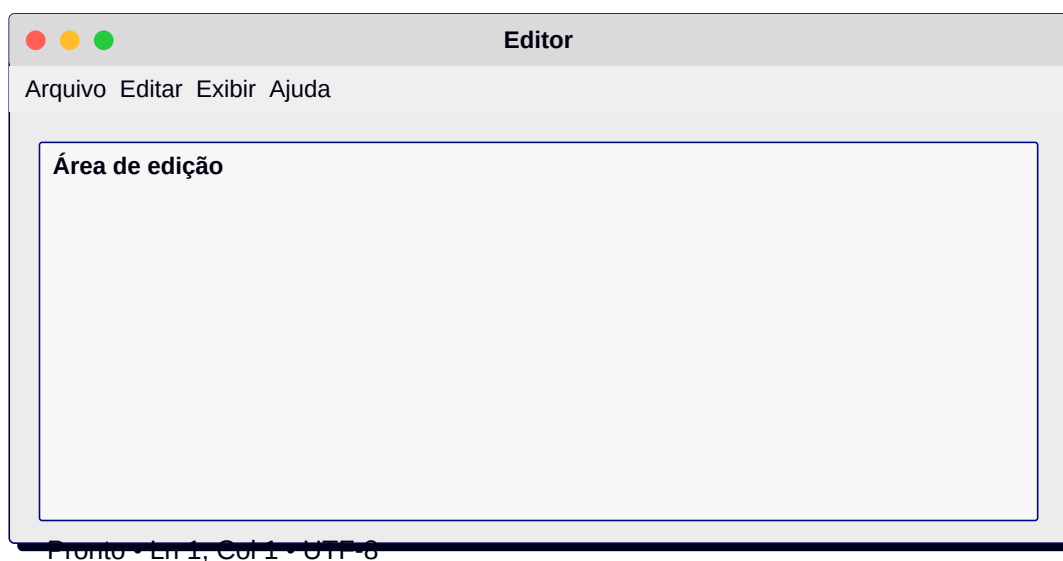
```
c = tk.Canvas(app, width=400, height=300, bg='white')
c.pack()
c.create_rectangle(20, 20, 200, 100, fill='#1F6FEB', outline='')
c.create_oval(220, 20, 380, 180, fill='#FFB020')
c.create_line(0, 250, 400, 250, width=3, fill='#444')
txt = c.create_text(200, 280, text='Canvas em ação', font=('Segoe UI', 14, 'bold'))
```



Capítulo 10 — Menus, Toolbar e StatusBar

10.1 Menu principal

```
menubar = tk.Menu(app)
arquivo = tk.Menu(menubar, tearoff=False)
arquivo.add_command(label='Novo', accelerator='Ctrl+N')
arquivo.add_command(label='Abrir...', accelerator='Ctrl+O')
arquivo.add_separator()
arquivo.add_command(label='Sair', command=app.destroy)
menubar.add_cascade(label='Arquivo', menu=arquivo)
app.config(menu=menubar)
```



10.2 Barra de status

```
status = tk.Label(app, text='Pronto', anchor='w', bd=1, relief='sunken')
status.pack(side='bottom', fill='x')
```

Capítulo 11 — Diálogos: messagebox, filedialog, colorchooser

11.1 messagebox

```
from tkinter import messagebox
messagebox.showinfo('Sucesso', 'Dados salvos!')
messagebox.showwarning('Atenção', 'Verifique os campos.')
messagebox.showerror('Erro', 'Falha na conexão.')
resposta = messagebox.askyesno('Confirmar', 'Excluir registro?')
```

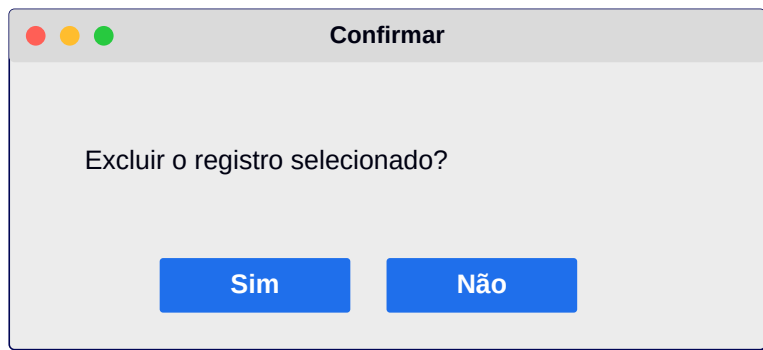
11.2 filedialog

```
from tkinter import filedialog
arq = filedialog.askopenfilename(filetypes=[('Imagens', '*.png *.jpg'), ('Tudo', '*..*')])
pasta = filedialog.askdirectory()
salvar = filedialog.asksaveasfilename(defaultextension='.txt')
```

11.3 colorchooser

```
from tkinter import colorchooser
cor = colorchooser.askcolor(title='Escolha a cor')[1]
```

Resumo visual



Capítulo 12 — Toplevel, modais e múltiplas janelas

12.1 Abrindo nova janela

```
def abrir_sobre():
    top = tk.Toplevel(app)
    top.title('Sobre'); top.geometry('320x180')
    tk.Label(top, text='Meu App v1.0').pack(pady=30)
    top.transient(app)      # acompanha a janela mãe
    top.grab_set()          # modal
    top.focus()
```

12.2 Comunicação entre janelas

Use variáveis Tk compartilhadas, atributos da janela mãe, ou retornos via callbacks.

Capítulo 13 — Frames, LabelFrame e PanedWindow

13.1 Frame

Container invisível usado para agrupar widgets e aplicar layouts independentes.

```
topo = tk.Frame(app, bg='#0B3D91', height=60); topo.pack(fill='x')
corpo = tk.Frame(app); corpo.pack(fill='both', expand=True)
```

13.2 LabelFrame

```
grupo = tk.LabelFrame(app, text='Endereço', padx=10, pady=10)
grupo.pack(fill='x', padx=10, pady=10)
```

13.3 PanedWindow

Permite redimensionar áreas arrastando o divisor.

Capítulo 14 — Estilos com ttk.Style e temas

14.1 Por que ttk?

Os widgets do módulo `tkinter.ttk` são renderizados com o tema nativo do SO e suportam **estilos** CSS-like via `ttk.Style`.

```
from tkinter import ttk
style = ttk.Style()
print(style.theme_names())      # ('clam', 'alt', 'default', 'classic', ...)
style.theme_use('clam')

style.configure('Accent.TButton', background='#1F6FEB', foreground='white',
               padding=10, font=('Segoe UI', 11, 'bold'))
style.map('Accent.TButton', background=[('active', '#0B5ED7')])

ttk.Button(app, text='Salvar', style='Accent.TButton').pack(padx=20, pady=20)
```


Capítulo 15 — Imagens com PhotoImage e Pillow

15.1 PhotoImage (PNG/GIF)

```
img = tk.PhotoImage(file='logo.png')
tk.Label(app, image=img).pack()
# ATENÇÃO: mantenha a referência (variável global ou self.img)
```

15.2 Pillow para JPG, redimensionar etc.

```
from PIL import Image, ImageTk
foto = Image.open('foto.jpg').resize((200, 200))
self.img = ImageTk.PhotoImage(foto)
tk.Label(app, image=self.img).pack()
```

Capítulo 16 — Threads, after() e tarefas longas

16.1 Por que não bloquear o mainloop?

Tarefas demoradas (download, processamento) congelam a interface se rodarem no thread principal. Solução: threading ou agendamento com after().

```
def passo():
    contador.set(contador.get() + 1)
    app.after(1000, passo)  # repete a cada 1s
app.after(1000, passo)
```

16.2 Thread para tarefa pesada

```
import threading, time
def baixar():
    time.sleep(5)
    app.after(0, lambda: status.set('Concluído'))
threading.Thread(target=baixar, daemon=True).start()
```

Capítulo 17 — Projeto: Bloco de Notas

17.1 Objetivo

Criar um editor de texto simples com menu Arquivo (Novo, Abrir, Salvar) e barra de status mostrando o tamanho do texto.

```
import tkinter as tk
from tkinter import filedialog, messagebox

app = tk.Tk(); app.title('Notas'); app.geometry('700x500')

texto = tk.Text(app, wrap='word', undo=True, font=('Consolas', 12))
texto.pack(fill='both', expand=True)

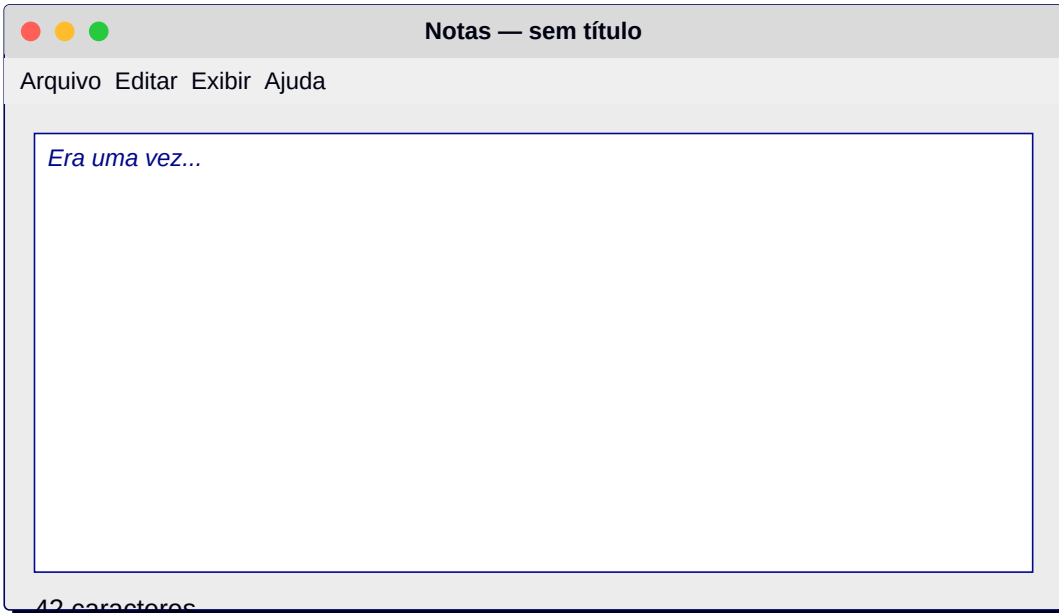
status = tk.Label(app, text='0 caracteres', anchor='w', bd=1, relief='sunken')
status.pack(side='bottom', fill='x')

def atualizar(*_):
    n = len(texto.get('1.0', 'end-1c'))
    status.config(text=f'{n} caracteres')
    texto.bind('<KeyRelease>', atualizar)

def novo(): texto.delete('1.0', 'end'); atualizar()
def abrir():
    arq = filedialog.askopenfilename(filetypes=[('TXT', '*.txt')])
    if arq:
        with open(arq, encoding='utf-8') as f:
            texto.delete('1.0', 'end'); texto.insert('1.0', f.read())
        atualizar()
def salvar():
    arq = filedialog.asksaveasfilename(defaulttextextension='.txt')
    if arq:
        with open(arq, 'w', encoding='utf-8') as f:
            f.write(texto.get('1.0', 'end-1c'))
        messagebox.showinfo('OK', 'Arquivo salvo!')

mb = tk.Menu(app); arq = tk.Menu(mb, tearoff=False)
arq.add_command(label='Novo', command=novo)
arq.add_command(label='Abrir...', command=abrir)
arq.add_command(label='Salvar...', command=salvar)
arq.add_separator(); arq.add_command(label='Sair', command=app.destroy)
mb.add_cascade(label='Arquivo', menu=arq); app.config(menu=mb)

app.mainloop()
```



Capítulo 18 — Projeto: Calculadora

18.1 Layout em grid

Visor no topo (Entry), 4 colunas de botões formando o teclado.

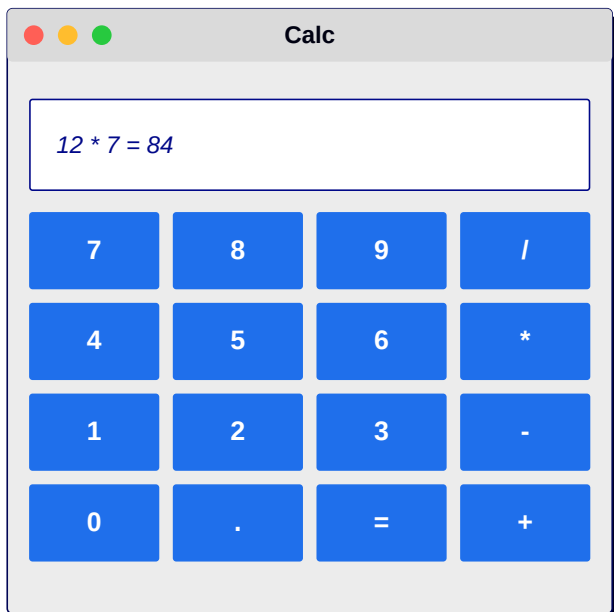
```
import tkinter as tk
app = tk.Tk(); app.title('Calc'); app.geometry('260x340')

visor = tk.Entry(app, font=('Segoe UI', 22), justify='right', bd=4)
visor.grid(row=0, column=0, columnspan=4, sticky='we', padx=4, pady=4)

botoes = [
    ('7',1,0),('8',1,1),('9',1,2),('/',1,3),
    ('4',2,0),('5',2,1),('6',2,2),('*',2,3),
    ('1',3,0),('2',3,1),('3',3,2),('-',3,3),
    ('0',4,0),('.',4,1),('=',4,2),('+',4,3),
]
def aperta(t):
    if t == '=':
        try: visor.insert('end', ' = ' + str(eval(visor.get())))
        except Exception: visor.insert('end', ' ERRO')
    else:
        visor.insert('end', t)

for t,r,c in botoes:
    tk.Button(app, text=t, font=('Segoe UI', 14), width=4, height=2,
              command=lambda x=t: aperta(x)).grid(row=r, column=c, padx=2, pady=2)

for i in range(4): app.columnconfigure(i, weight=1)
app.mainloop()
```



Nota: Usar `eval()` em uma calculadora real é arriscado. Em produção utilize um parser próprio.

Capítulo 19 — Projeto: To-Do List com SQLite

19.1 Arquitetura

Banco em todo.db, tabela tarefas(id, texto, feito). UI lista as tarefas e oferece adicionar/concluir/excluir.

19.2 Código

```
import sqlite3, tkinter as tk
from tkinter import ttk, messagebox

con = sqlite3.connect('todo.db')
con.execute('CREATE TABLE IF NOT EXISTS tarefas(id INTEGER PRIMARY KEY, texto TEXT, feito INTEGER D

app = tk.Tk(); app.title('To-Do'); app.geometry('500x400')

entrada = ttk.Entry(app, font=('Segoe UI', 12))
entrada.pack(fill='x', padx=10, pady=10)

tv = ttk.Treeview(app, columns=('texto', 'feito'), show='headings')
tv.heading('texto', text='Tarefa'); tv.heading('feito', text='Feito')
tv.column('feito', width=60, anchor='center')
tv.pack(fill='both', expand=True, padx=10)

def carregar():
    tv.delete(*tv.get_children())
    for i, t, f in con.execute('SELECT * FROM tarefas'):
        tv.insert('', 'end', iid=i, values=(t, '✓' if f else ''))

def add():
    t = entrada.get().strip()
    if not t: return
    con.execute('INSERT INTO tarefas(texto) VALUES (?)', (t,))
    con.commit(); entrada.delete(0, 'end'); carregar()

def concluir():
    s = tv.selection()
    if s:
        con.execute('UPDATE tarefas SET feito=1 WHERE id=?', (s[0],))
        con.commit(); carregar()

def excluir():
    s = tv.selection()
    if s:
        con.execute('DELETE FROM tarefas WHERE id=?', (s[0],))
        con.commit(); carregar()

barra = tk.Frame(app); barra.pack(pady=10)
ttk.Button(barra, text='Adicionar', command=add).pack(side='left', padx=4)
ttk.Button(barra, text='Concluir', command=concluir).pack(side='left', padx=4)
ttk.Button(barra, text='Excluir', command=excluir).pack(side='left', padx=4)

carregar(); app.mainloop()
```

To-Do List

Nova tarefa...

Estudar Tkinter

Lavar a louça

Comprar pão

Ler 30 páginas

✓

Adicionar

Concluir

Excluir

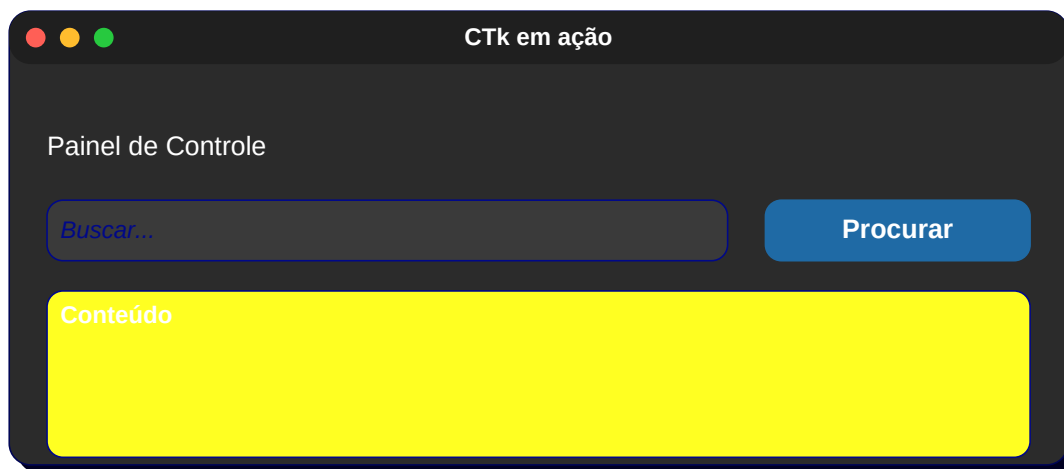
Parte II — CustomTkinter

CustomTkinter (CTk) é uma biblioteca moderna construída sobre o Tkinter. Oferece widgets com visual **arredondado e plano**, suporte a **tema escuro**, **cores customizáveis** e melhor aparência em alta resolução. A API é muito parecida com a do Tkinter, então praticamente tudo o que você aprendeu na Parte I se aplica aqui.

Capítulo 20 — Apresentando o CustomTkinter

20.1 Por que usar?

- Visual moderno (estilo de apps web).
- Tema escuro/claro nativo.
- Widgets com *scaling* automático para telas 4K.
- Compatível com Windows, macOS e Linux.



Capítulo 21 — Instalação e primeira CTK()

21.1 Instalação

```
pip install customtkinter pillow
```

21.2 Hello world

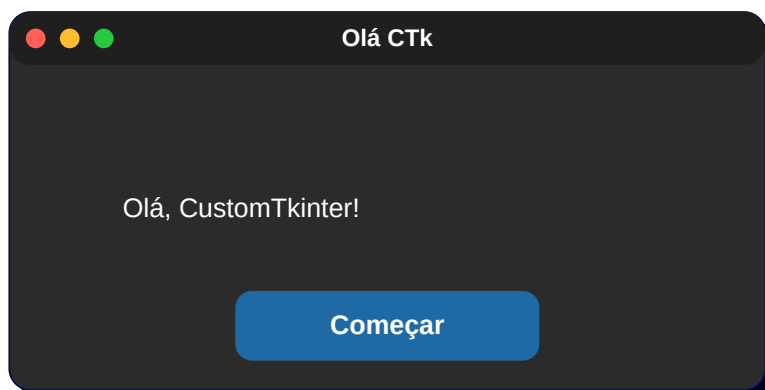
```
import customtkinter as ctk

ctk.set_appearance_mode('dark')      # 'light', 'dark', 'system'
ctk.set_default_color_theme('blue')  # 'blue', 'green', 'dark-blue'

app = ctk.CTk()
app.title('Olá CTK'); app.geometry('420x220')

ctk.CTkLabel(app, text='Olá, CustomTkinter!',
              font=ctk.CTkFont(size=20, weight='bold')).pack(pady=30)
ctk.CTkButton(app, text='Começar').pack(pady=10)

app.mainloop()
```



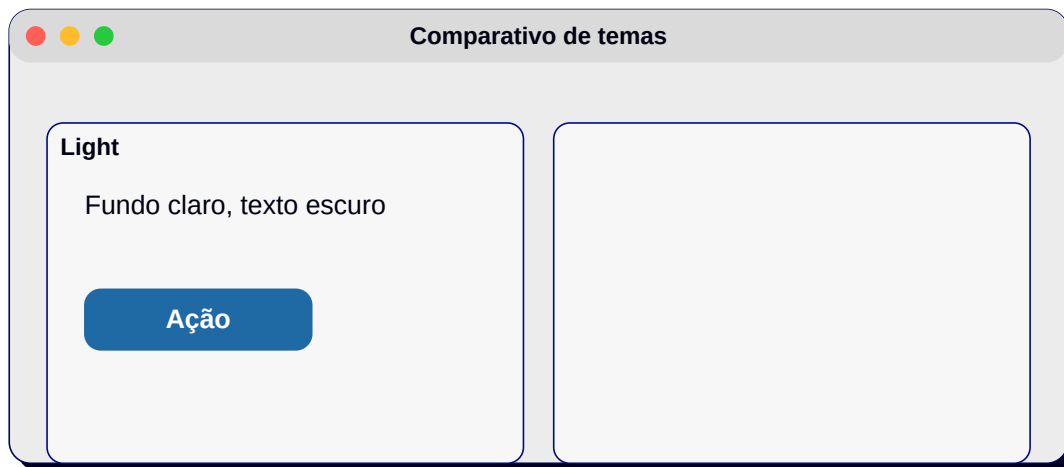
Capítulo 22 — Aparência: light, dark, system e color themes

22.1 Modo de aparência

```
ctk.set_appearance_mode('system') # segue o SO
ctk.set_appearance_mode('dark')
ctk.set_appearance_mode('light')
```

22.2 Trocar tema em tempo de execução

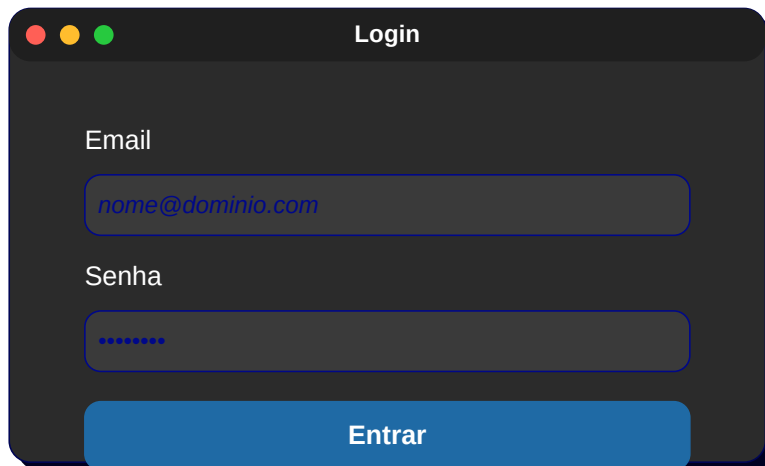
```
def alternar():
    novo = 'dark' if ctk.get_appearance_mode() == 'Light' else 'light'
    ctk.set_appearance_mode(novo)
ctk.CTkButton(app, text='Trocar tema', command=alternar).pack()
```



Capítulo 23 — Widgets CTK: Button, Label, Entry, Frame

23.1 CTKLabel e CTKButton

```
ctk.CTkLabel(app, text='Email').pack()  
ctk.CTkEntry(app, placeholder_text='nome@dominio.com', width=260).pack(pady=6)  
ctk.CTkButton(app, text='Entrar', corner_radius=20,  
              fg_color='#1F6AA5', hover_color='#144870').pack(pady=10)
```



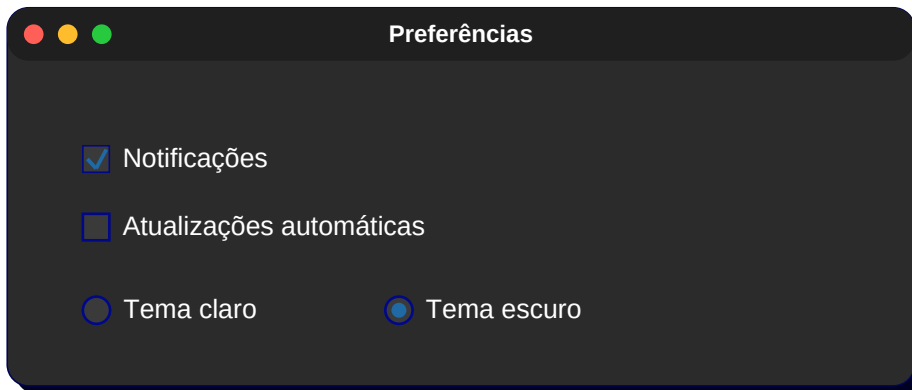
23.2 CTKFrame

```
card = ctk.CTkFrame(app, corner_radius=14)  
card.pack(padx=20, pady=20, fill='both', expand=True)
```

Capítulo 24 — CTkSwitch, CTkCheckBox, CTkRadioButton

24.1 Switch (interruptor)

```
sw = ctk.CTkSwitch(app, text='Notificações', onvalue=1, offvalue=0)
sw.pack(pady=10); sw.select() # já liga
```



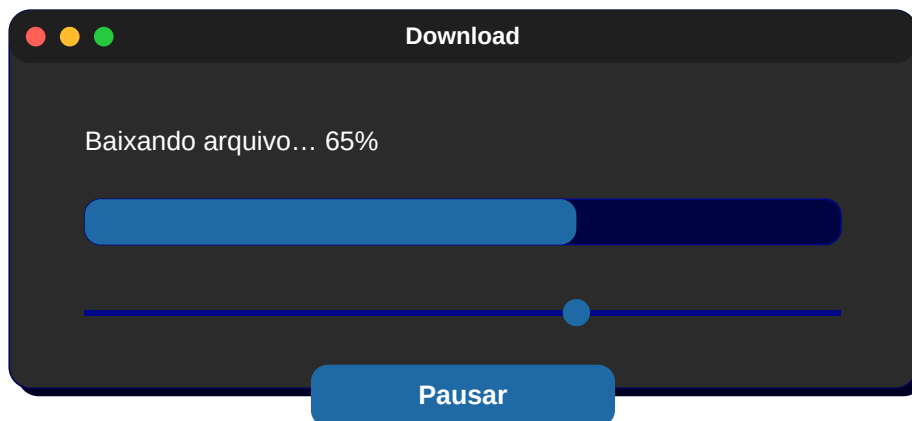
Capítulo 25 — CtkSlider, CtkProgressBar, CtkSegmentedButton

25.1 Slider

```
ctk.CTkSlider(app, from_=0, to=100, command=lambda v: print(v)).pack(fill='x', padx=20)
```

25.2 ProgressBar

```
pb = ctk.CTkProgressBar(app); pb.pack(fill='x', padx=20, pady=20)
pb.set(0.65)
```



25.3 SegmentedButton

```
seg = ctk.CTkSegmentedButton(app, values=['Dia', 'Semana', 'Mês'])
seg.set('Semana'); seg.pack(pady=10)
```

Capítulo 26 — CTkOptionMenu, CTkComboBox e CTkTextbox

26.1 OptionMenu

```
ctk.CTkOptionMenu(app, values=['Pequeno', 'Médio', 'Grande']).pack(pady=8)
```

26.2 ComboBox

```
ctk.CTkComboBox(app, values=['Python', 'C', 'Java', 'Go']).pack(pady=8)
```

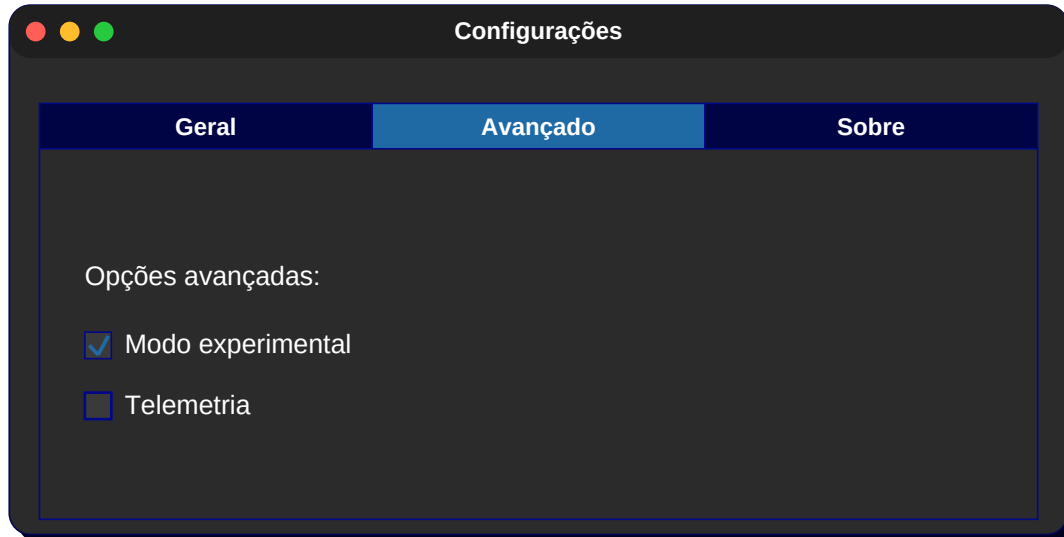
26.3 Textbox (Text)

```
tb = ctk.CTkTextbox(app, width=400, height=160)  
tb.pack(padx=20, pady=20); tb.insert('1.0', 'Escreva aqui...')
```

Capítulo 27 — CtkTabview, CtkScrollableFrame

27.1 Tabview

```
tabs = ctk.CtkTabview(app); tabs.pack(fill='both', expand=True)
tabs.add('Geral'); tabs.add('Avançado'); tabs.add('Sobre')
ctk.CtkLabel(tabs.tab('Geral'), text='Conteúdo Geral').pack(pady=20)
```



27.2 ScrollableFrame

```
sf = ctk.CtkScrollableFrame(app, label_text='Itens')
sf.pack(fill='both', expand=True, padx=10, pady=10)
for i in range(40):
    ctk.CtkLabel(sf, text=f'Linha {i}').pack(anchor='w', padx=10)
```


Capítulo 28 — CTkImage, CTkFont e ícones

28.1 CTkImage

Usa Pillow internamente; responde ao DPI automaticamente.

```
from PIL import Image
img = ctk.CTkImage(light_image=Image.open('logo_light.png'),
                  dark_image=Image.open('logo_dark.png'),
                  size=(64,64))
ctk.CTkLabel(app, image=img, text='').pack()
```

28.2 CTkFont

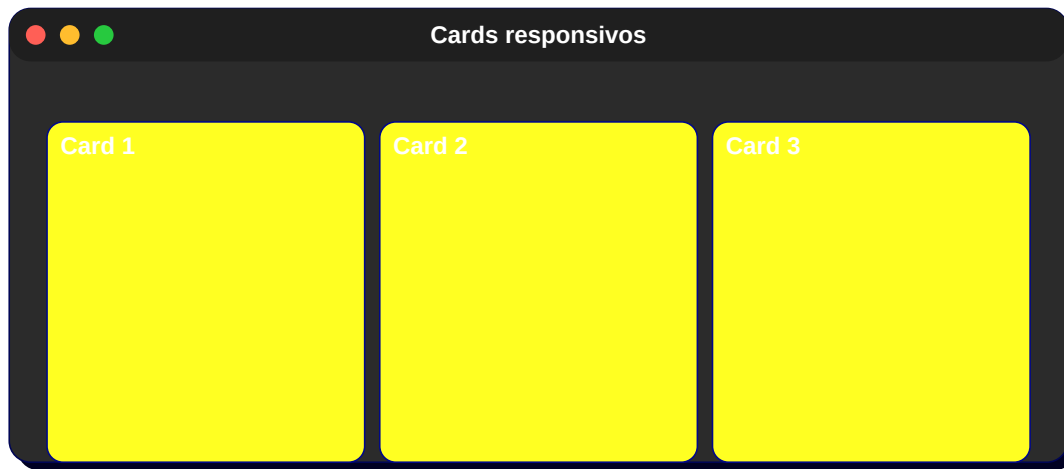
```
titulo = ctk.CTkFont(family='Segoe UI', size=22, weight='bold')
ctk.CTkLabel(app, text='Painel', font=titulo).pack()
```

Capítulo 29 — Layouts responsivos com grid

29.1 Grid + weight

```
app.grid_columnconfigure((0,1,2), weight=1)
app.grid_rowconfigure(0, weight=1)

for i in range(3):
    card = ctk.CTkFrame(app)
    card.grid(row=0, column=i, padx=10, pady=10, sticky='nsew')
    ctk.CTkLabel(card, text=f'Card {i+1}').pack(pady=20)
```



Capítulo 30 — Projeto: Dashboard moderno

30.1 Layout: sidebar + conteúdo

Uma sidebar à esquerda com botões de navegação e uma área principal com cards de métricas e gráficos. O exemplo abaixo monta a estrutura básica.

```
import customtkinter as ctk
ctk.set_appearance_mode('dark'); ctk.set_default_color_theme('blue')

app = ctk.CTk(); app.title('Dashboard'); app.geometry('960x600')
app.grid_columnconfigure(1, weight=1); app.grid_rowconfigure(0, weight=1)

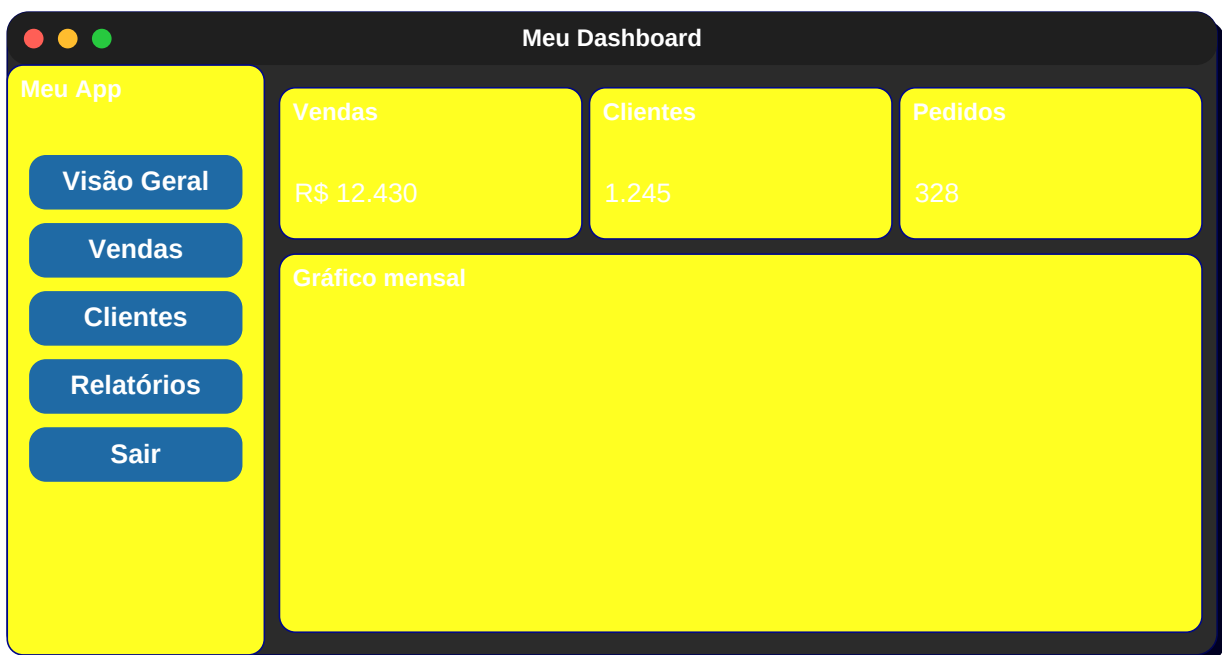
side = ctk.CTkFrame(app, width=200, corner_radius=0); side.grid(row=0, column=0, sticky='ns')
ctk.CTkLabel(side, text='Meu App', font=ctk.CTkFont(size=18, weight='bold')).pack(pady=20)
for nome in ['Visão Geral', 'Vendas', 'Clientes', 'Relatórios', 'Sair']:
    ctk.CTkButton(side, text=nome, anchor='w', fg_color='transparent',
                  hover_color='#1F2937', width=180).pack(pady=4, padx=10)

main = ctk.CTkFrame(app); main.grid(row=0, column=1, sticky='nsew', padx=10, pady=10)
main.grid_columnconfigure((0,1,2), weight=1)

for i, (titulo, valor) in enumerate([('Vendas', 'R$ 12.430'),
                                     ('Clientes', '1.245'),
                                     ('Pedidos', '328')]):
    card = ctk.CTkFrame(main, height=110, corner_radius=14)
    card.grid(row=0, column=i, sticky='nsew', padx=8, pady=8)
    ctk.CTkLabel(card, text=titulo, text_color='gray70').pack(pady=(14,4))
    ctk.CTkLabel(card, text=valor, font=ctk.CTkFont(size=22, weight='bold')).pack()

grafico = ctk.CTkFrame(main, corner_radius=14)
grafico.grid(row=1, column=0, columnspan=3, sticky='nsew', padx=8, pady=8)
ctk.CTkLabel(grafico, text='[Gráfico mensal]').pack(pady=80)

app.mainloop()
```



Capítulo 31 — Projeto: Login + Cadastro

31.1 Tela única com dois frames

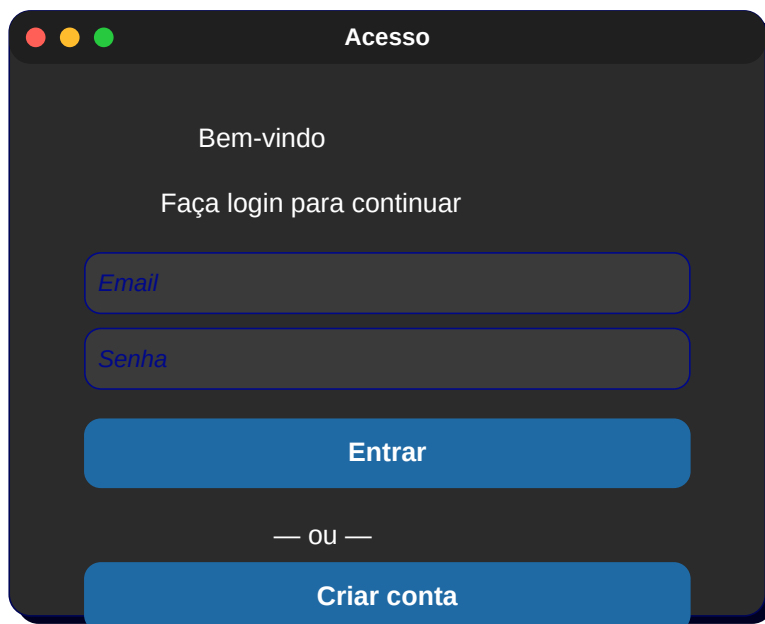
```
import customtkinter as ctk
ctk.set_appearance_mode('dark')

app = ctk.CTk(); app.title('Acesso'); app.geometry('420x520')

ctk.CTkLabel(app, text='Bem-vindo', font=ctk.CTkFont(size=24, weight='bold')).pack(pady=20)
ctk.CTkLabel(app, text='Faça login para continuar', text_color='gray70').pack()

ctk.CTkEntry(app, placeholder_text='Email', width=300).pack(pady=10)
ctk.CTkEntry(app, placeholder_text='Senha', show='*', width=300).pack(pady=10)
ctk.CTkButton(app, text='Entrar', width=300, height=42, corner_radius=20).pack(pady=20)

ctk.CTkLabel(app, text='— ou —', text_color='gray60').pack()
ctk.CTkButton(app, text='Criar conta', width=300, fg_color='transparent',
              border_width=1, hover_color='#1F2937').pack(pady=20)
app.mainloop()
```



Capítulo 32 — Empacotando seu app (PyInstaller)

32.1 Gerando .exe

```
pip install pyinstaller
pyinstaller --noconsole --onefile --icon=app.ico minha_app.py
# saída em dist/minha_app.exe
```

32.2 Incluindo assets

```
pyinstaller --noconsole --add-data 'assets;assets' minha_app.py
```

32.3 Customtkinter requer cuidado extra

Para CTK, é comum usar o spec file e incluir os JSONs de tema:

```
# customtkinter precisa de --collect-all
pyinstaller --noconsole --onefile --collect-all customtkinter minha_app.py
```

Apêndice A — Referência Rápida

A.1 Eventos comuns

Evento	Descrição
<Button-1>	Clique esquerdo
<Button-3>	Clique direito
<Double-Button-1>	Duplo clique
<Motion>	Mouse se move
<Enter>	Mouse entrou no widget
<Leave>	Mouse saiu
<Key>	Qualquer tecla
<Return>	Enter
<Escape>	Esc
<FocusIn>	Recebeu foco
<Configure>	Janela redimensionada

A.2 Opções de fonte mais comuns

('Segoe UI', 11), ('Segoe UI', 12, 'bold'), ('Consolas', 11), ('Arial', 14, 'italic').

A.3 Cheat-sheet pack / grid / place

```
# pack
widget.pack(side='top' | 'bottom' | 'left' | 'right',
            fill='x' | 'y' | 'both', expand=True/False,
            padx=N, pady=N, anchor='nw' | 'n' | ...)

# grid
widget.grid(row=R, column=C, rowspan=N, columnspan=N,
            sticky='nsew', padx=N, pady=N)
parent.rowconfigure(R, weight=1)
parent.columnconfigure(C, weight=1)

# place
widget.place(x=X, y=Y, relx=0..1, rely=0..1,
            width=N, height=N, relwidth=0..1, relheight=0..1,
            anchor='nw' | 'center' | ...)
```

A.4 Mapa Ctk ↔ Tk

Tkinter	CustomTkinter
Tk()	ctk.CTk()
Toplevel	ctk.CTkToplevel
Frame	ctk.CTkFrame
Label	ctk.CTkLabel

Button	ctk.CTkButton
Entry	ctk.CTkEntry
Checkbutton	ctk.CTkCheckBox / ctk.CTkSwitch
Radiobutton	ctk.CTkRadioButton
Scale	ctk.CTkSlider
Progressbar (ttk)	ctk.CTkProgressBar
Combobox (ttk)	ctk.CTkComboBox / CTkOptionMenu
Text	ctk.CTkTextbox
Notebook (ttk)	ctk.CTkTabview
Canvas com scroll	ctk.CTkScrollableFrame

A.5 Encerramento

Você concluiu a apostila! Continue praticando construindo pequenos projetos (relógio, agenda, conversor de moedas, leitor de RSS, etc.) e estudando a documentação oficial:

- <https://docs.python.org/3/library/tkinter.html>
- <https://customtkinter.tomschimansky.com>

Bons estudos e excelentes interfaces! — Edição 2026

Apêndice B — Galeria de Telas Comentadas

Para fixar tudo o que aprendemos, esta galeria reúne **20 telas extras** com código, descrição passo a passo e mockup desenhado. Cada exemplo aborda uma combinação diferente de widgets e layouts para inspirar seus próprios projetos.

Formulário de contato

Campos Nome/Email/Mensagem em Tk. Use grid com sticky='we' e columnconfigure(1, weight=1).

```
frm = tk.Frame(app); frm.pack(padx=20, pady=20, fill='x')
tk.Label(frm, text='Nome').grid(row=0,column=0,sticky='e',padx=4,pady=4)
tk.Entry(frm).grid(row=0,column=1,sticky='we',pady=4)
tk.Label(frm, text='Email').grid(row=1,column=0,sticky='e',padx=4,pady=4)
tk.Entry(frm).grid(row=1,column=1,sticky='we',pady=4)
tk.Label(frm, text='Mensagem').grid(row=2,column=0,sticky='ne',padx=4,pady=4)
tk.Text(frm, height=6).grid(row=2,column=1,sticky='we',pady=4)
tk.Button(frm, text='Enviar').grid(row=3,column=1,sticky='e',pady=8)
frm.columnconfigure(1, weight=1)
```



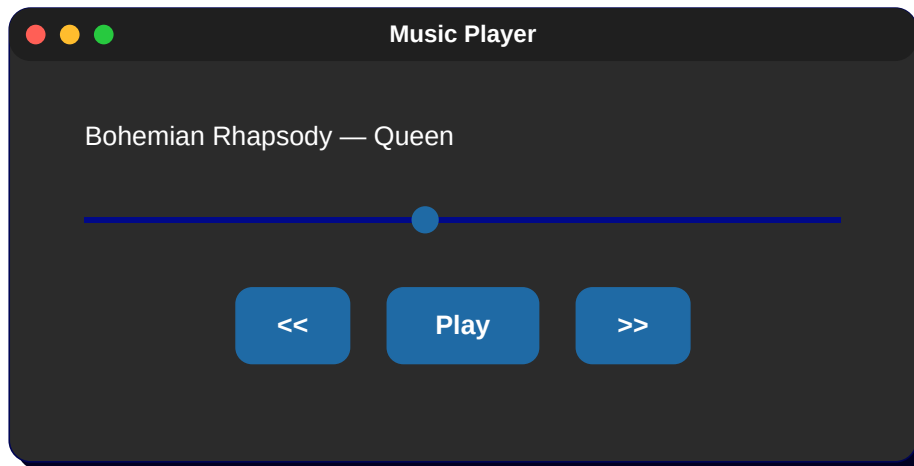
The screenshot shows a Tkinter window titled "Contato" with a light gray background. It contains three input fields and a button. The first field is labeled "Nome" and contains the text "Maria Souza". The second field is labeled "Email" and contains the text "maria@exemplo.com". The third field is labeled "Mensagem" and contains the text "Olá, gostaria de saber...". A blue button labeled "Enviar" is located at the bottom right of the form.

Nome	Maria Souza
Email	maria@exemplo.com
Mensagem	Olá, gostaria de saber...
Enviar	

Player de música (mock)

Slider de progresso, botões de play/pause/next, label com nome da faixa.

```
top = tk.Frame(app); top.pack(pady=20)
tk.Label(top, text='Now playing: Bohemian Rhapsody', font=('Segoe UI',12,'bold')).pack()
prog = ttk.Scale(app, from_=0, to=100); prog.pack(fill='x', padx=40, pady=10); prog.set(45)
bar = tk.Frame(app); bar.pack(pady=10)
for s in ('■', '■', '■'):
    tk.Button(bar, text=s, font=('Segoe UI',16), width=3).pack(side='left', padx=6)
```



Galeria em grid 3x3

Frames simulando miniaturas com border-radius.

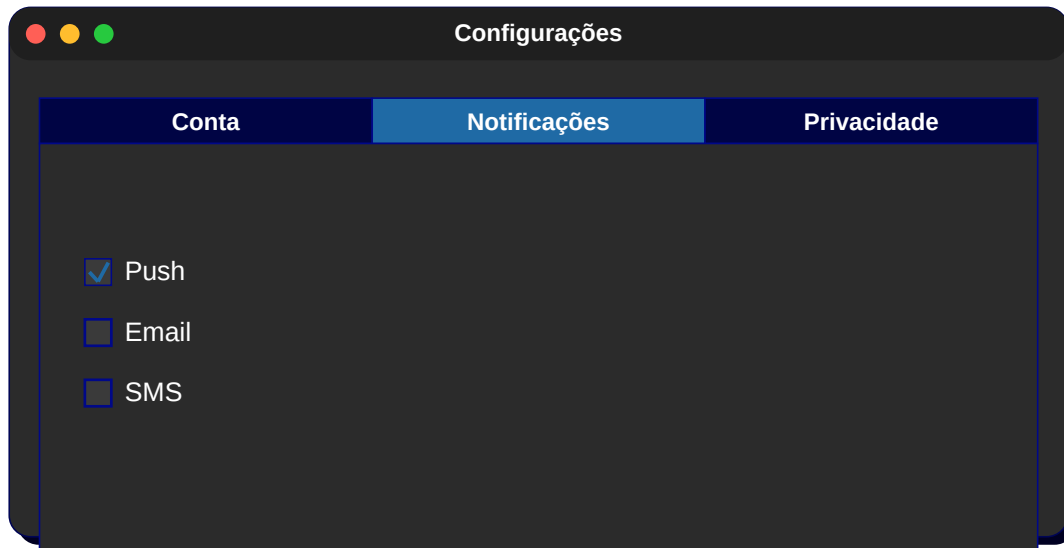
```
gal = ctk.CTkFrame(app); gal.pack(fill='both', expand=True, padx=10, pady=10)
for r in range(3):
    for c in range(3):
        ctk.CTkFrame(gal, width=120, height=80, corner_radius=10).grid(row=r, column=c, padx=8, pady=8)
```



Configurações com Tabview

Três abas: Conta, Notificações, Privacidade. Em cada uma, switches e entradas.

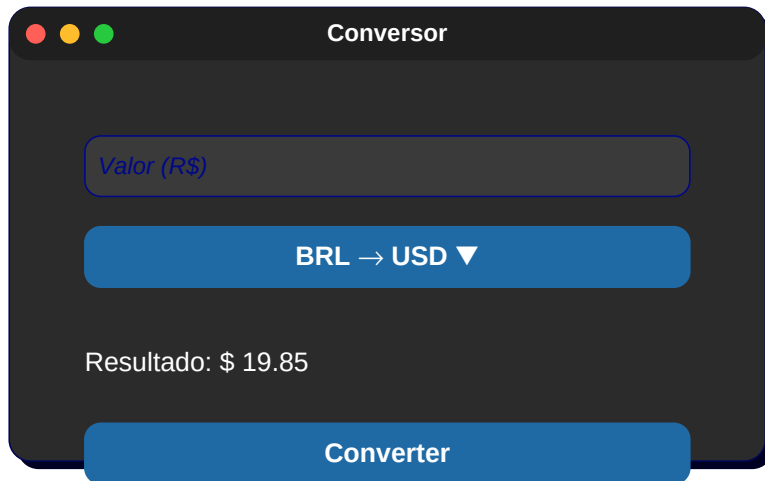
```
tabs = ctk.CTkTabview(app); tabs.pack(fill='both', expand=True, padx=10, pady=10)
for n in ['Conta', 'Notificações', 'Privacidade']: tabs.add(n)
ctk.CTkSwitch(tabs.tab('Notificações'), text='Push').pack(pady=10, anchor='w', padx=10)
ctk.CTkSwitch(tabs.tab('Notificações'), text='Email').pack(pady=10, anchor='w', padx=10)
```



Conversor de moedas

Entrada numérica + OptionMenu de moedas + label resultado.

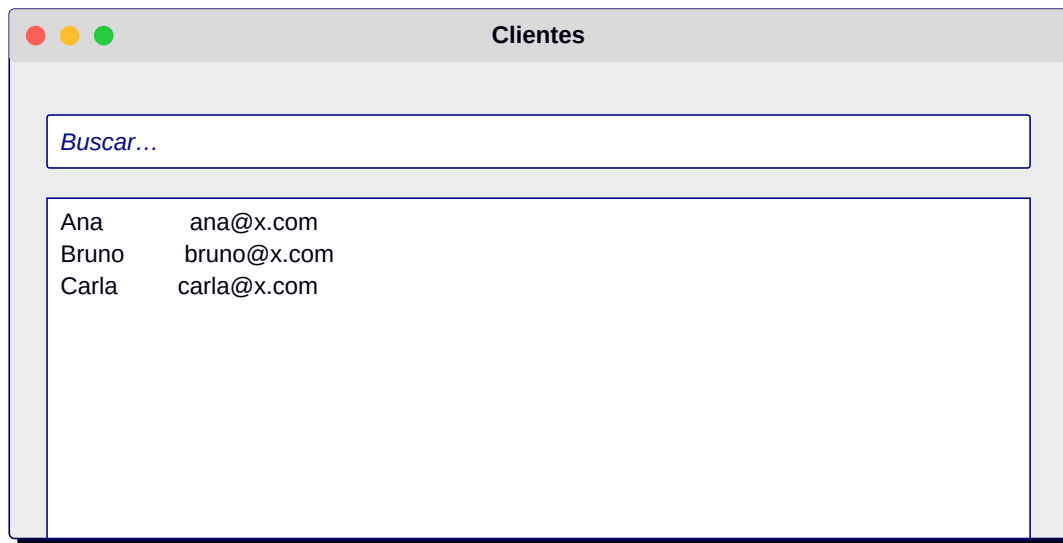
```
v = ctk.CTkEntry(app, placeholder_text='Valor'); v.pack(pady=8)
ctk.CTkOptionMenu(app, values=['BRL→USD', 'USD→BRL', 'BRL→EUR', 'EUR→BRL']).pack(pady=4)
res = ctk.CTkLabel(app, text='Resultado: -', font=ctk.CTkFont(size=18, weight='bold'))
res.pack(pady=20)
```



Pesquisa com Treeview

Campo de busca filtra a tabela em tempo real.

```
import tkinter.ttk as ttk
e = tk.Entry(app); e.pack(fill='x', padx=10, pady=6)
tv = ttk.Treeview(app, columns=('nome', 'email'), show='headings')
tv.heading('nome', text='Nome'); tv.heading('email', text='Email')
tv.pack(fill='both', expand=True)
dados = [('Ana', 'ana@x.com'), ('Bruno', 'bruno@x.com'), ('Carla', 'carla@x.com')]
def filtrar(*_):
    q = e.get().lower()
    tv.delete(*tv.get_children())
    for n,em in dados:
        if q in n.lower() or q in em.lower(): tv.insert('', 'end', values=(n,em))
e.bind('<KeyRelease>', filtrar); filtrar()
```



Wizard de 3 passos

Toplevel ou Frame trocando conteúdo conforme botão Próximo.

```
from tkinter import ttk
passo = tk.IntVar(value=1)
container = tk.Frame(app); container.pack(fill='both', expand=True)
lbl = tk.Label(container, text='Passo 1: dados pessoais', font=('Segoe UI',14))
lbl.pack(pady=40)
def proximo():
    p = passo.get()+1
    if p>3: app.destroy()
    else: passo.set(p); lbl.config(text=f'Passo {p}')
tk.Button(app, text='Próximo →', command=proximo).pack(pady=10)
```

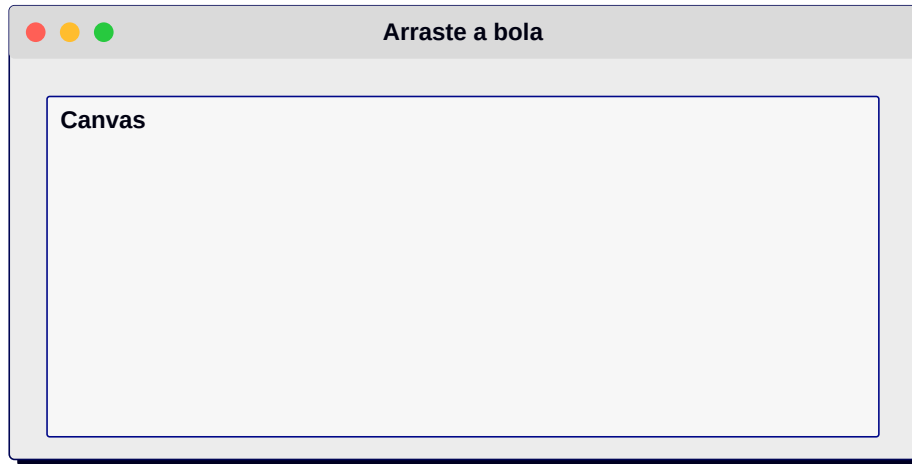


The screenshot shows a Tkinter window titled "Wizard — Passo 2 de 3". Inside the window, there is a frame titled "Passo 2 — Endereço". This frame contains two text input fields: the first is labeled "Rua..." and the second is labeled "Cidade...". At the bottom of the window, there are two buttons: "← Voltar" on the left and "Próximo →" on the right.

Drag-and-drop simples (Canvas)

Mover formas com o mouse usando bind ''.

```
c = tk.Canvas(app, width=400, height=300, bg='white'); c.pack()  
obj = c.create_oval(100,100,160,160, fill='#1F6FEB', outline='')  
def arrasta(e):  
    c.coords(obj, e.x-30, e.y-30, e.x+30, e.y+30)  
c.bind('<B1-Motion>', arrasta)
```



Cronômetro

Usa `after()` para atualizar a cada 100 ms.

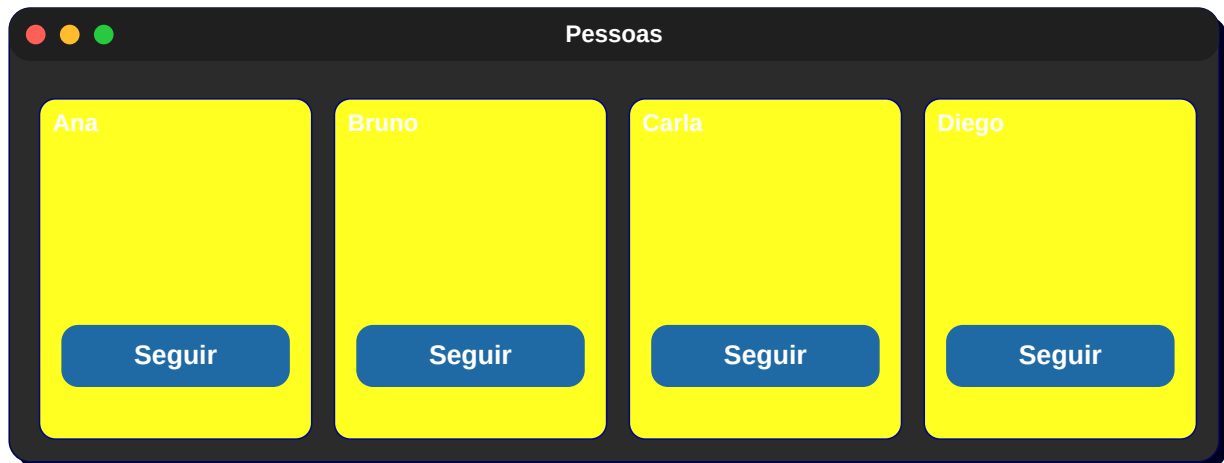
```
import time
ini = [None]
lbl = tk.Label(app, text='00:00.0', font=('Consolas',32)); lbl.pack(pady=20)
def tick():
    if ini[0] is not None:
        t = time.time()-ini[0]
        m,s = divmod(t,60); lbl.config(text=f'{int(m):02d}:{s:04.1f}')
        app.after(100, tick)
def start(): ini[0]=time.time()
def stop(): ini[0]=None
tk.Button(app, text='Iniciar', command=start).pack(side='left',padx=10)
tk.Button(app, text='Parar', command=stop).pack(side='left',padx=10)
tick()
```



Galeria de avatares + ações

Cards horizontais com imagem (mock), nome e botão Seguir.

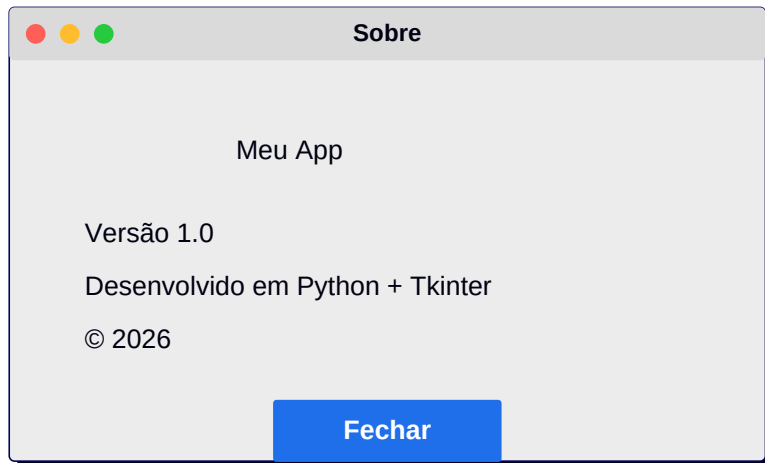
```
row = ctk.CTkFrame(app); row.pack(fill='x', padx=10, pady=10)
for nome in ['Ana', 'Bruno', 'Carla', 'Diego']:
    c = ctk.CTkFrame(row, corner_radius=12); c.pack(side='left', padx=8)
    ctk.CTkLabel(c, text='■', font=ctk.CTkFont(size=32)).pack(pady=10, padx=20)
    ctk.CTkLabel(c, text=nome).pack()
    ctk.CTkButton(c, text='Seguir', width=80).pack(pady=8)
```



Tela 'Sobre'

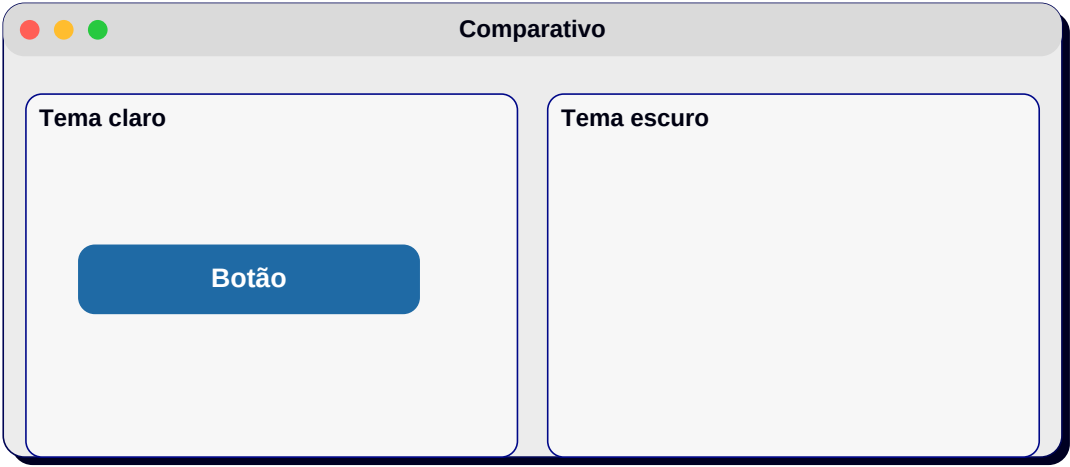
Imagem central, parágrafos e botão fechar.

```
top = tk.Toplevel(app); top.title('Sobre'); top.geometry('360x260')
tk.Label(top, text='Meu App', font=('Segoe UI',16,'bold')).pack(pady=14)
tk.Label(top, text='Versão 1.0\nDesenvolvido em Python + Tkinter\n2026').pack()
tk.Button(top, text='Fechar', command=top.destroy).pack(pady=20)
```



Tema escuro vs claro lado a lado

Coloque dois Frames (um CTK 'light' simulado e outro 'dark').



Apêndice C — Exercícios Propostos

Para fixar o conteúdo, tente resolver os exercícios abaixo. Tente sozinho(a) antes de procurar a solução; o aprendizado vem da tentativa.

1. Crie um conversor de temperatura (Celsius ■ Fahrenheit) com Tkinter usando grid.
2. Adicione validação no formulário de cadastro: email deve conter @, senha deve ter ≥ 6 caracteres.
3. Faça um relógio digital atualizando a cada segundo com after().
4. Implemente uma tela de loading que feche sozinha após 3s e abra a tela principal.
5. Construa uma lista de favoritos: ao clicar em um item da Listbox, ele se move para uma segunda Listbox.
6. Crie um leitor de CSV: filedialog para escolher o arquivo, Treeview para mostrar os dados.
7. Faça um quiz com 5 perguntas, RadioButtons para alternativas e contagem de acertos no final.
8. Em CustomTkinter, implemente um menu lateral com 4 telas alternáveis via botões.
9. Crie um conversor de Markdown → HTML usando Text widget e botão 'Converter'.
10. Faça um pequeno paint: Canvas + botões de cor + slider para espessura.
11. Construa um gerador de senhas aleatórias com slider para tamanho e checkboxes de tipos.
12. Implemente uma agenda telefônica persistente em JSON, com adicionar/editar/excluir.
13. Crie um media viewer: lista de arquivos + preview de imagens com Pillow.
14. Faça um cliente de chat fake com Text widget que envia mensagens para si mesmo.
15. Empacote um dos projetos acima com PyInstaller e teste o .exe em outra máquina.

Apêndice D — Dicas finais e armadilhas comuns

D.1 'Minha janela não fica aberta'

Você esqueceu de chamar `app.mainloop()` no final do código.

D.2 'Minha imagem some'

Você não manteve a referência ao `PhotoImage`. Salve em uma variável global ou em `self.img`.

D.3 'Layout quebrou ao redimensionar'

Configure `rowconfigure/columnconfigure` com `weight` e use `sticky='nsew'`.

D.4 'Botão executou imediatamente'

Você passou `command=funcao()` com parênteses. Correto: `command=funcao`.

D.5 'A GUI congela'

Tarefas longas no thread principal travam o `mainloop`. Use `threading` ou `after()`.

D.6 'No CTK meu botão fica preto'

Você definiu `fg_color` mas esqueceu `hover_color`. Sempre defina os dois para uma UX consistente.

D.7 'Após PyInstaller falta um arquivo'

Use `--add-data` ou `--collect-all customtkinter` para garantir que os recursos sejam empacotados.

Parabéns por chegar até aqui! Esta apostila cobriu desde os fundamentos do Tkinter até o visual moderno do CustomTkinter, com mais de 30 telas desenhadas. Continue praticando — interfaces gráficas são uma das formas mais gratificantes de programar.

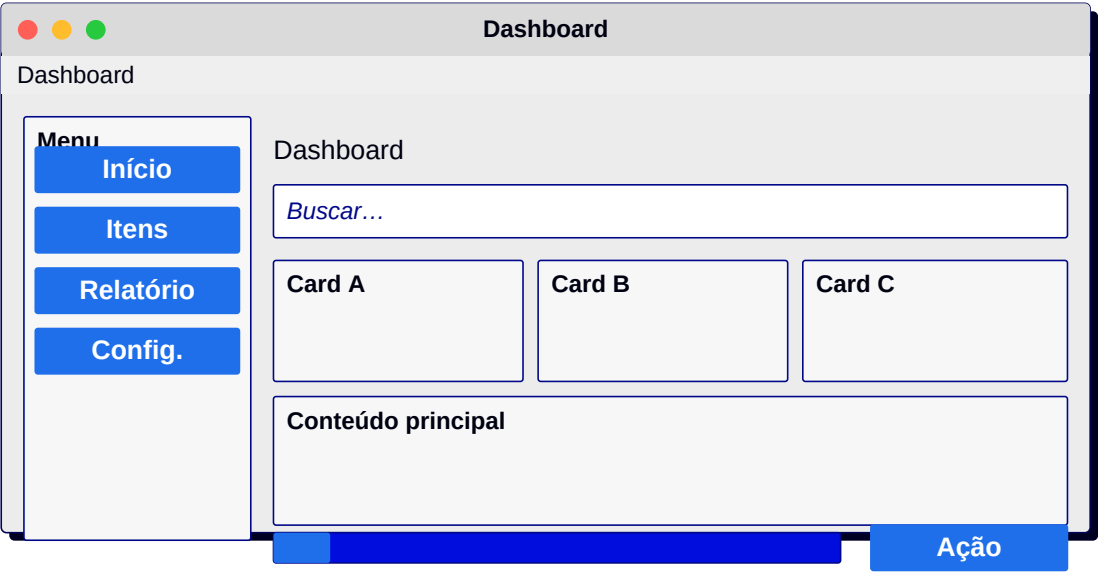
Fim da apostila — Edição 2026.

Parte V — Atlas de 120 Telas Comentadas

Esta seção final apresenta 120 mockups variados, cobrindo todos os tipos de tela que você encontrará em apps reais: dashboards, formulários, fluxos de pagamento, configurações, perfis, login social, onboarding, listagens, detalhes, modais, wizards, calendários, players, mapas, gráficos e muito mais. Cada tela vem acompanhada de descrição curta e indicação dos widgets envolvidos.

1. Dashboard

Sidebar + 3 cards de KPI + gráfico grande.



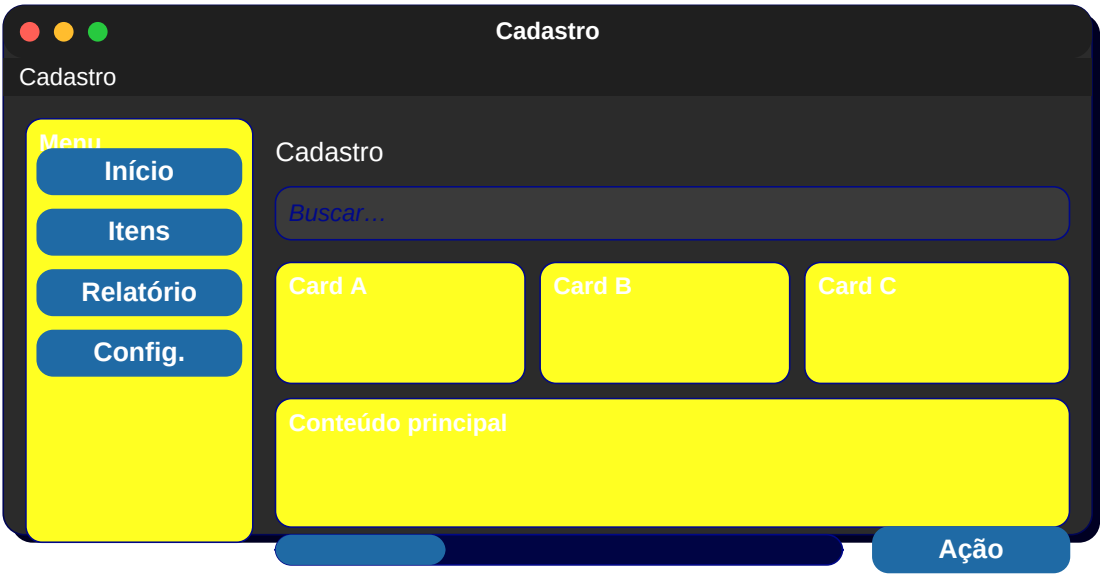
2. Login

Logo central + 2 entries + botão primário + link 'Esqueci a senha'.



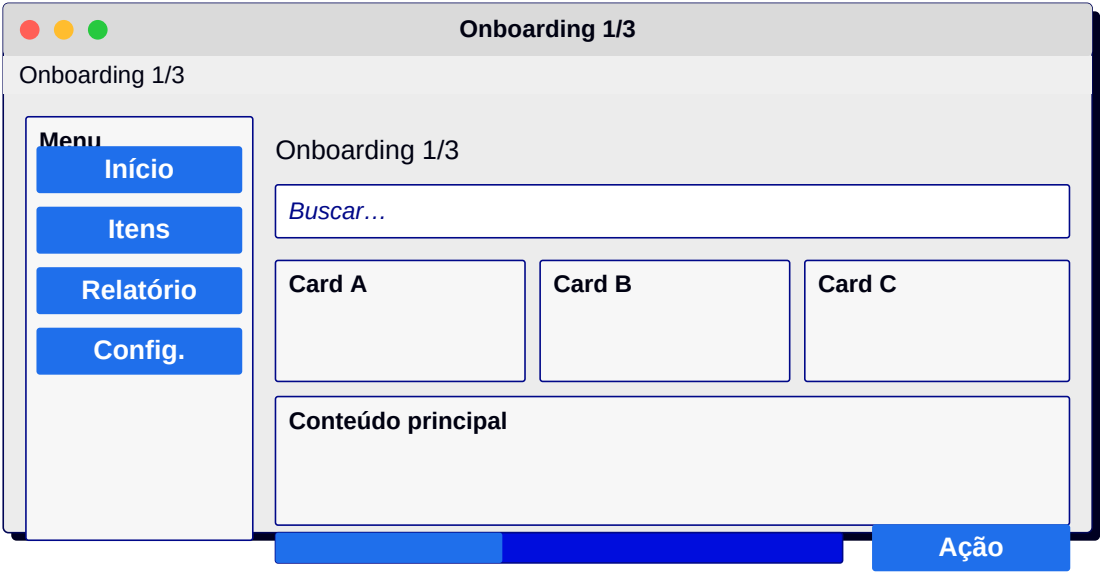
3. Cadastro

Vários campos em grid + checkbox de termos + botão de cadastro.



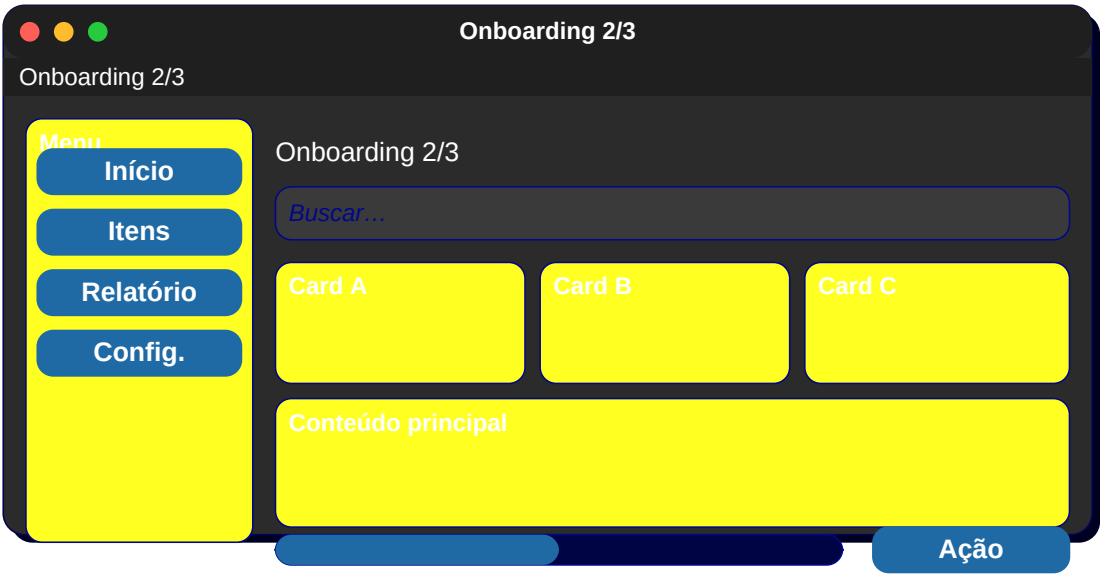
4. Onboarding 1/3

Imagem + título + descrição + botão Próximo.



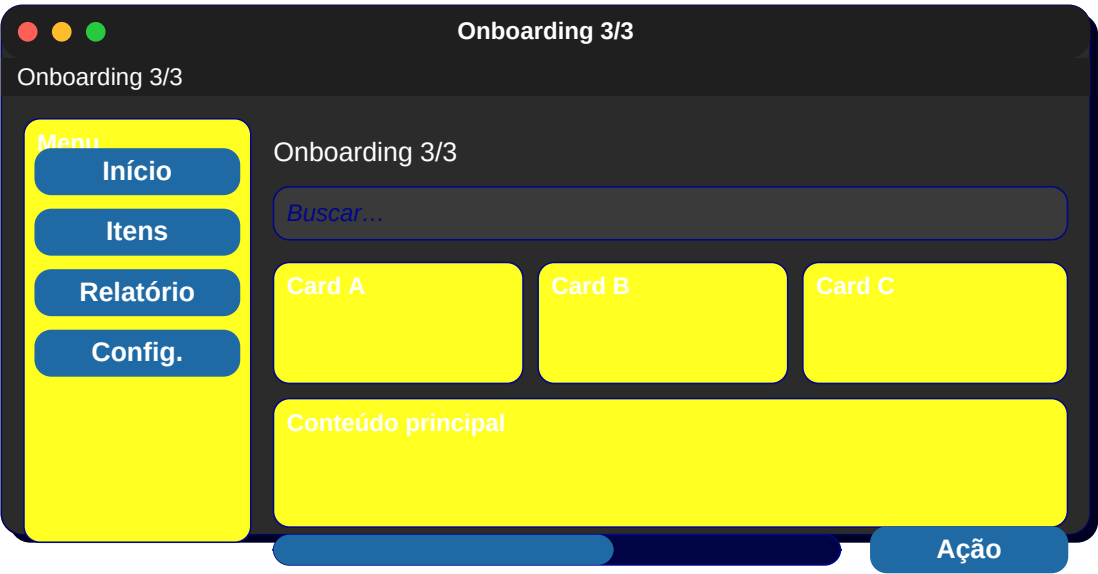
5. Onboarding 2/3

Mesma estrutura, conteúdo diferente.



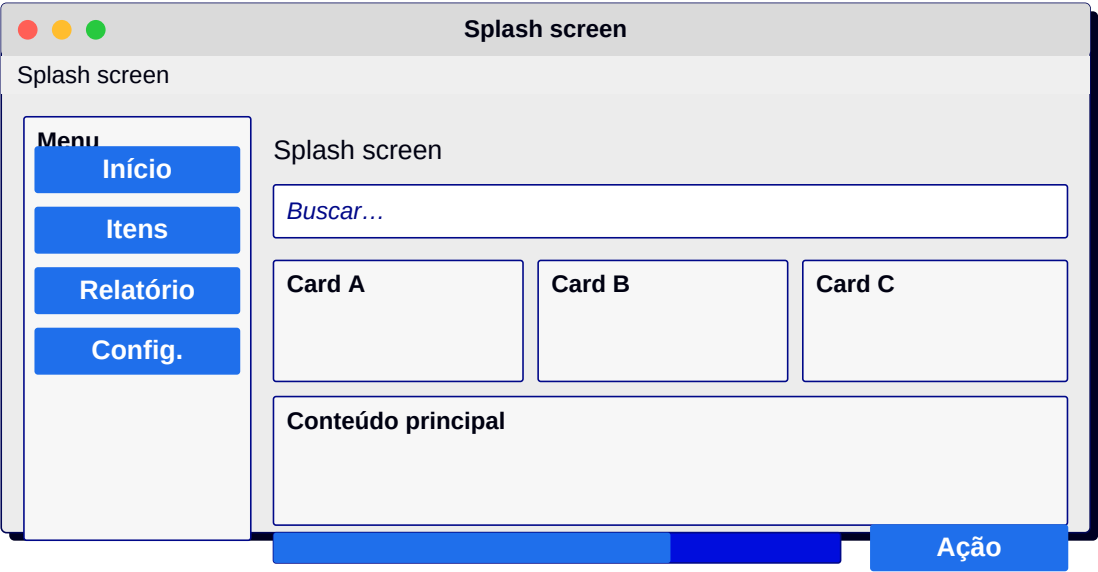
6. Onboarding 3/3

Botão final 'Começar'.



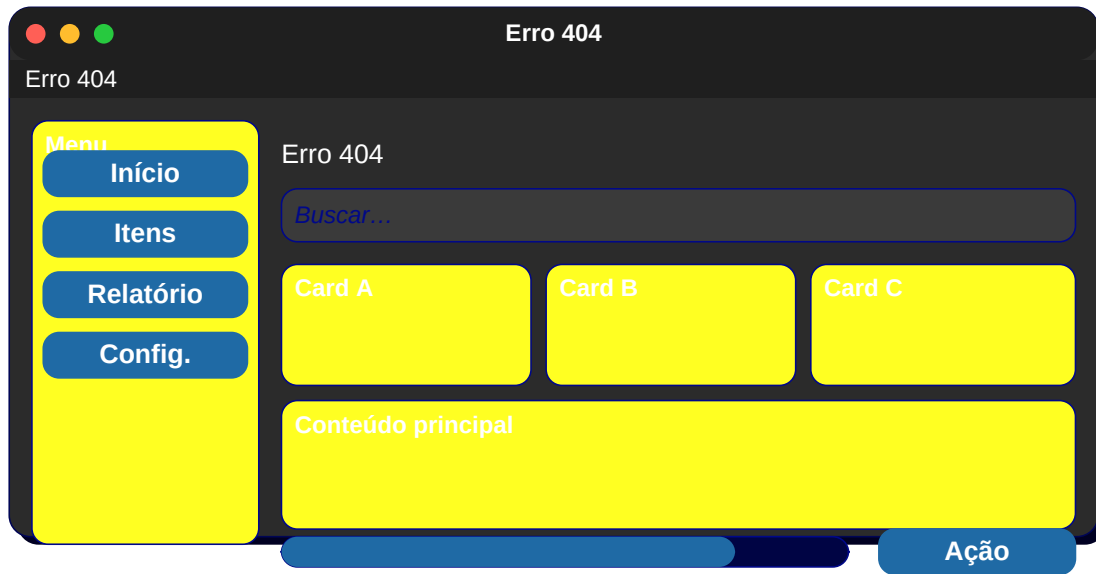
7. Splash screen

Logo grande centralizado + progress bar.



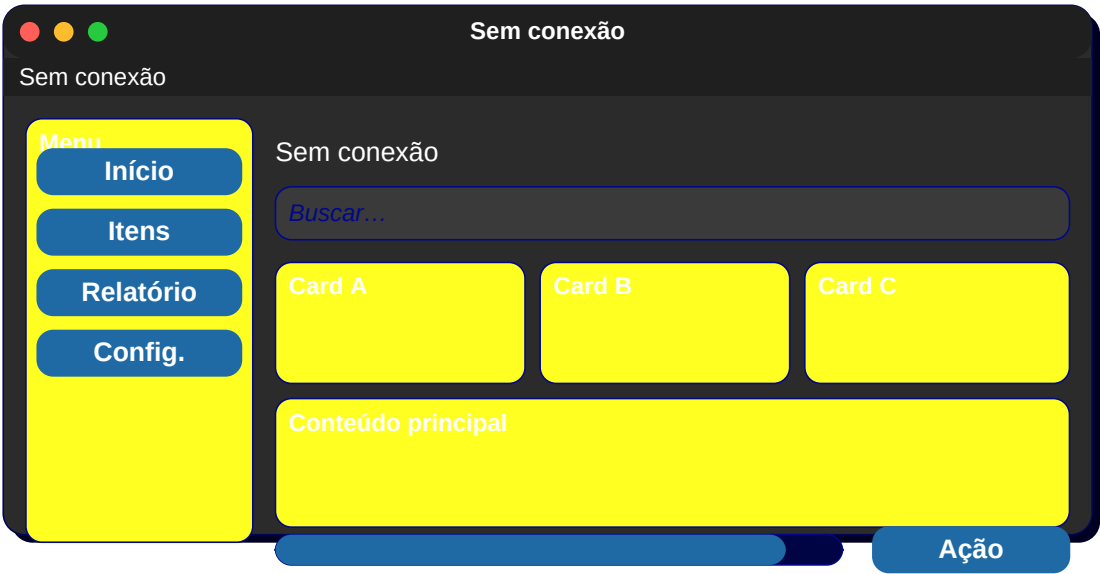
8. Erro 404

Ilustração + mensagem + botão Voltar.



9. Sem conexão

Ícone + mensagem + botão Tentar novamente.



10. Confirmação

Modal com pergunta + botões Sim/Não.

Confirmação

Confirmação

Menu

Início

Itens

Relatório

Config.

Confirmação

Buscar...

Card A

Card B

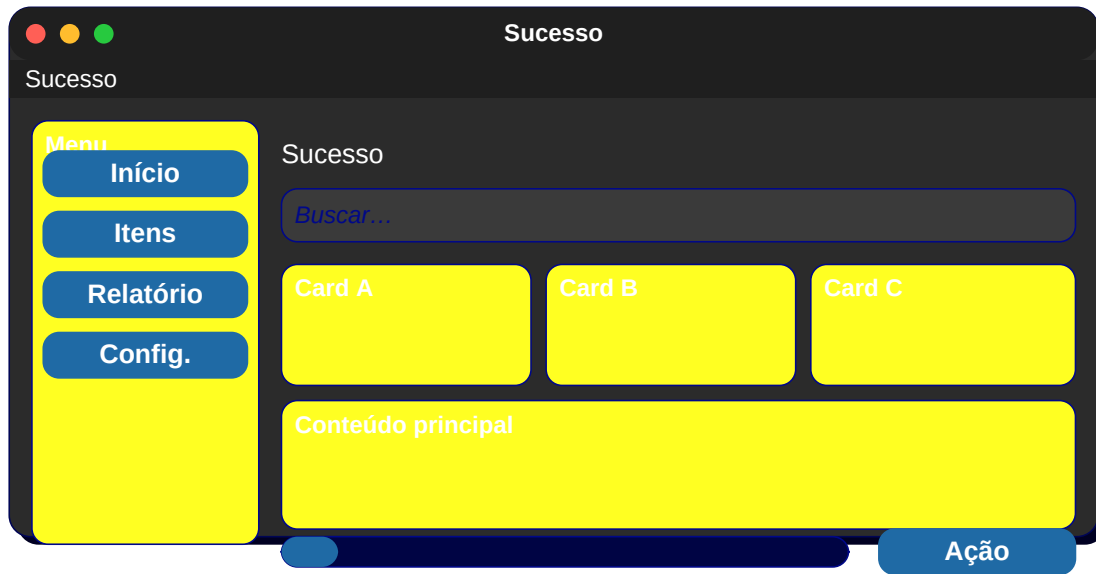
Card C

Conteúdo principal

Ação

11. Sucesso

Ícone verde + texto + botão OK.



12. Alerta

Ícone amarelo + texto + botões.



13. Perfil do usuário

Avatar + nome + bio + estatísticas + abas.

Perfil do usuário

Perfil do usuário

Menu

Início

Itens

Relatório

Config.

Perfil do usuário

Buscar...

Card A

Card B

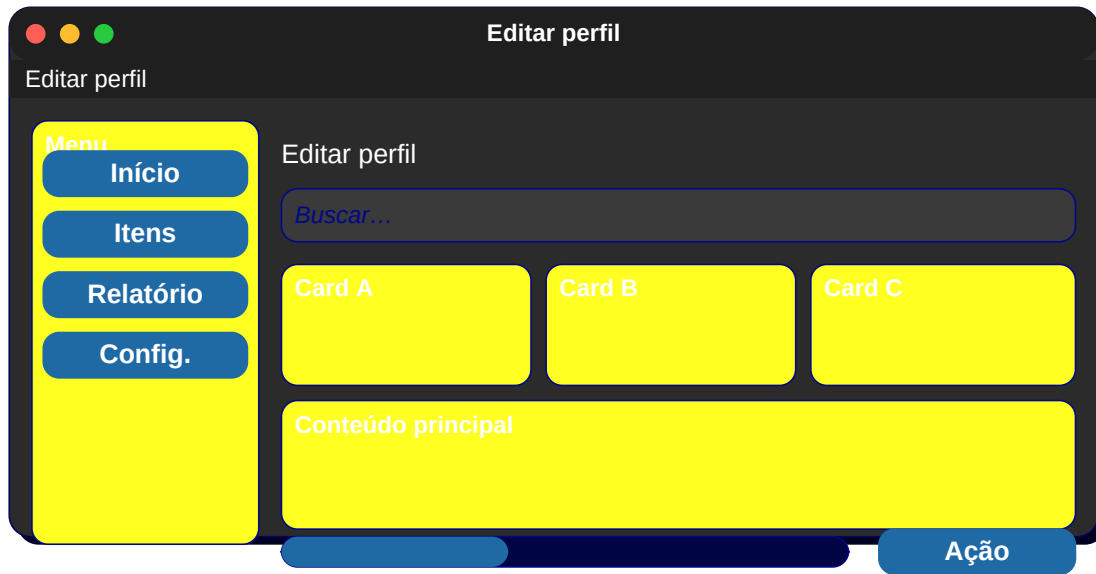
Card C

Conteúdo principal

Ação

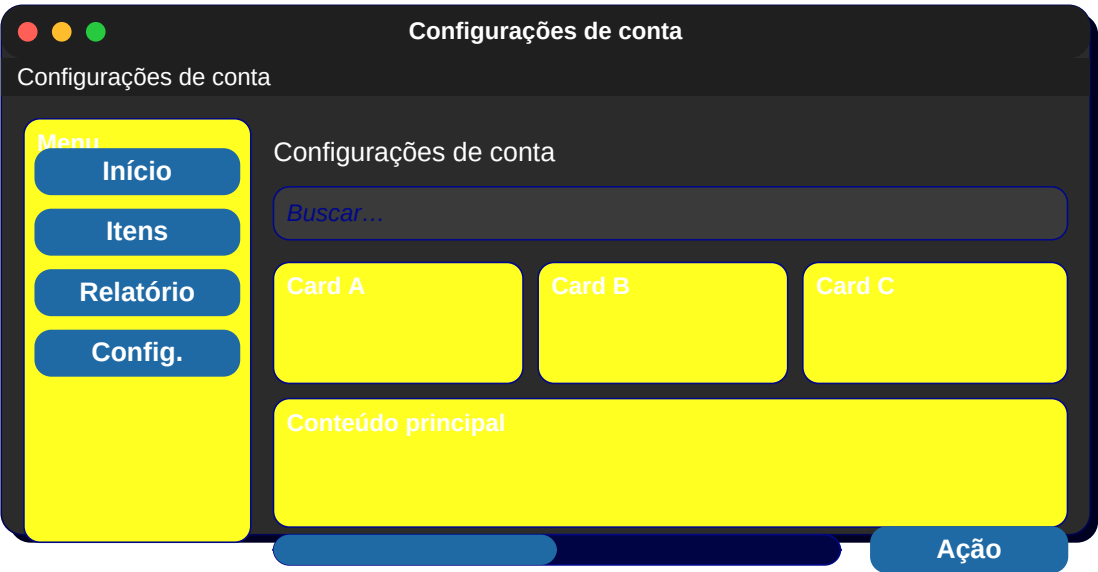
14. Editar perfil

Formulário com upload de foto + campos.



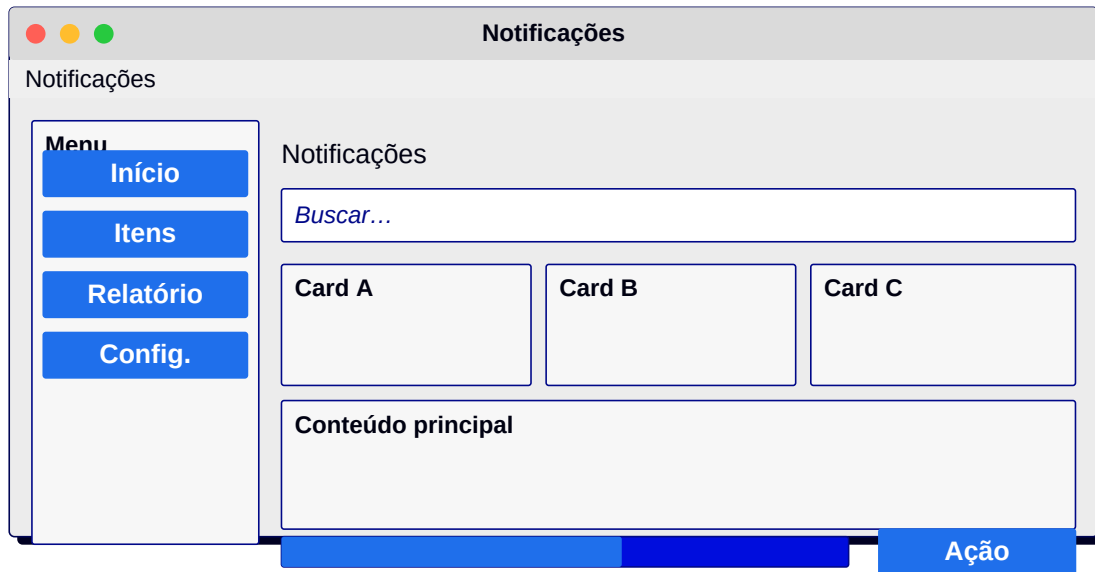
15. Configurações de conta

Lista de opções com switches.



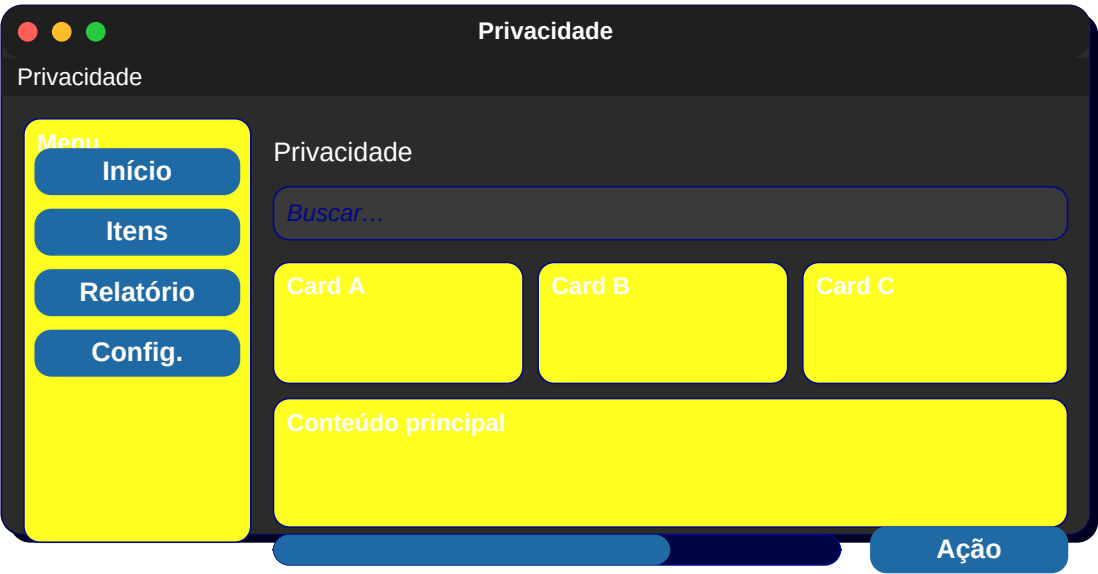
16. Notificações

Lista de itens com ícone + título + tempo.



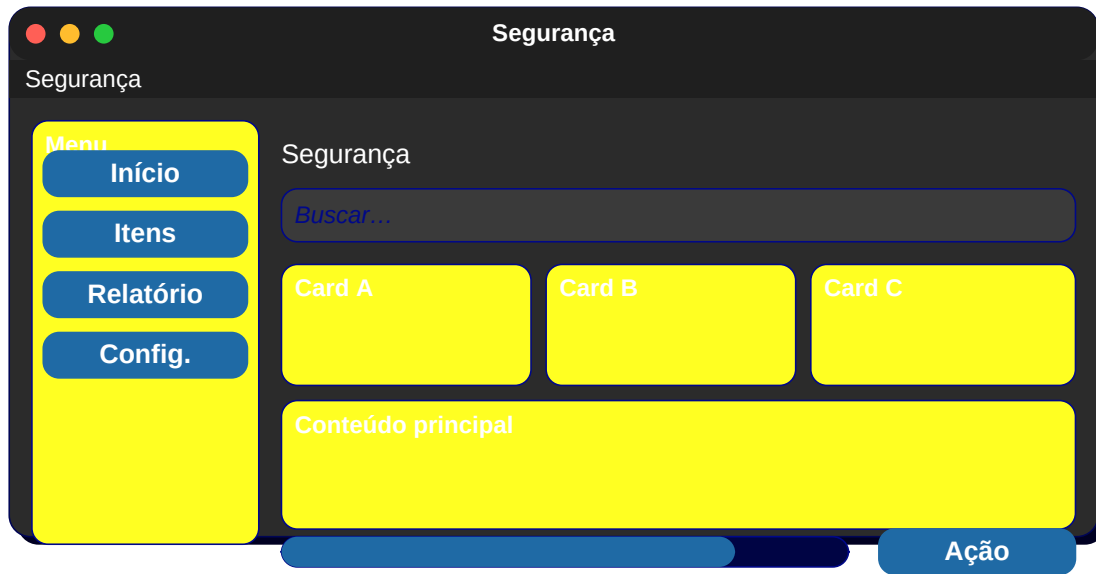
17. Privacidade

Switches + descrição longa.



18. Segurança

2FA + sessões ativas + alterar senha.



19. Pagamentos

Lista de métodos + botão Adicionar.

Pagamentos

Pagamentos

Menu

Início

Itens

Relatório

Config.

Pagamentos

Buscar...

Card A

Card B

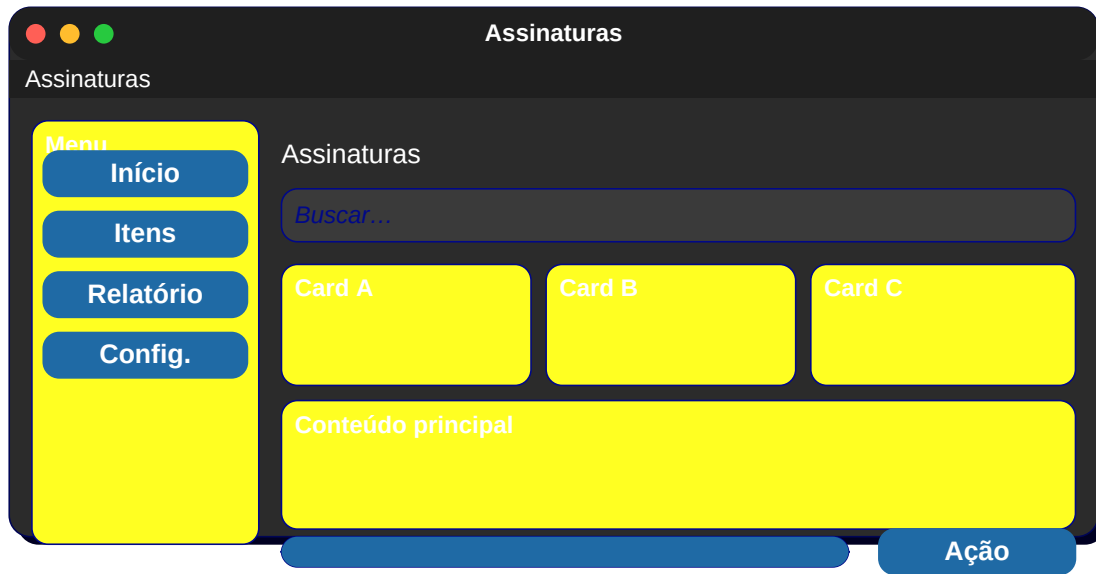
Card C

Conteúdo principal

Ação

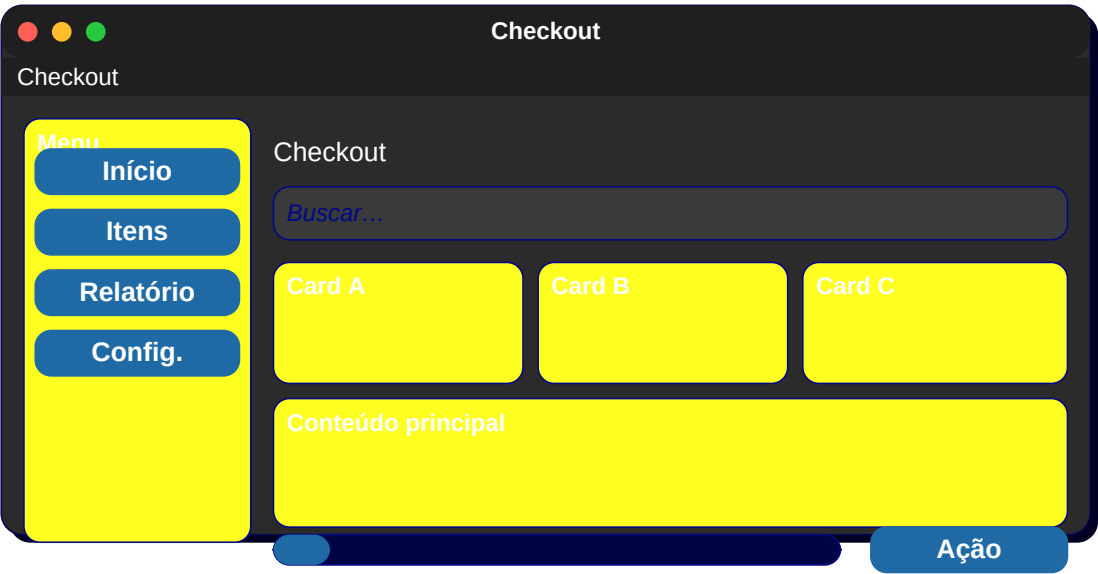
20. Assinaturas

3 planos lado a lado + botão Escolher.



21. Checkout

Resumo do pedido + endereço + pagamento + total.



22. Carrinho

Lista de itens + qtd + preço + total + finalizar.

Carrinho

Carrinho

Menu

Início

Itens

Relatório

Config.

Carrinho

Buscar...

Card A

Card B

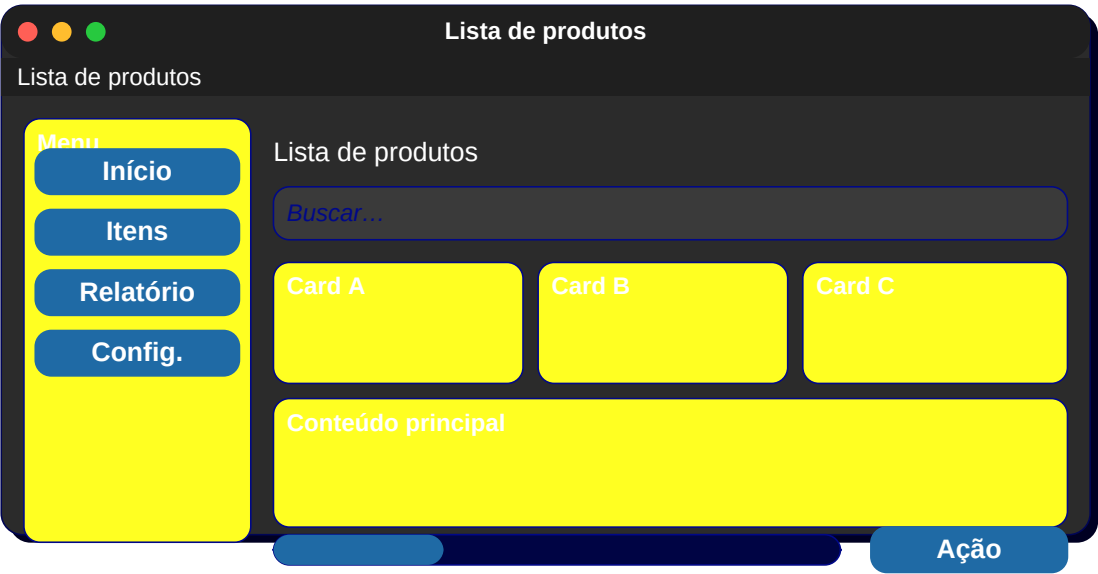
Card C

Conteúdo principal

Ação

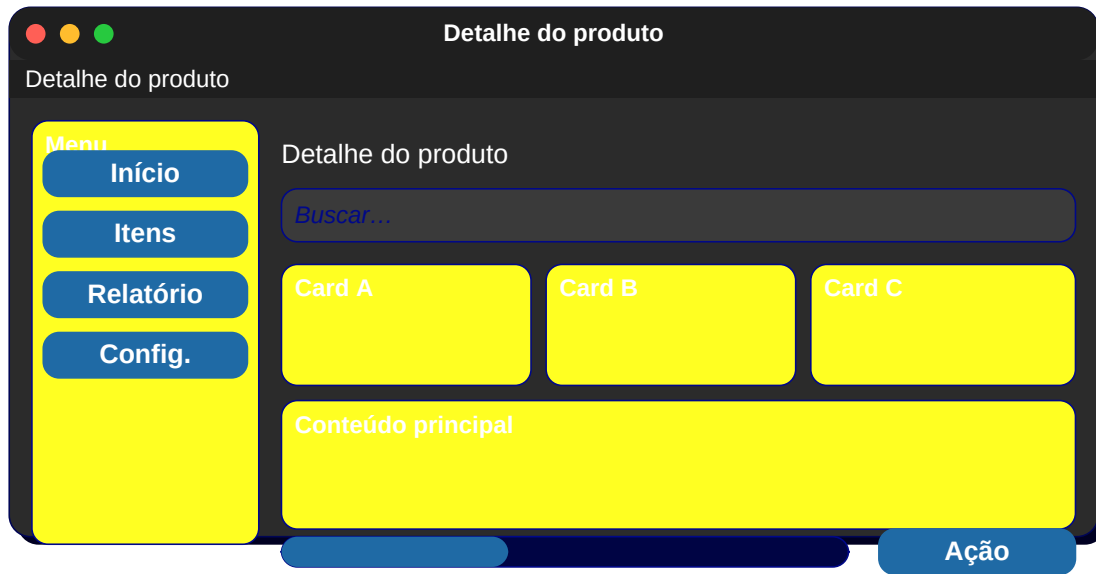
23. Lista de produtos

Grid 3x3 com imagem + título + preço.



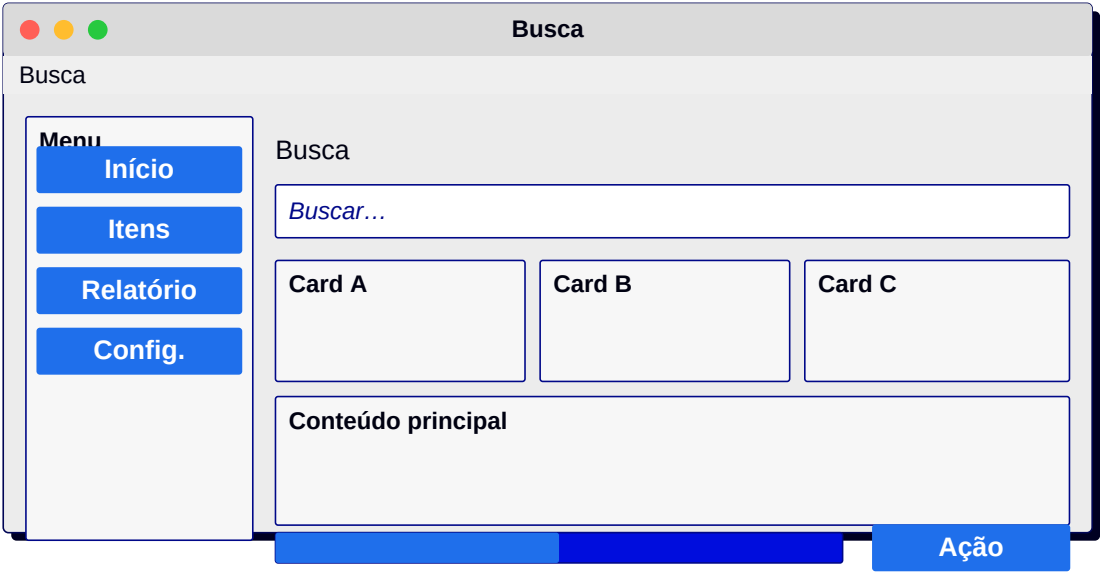
24. Detalhe do produto

Foto grande + descrição + variações + comprar.



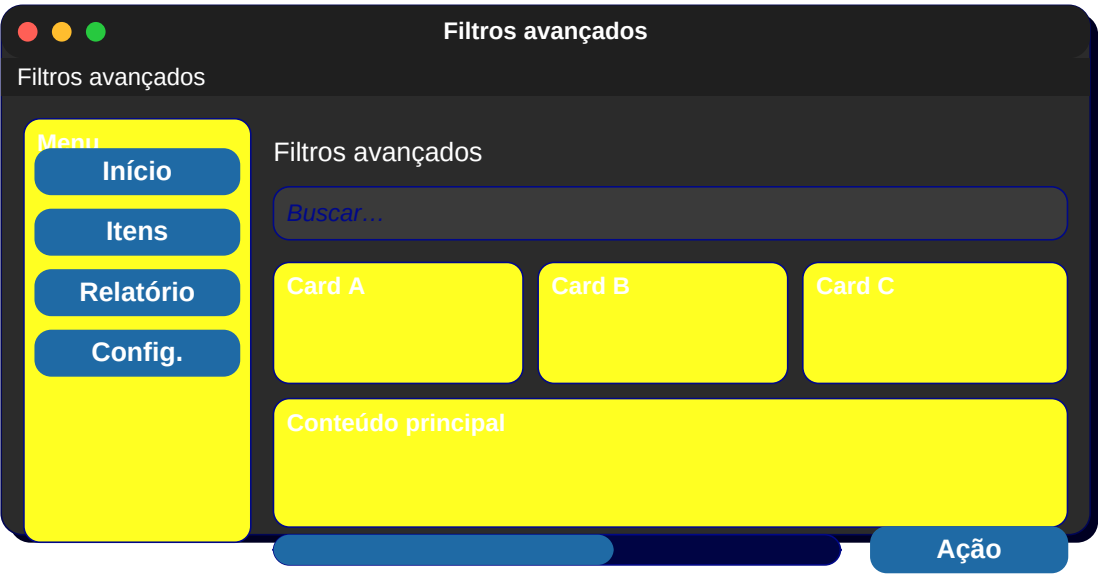
25. Busca

Entry com sugestões + filtros à esquerda + resultados.



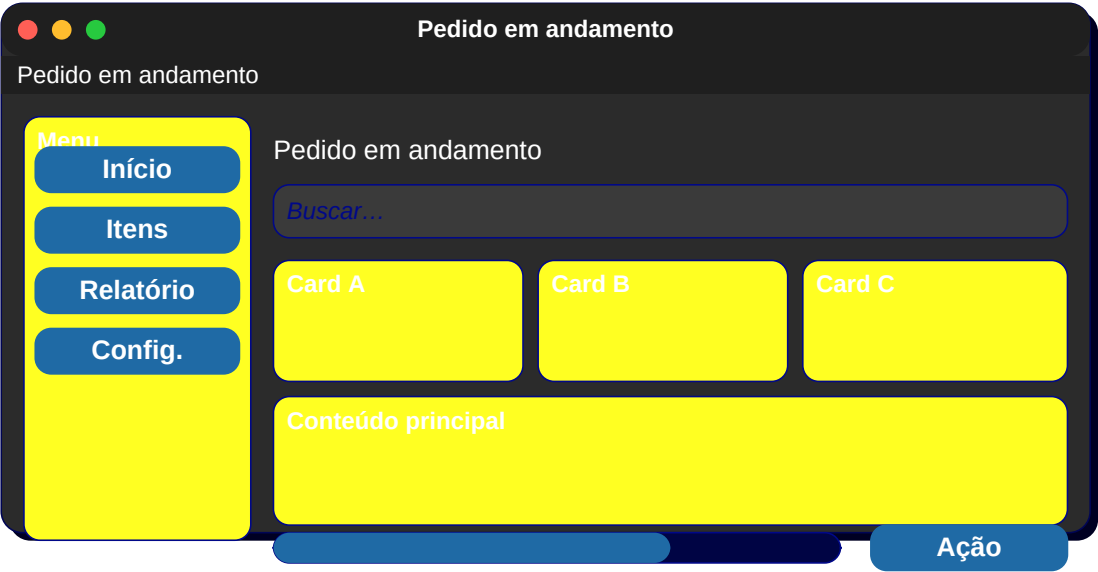
26. Filtros avançados

Sliders + checkboxes em painel lateral.



27. Pedido em andamento

Timeline horizontal com etapas.



28. Histórico de pedidos

Lista com data + valor + status.

Histórico de pedidos

Histórico de pedidos

Menu

Início

Itens

Relatório

Config.

Histórico de pedidos

Buscar...

Card A

Card B

Card C

Conteúdo principal

Ação

29. Mensagens

Lista de conversas + busca no topo.



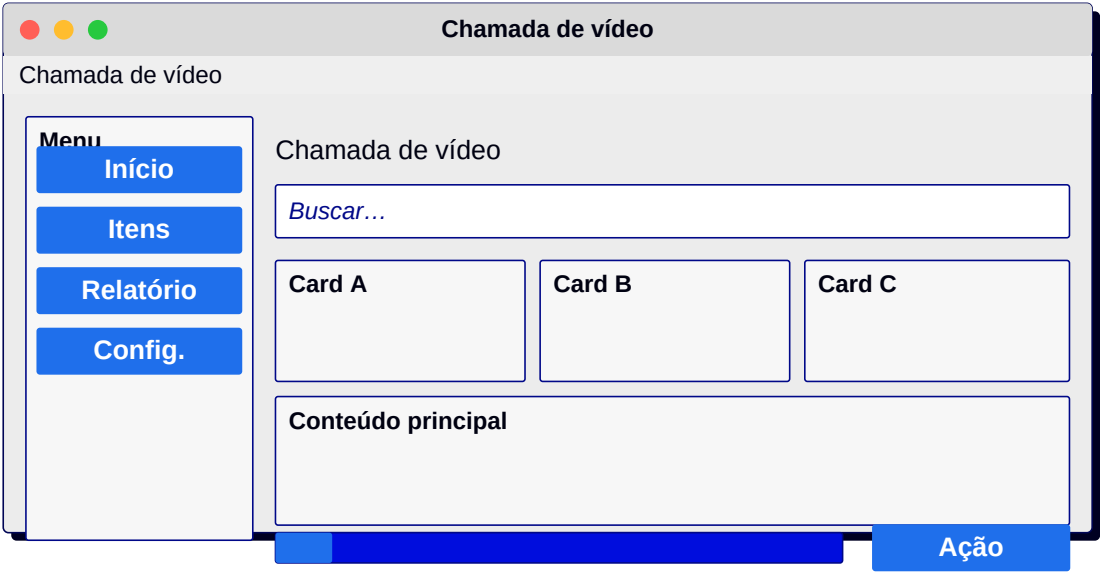
30. Chat aberto

Bolhas de mensagem + entrada + enviar.



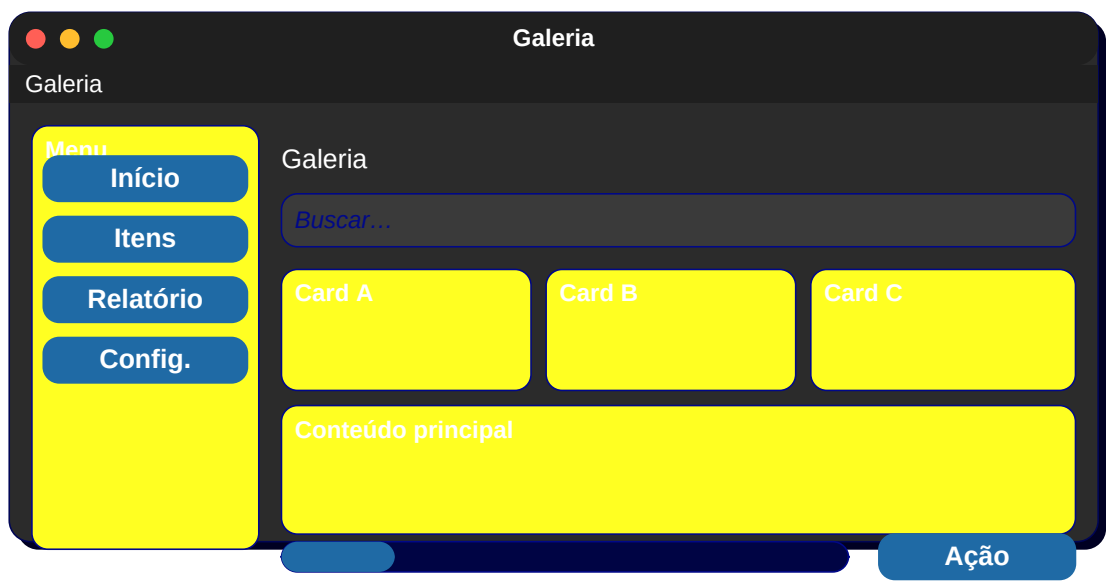
31. Chamada de vídeo

Vídeo grande + miniatura + controles.



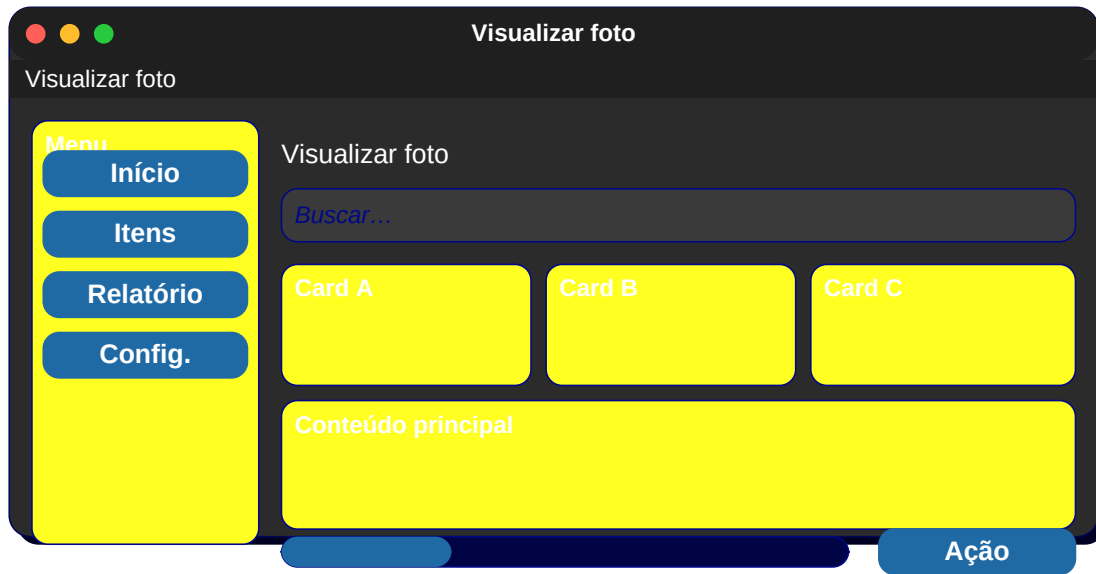
32. Galeria

Grid 4x4 de miniaturas.



33. Visualizar foto

Foto grande + curtir + compartilhar.



34. Upload de arquivo

Drop zone + barra de progresso.

Upload de arquivo

Upload de arquivo

Menu

Início

Itens

Relatório

Config.

Upload de arquivo

Buscar...

Card A

Card B

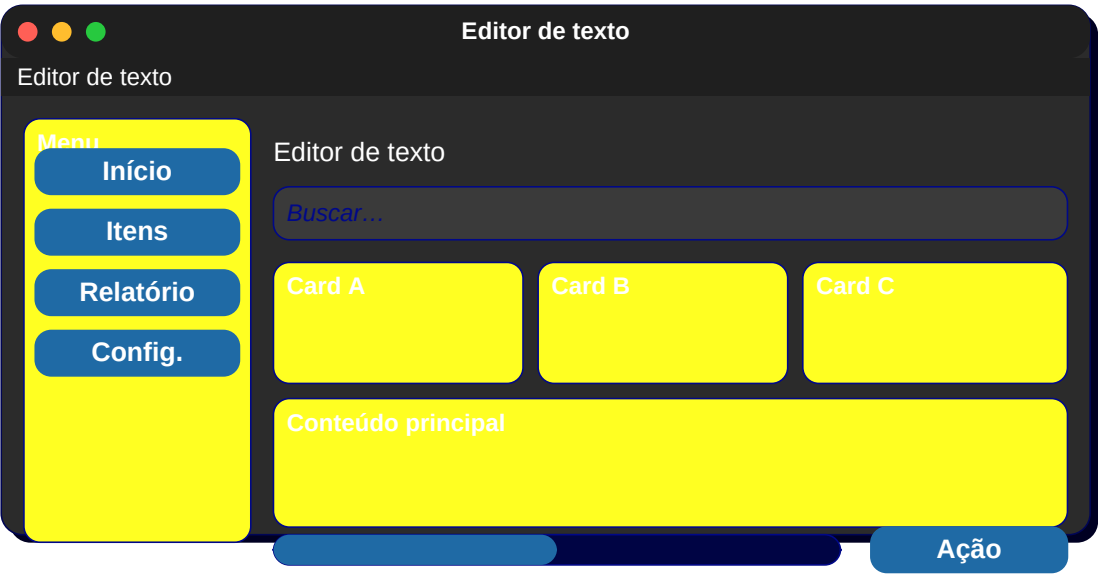
Card C

Conteúdo principal

Ação

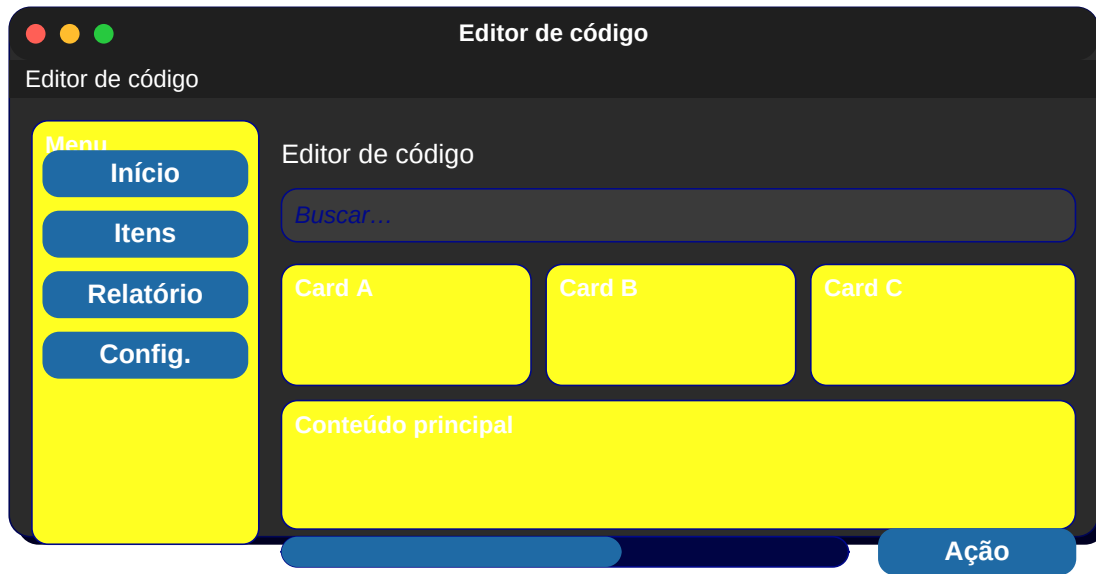
35. Editor de texto

Toolbar + área de edição + status bar.



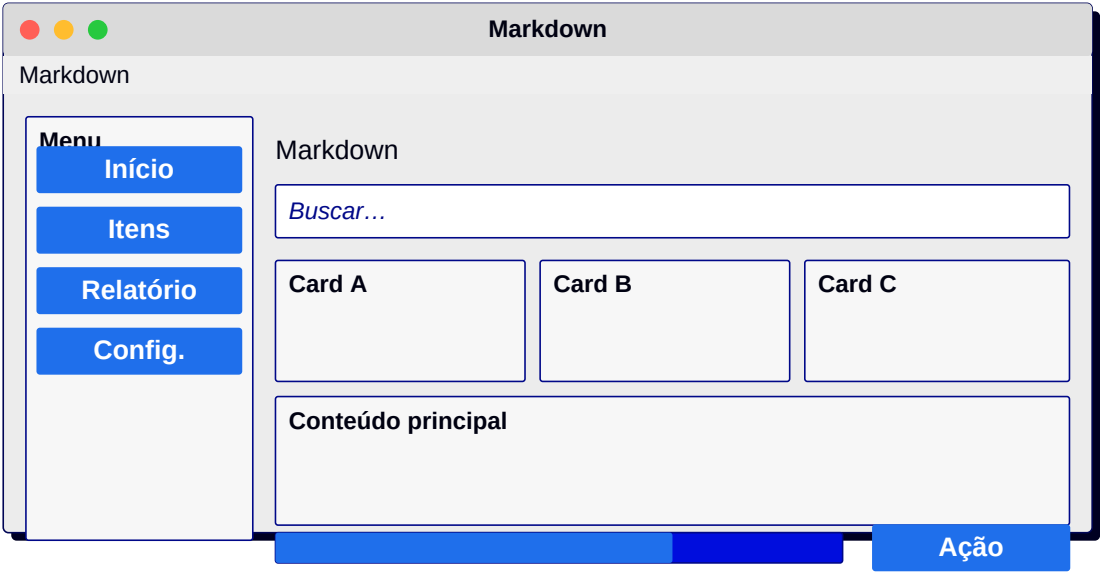
36. Editor de código

Sidebar de arquivos + editor + console.



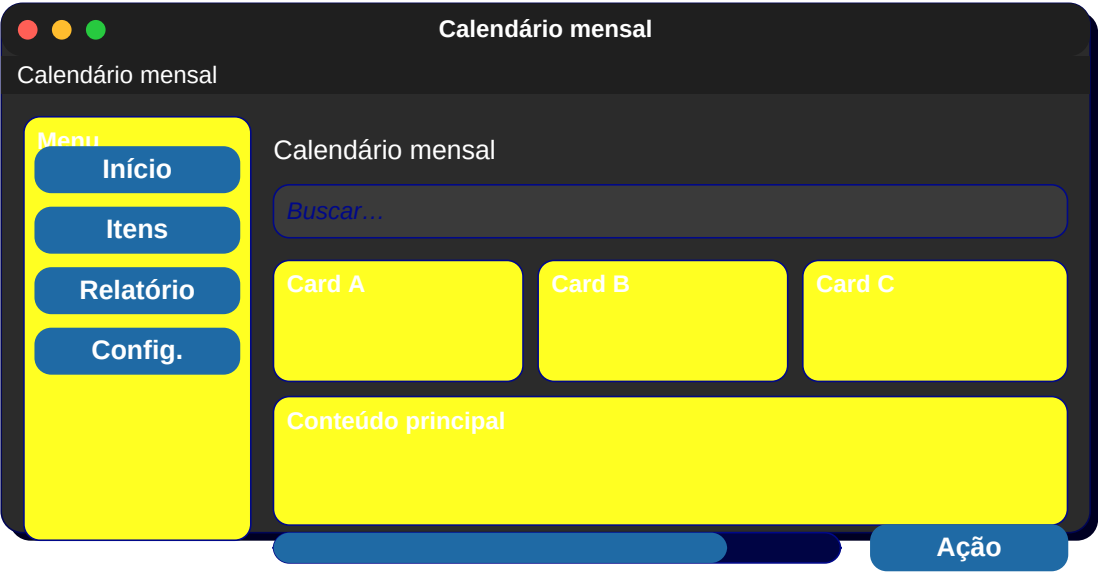
37. Markdown

Editor à esquerda + preview à direita.



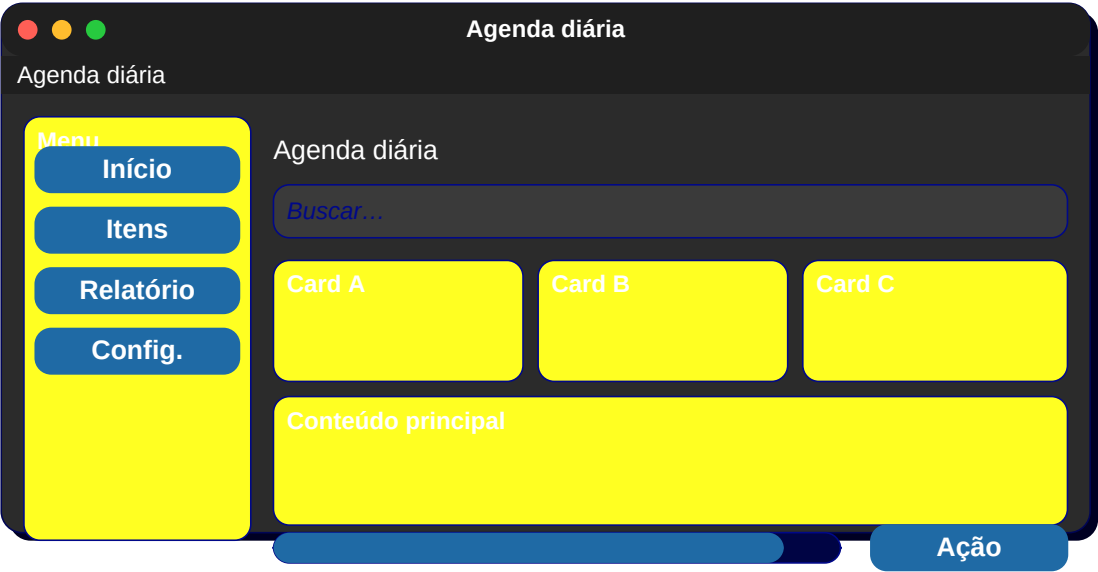
38. Calendário mensal

Grid 7x5 + navegação de mês.



39. Agenda diária

Linha do tempo com horários.



40. Criar evento

Formulário com data, hora, descrição.

Criar evento

Criar evento

Menu

Início

Itens

Relatório

Config.

Criar evento

Buscar...

Card A

Card B

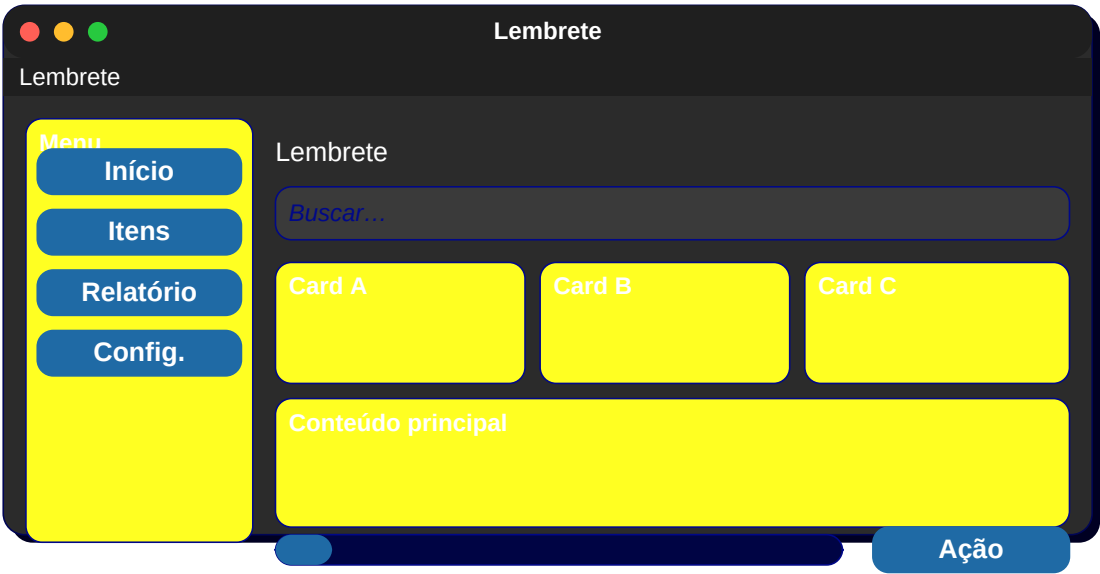
Card C

Conteúdo principal

Ação

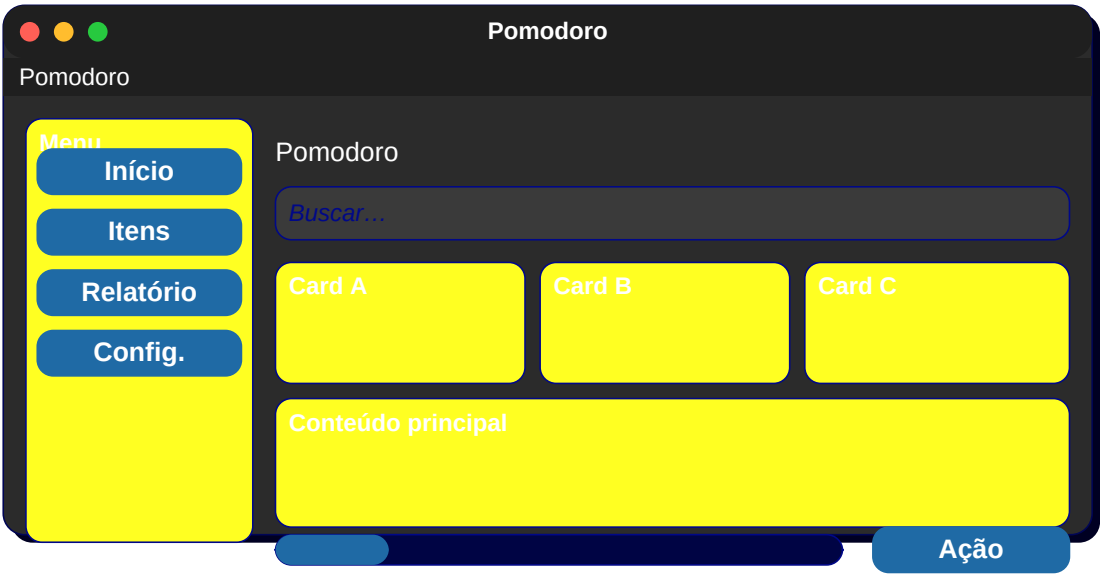
41. Lembrete

Modal com alarme + botão Soneca.



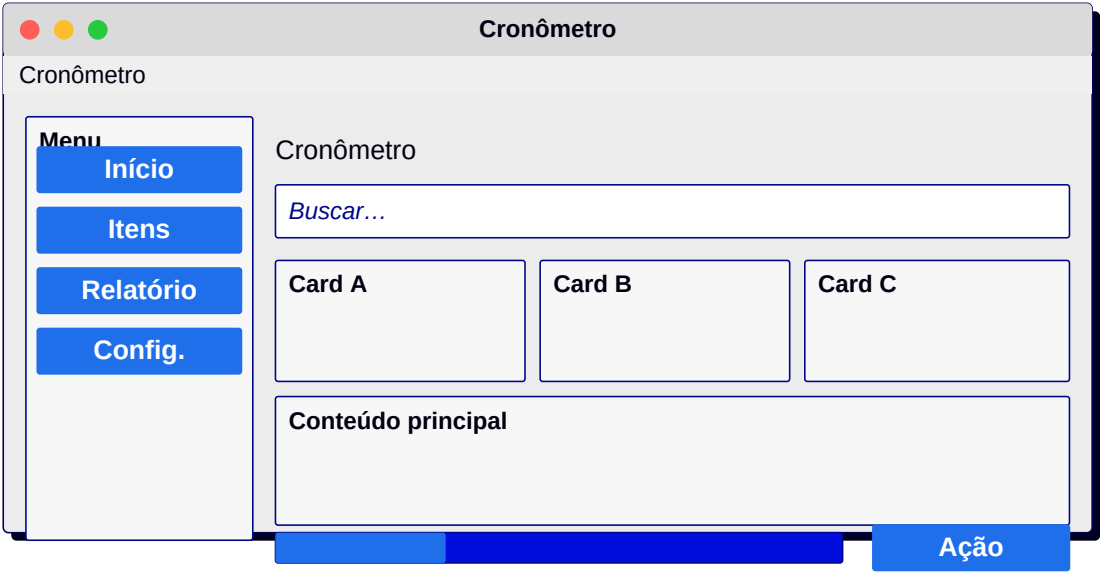
42. Pomodoro

Cronômetro grande + botões + histórico.



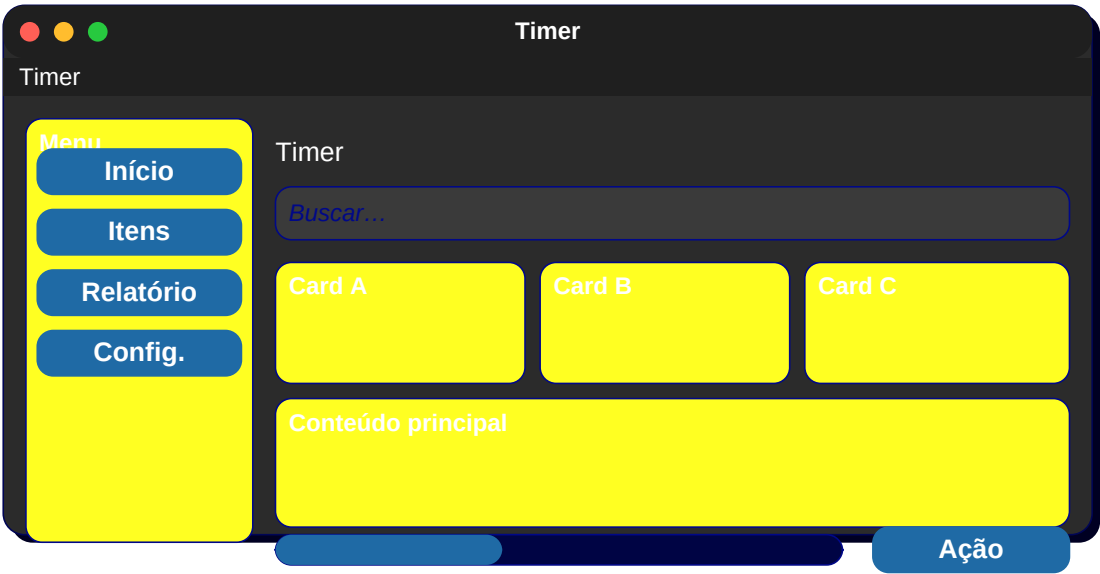
43. Cronômetro

Display + start/pause/lap.



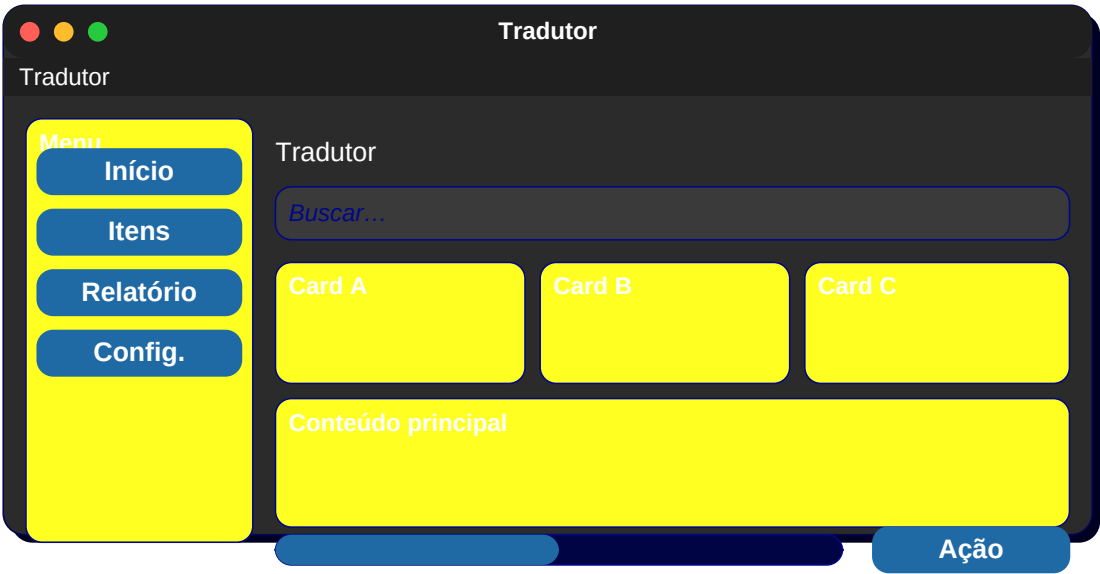
44. Timer

Roda de seleção de tempo + botão Iniciar.



45. Tradutor

Dois textareas + seleção de idioma.



46. Conversor

Entrada + dois selects + saída.

Conversor

Conversor

Menu

Início

Itens

Relatório

Config.

Conversor

Buscar...

Card A

Card B

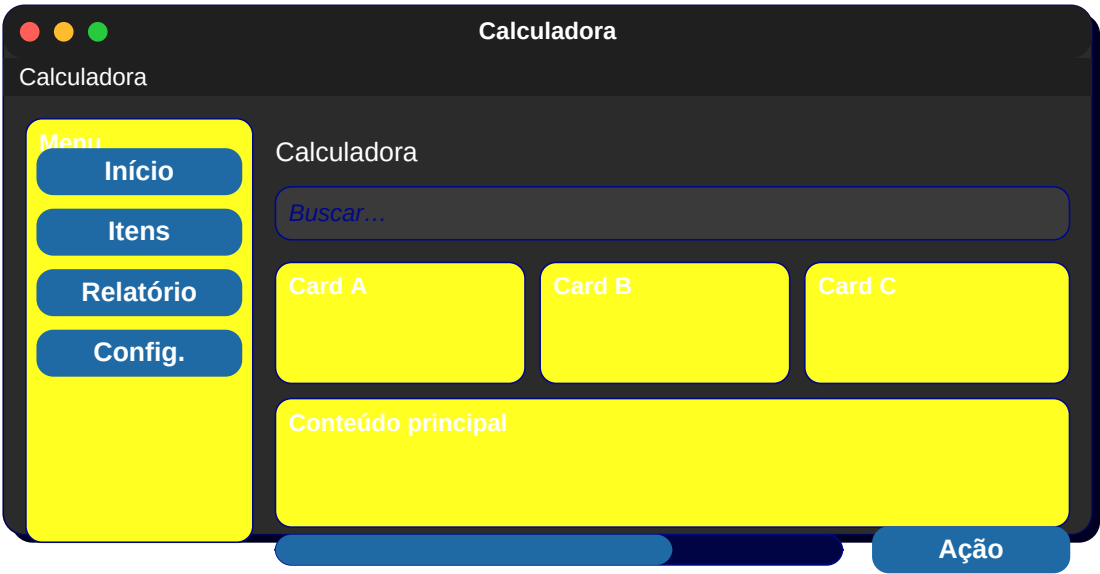
Card C

Conteúdo principal

Ação

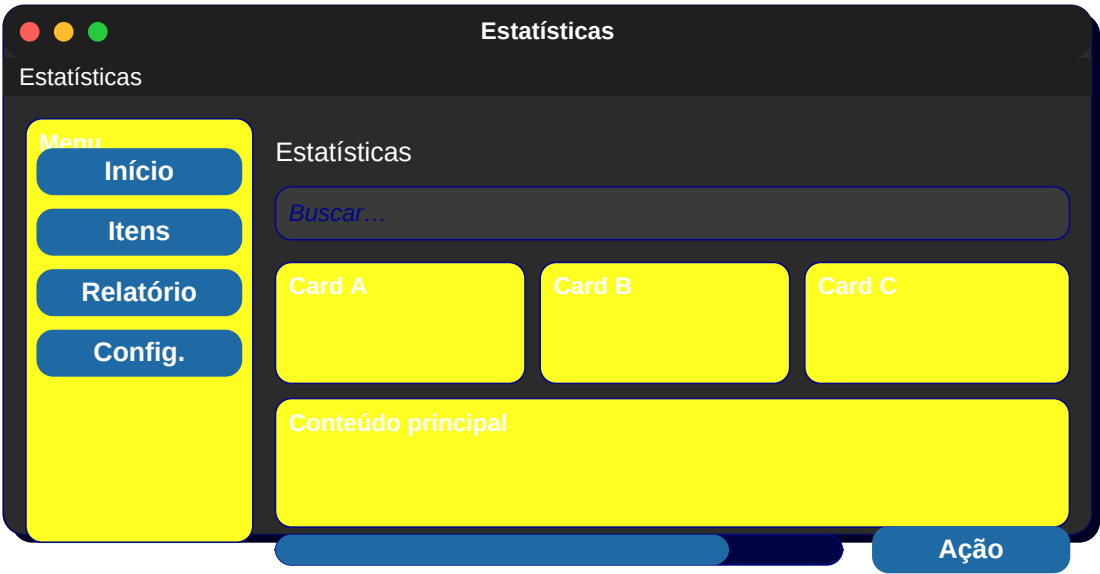
47. Calculadora

Display + teclado numérico.



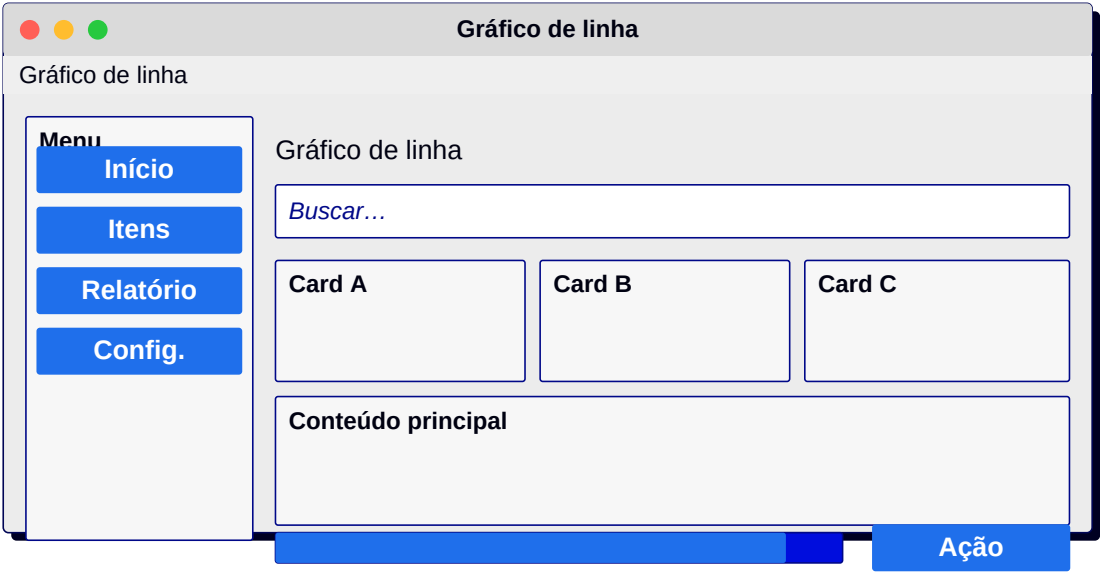
48. Estatísticas

Cards de números + gráfico de barras.



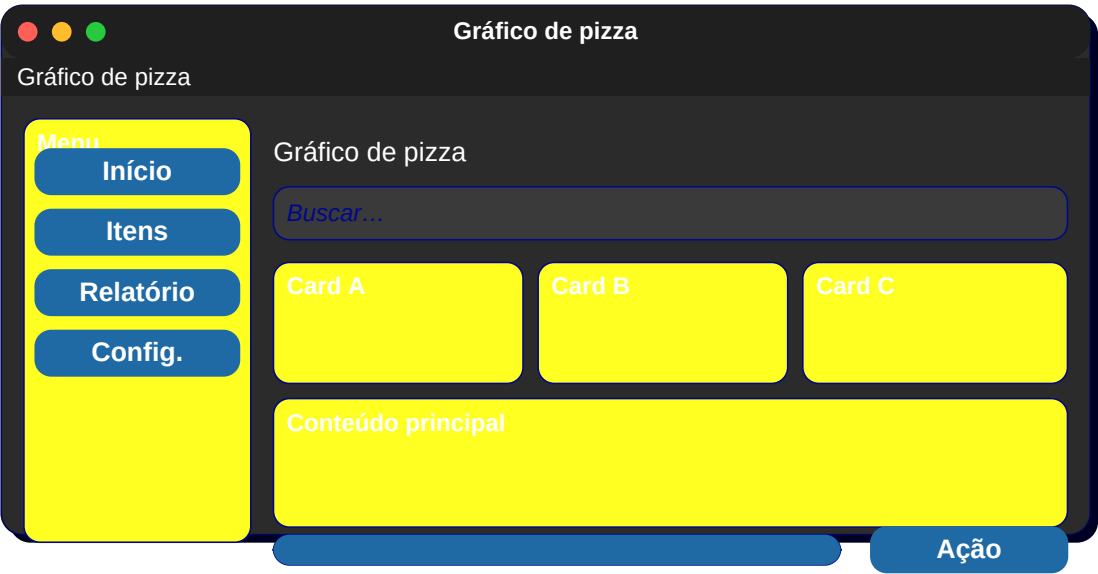
49. Gráfico de linha

Área principal com curva + legenda.



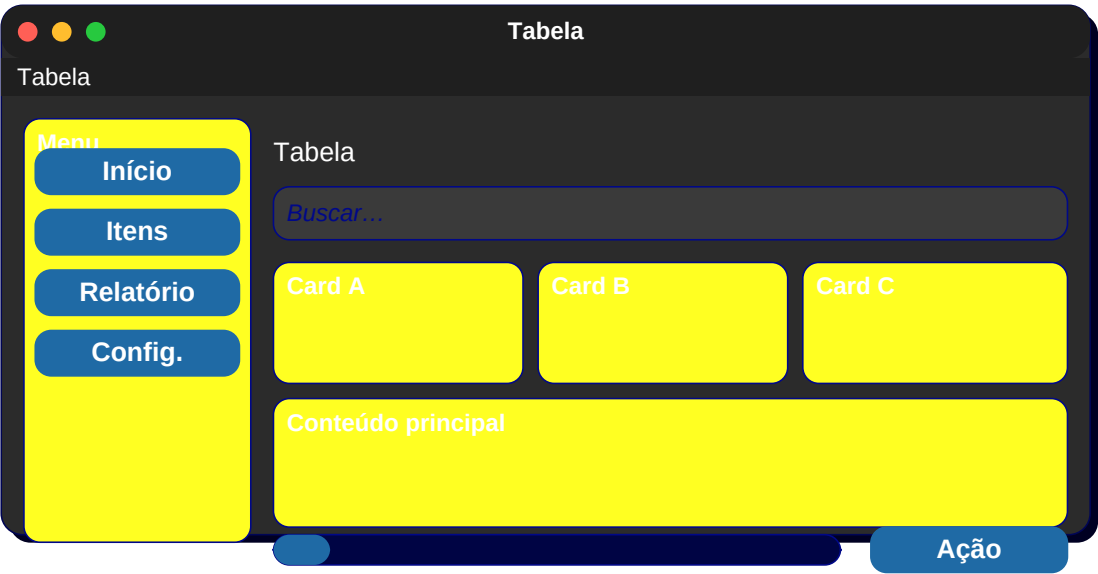
50. Gráfico de pizza

Círculo segmentado + legenda lateral.



51. Tabela

Cabeçalho + linhas + paginação.



52. Tabela editável

Inputs dentro das células.

Tabela editável

Menu

Início

Itens

Relatório

Config.

Tabela editável

Buscar...

Card A

Card B

Card C

Conteúdo principal

Ação

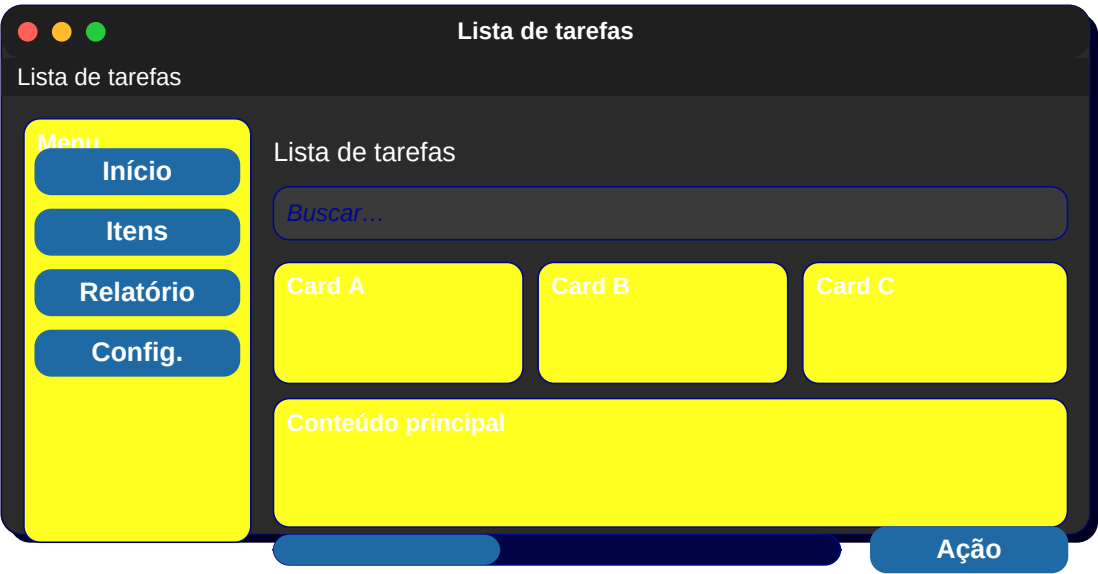
53. Kanban

3 colunas (Todo, Doing, Done) com cards.



54. Lista de tarefas

Checkbox + texto + botão excluir.



55. Detalhe de tarefa

Título + descrição + subtarefas + comentários.

Detalhe de tarefa

Detalhe de tarefa

Menu

Início

Itens

Relatório

Config.

Detalhe de tarefa

Buscar...

Card A

Card B

Card C

Conteúdo principal

Ação

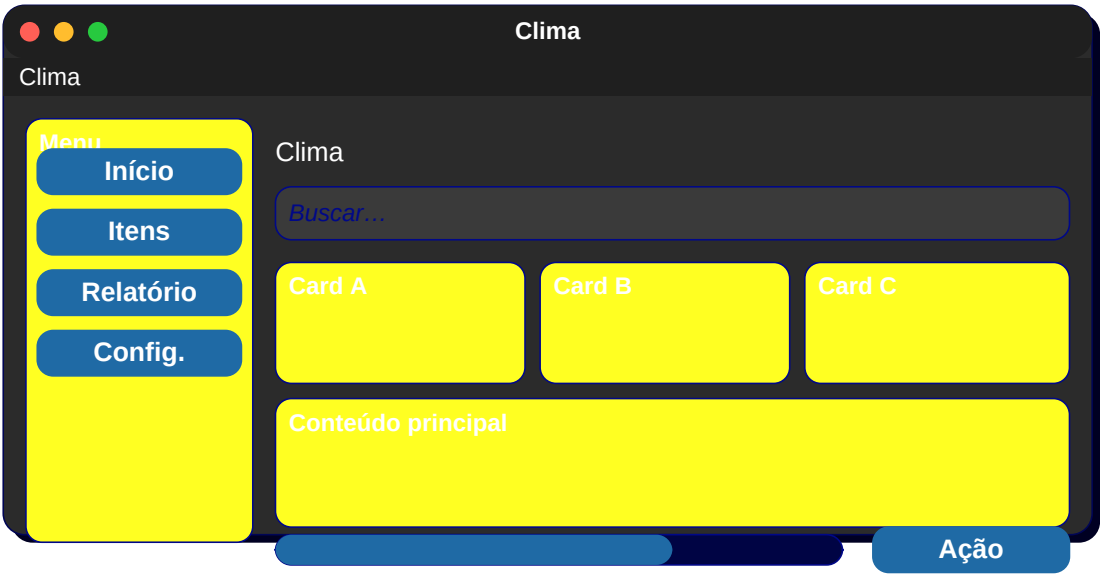
56. Mapa

Área de mapa + pin + painel lateral.



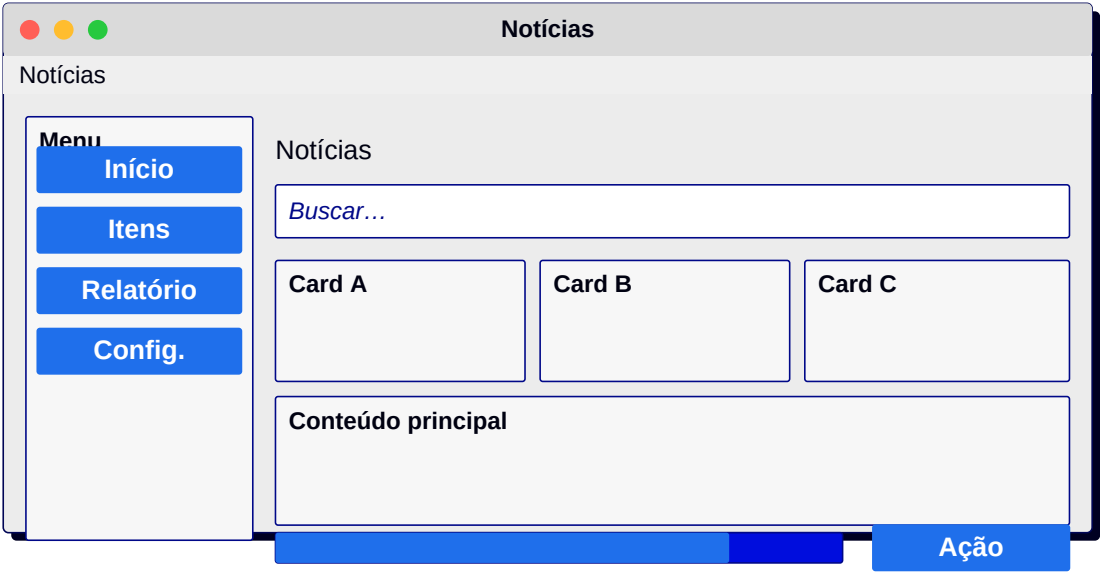
57. Clima

Cidade + temperatura grande + previsão 7 dias.



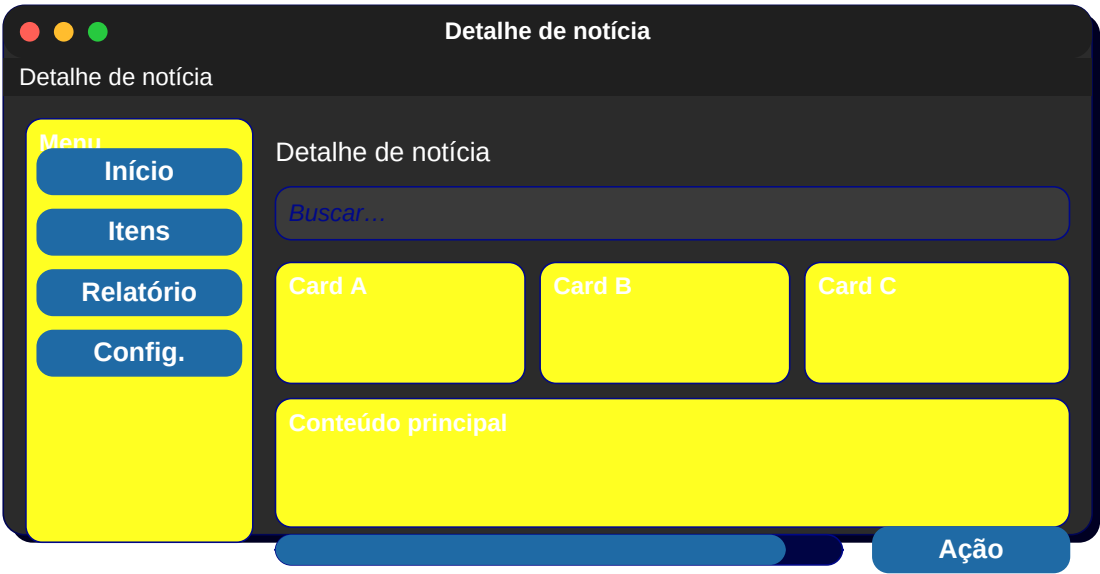
58. Notícias

Cards grandes com imagem + título.



59. Detalhe de notícia

Imagem topo + texto longo.



60. Blog

Lista de posts + sidebar.



61. Comentários

Lista aninhada de respostas.

Comentários

Comentários

Menu

Início

Itens

Relatório

Config.

Comentários

Buscar...

Card A

Card B

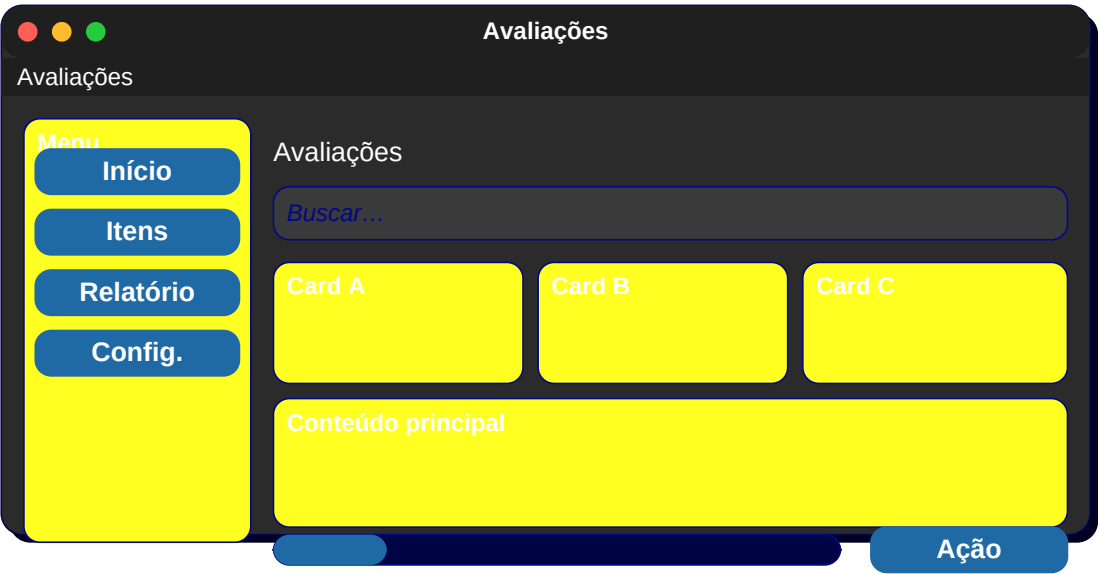
Card C

Conteúdo principal

Ação

62. Avaliações

Estrelas + texto + lista de reviews.



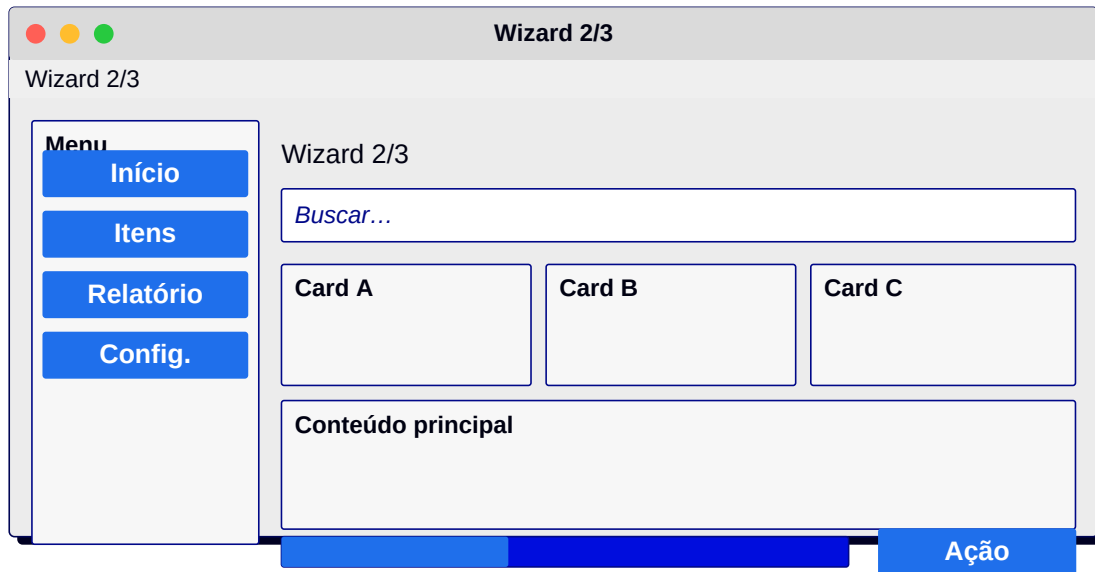
63. Wizard 1/3

Indicador de passo + formulário.



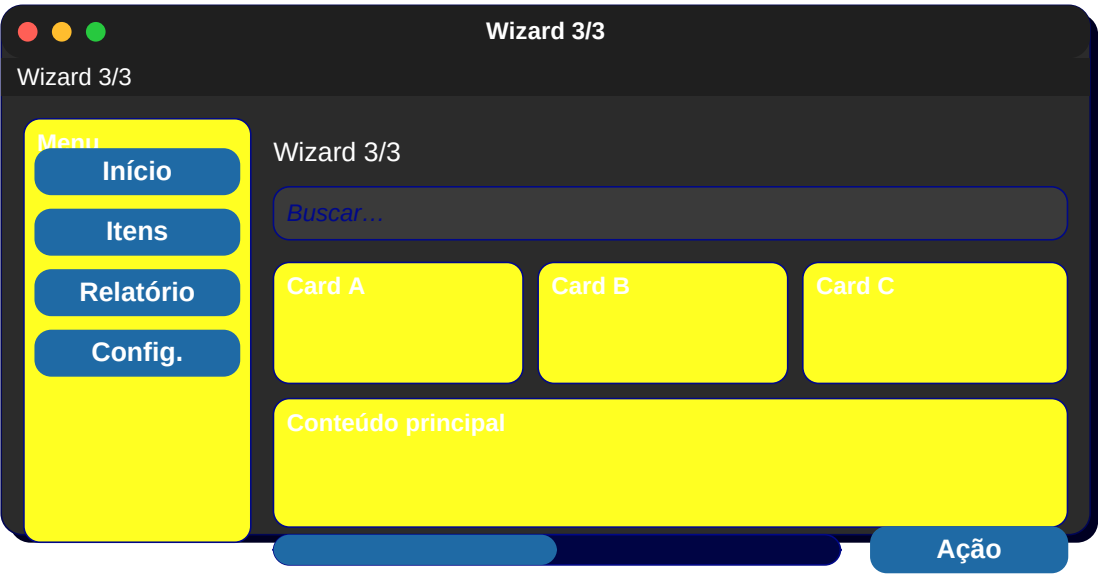
64. Wizard 2/3

Mesma base, novo conteúdo.



65. Wizard 3/3

Resumo + Confirmar.



66. Importar dados

Selecionar arquivo + mapear colunas.



67. Exportar

Escolher formato + filtros + Exportar.

Exportar

Exportar

Menu

Início

Itens

Relatório

Config.

Exportar

Buscar...

Card A

Card B

Card C

Conteúdo principal

Ação

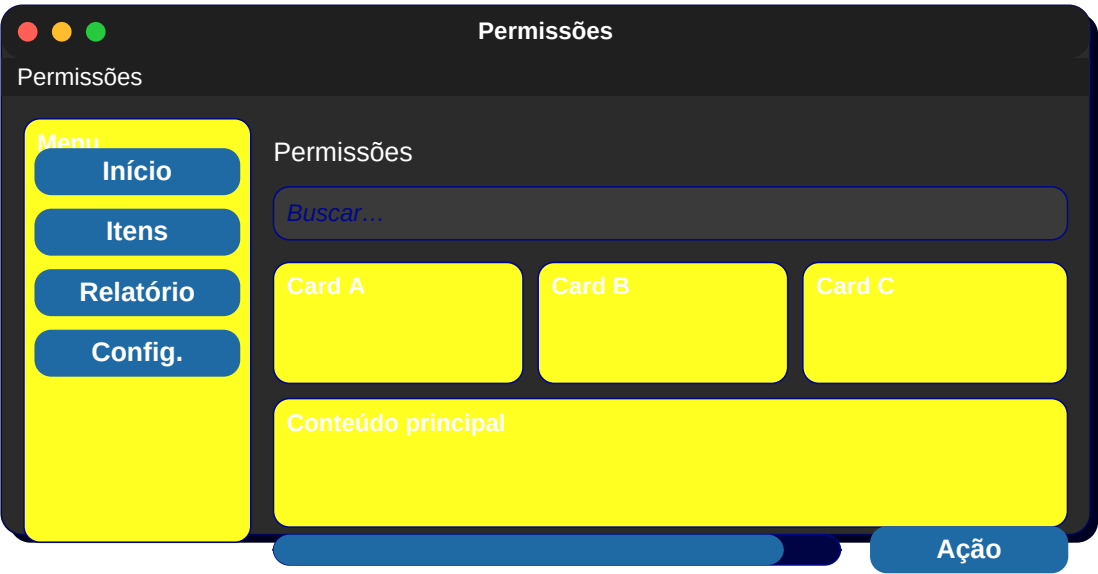
68. Backup

Lista de backups + Restaurar.



69. Permissões

Tabela usuário x permissão (matriz).



70. Logs

Lista monoespçada com filtros.

Logs

Logs

Menu

Início

Itens

Relatório

Config.

Logs

Buscar...

Card A

Card B

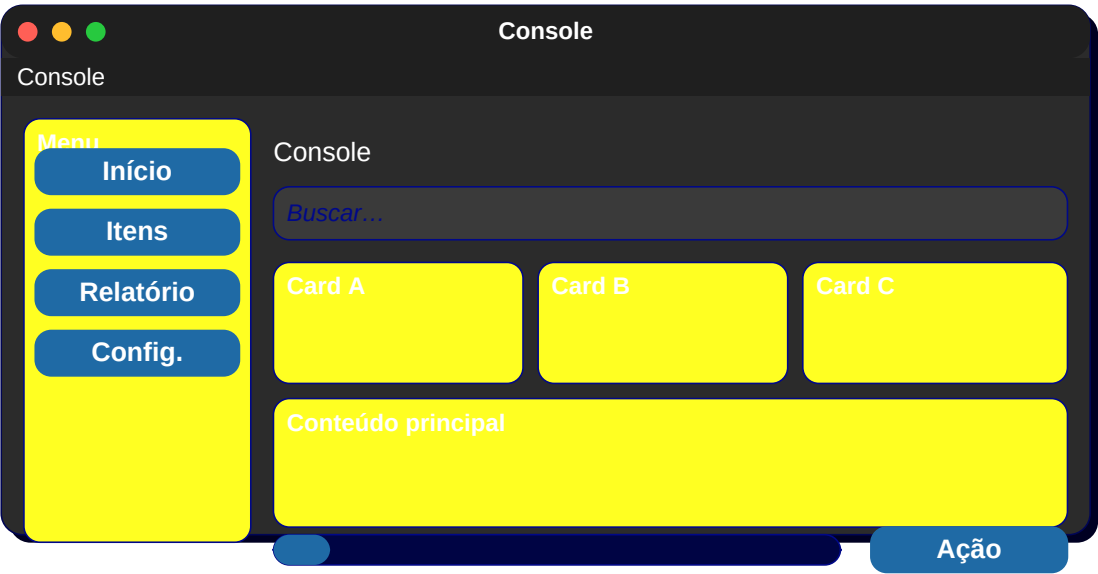
Card C

Conteúdo principal

Ação

71. Console

Terminal embutido.



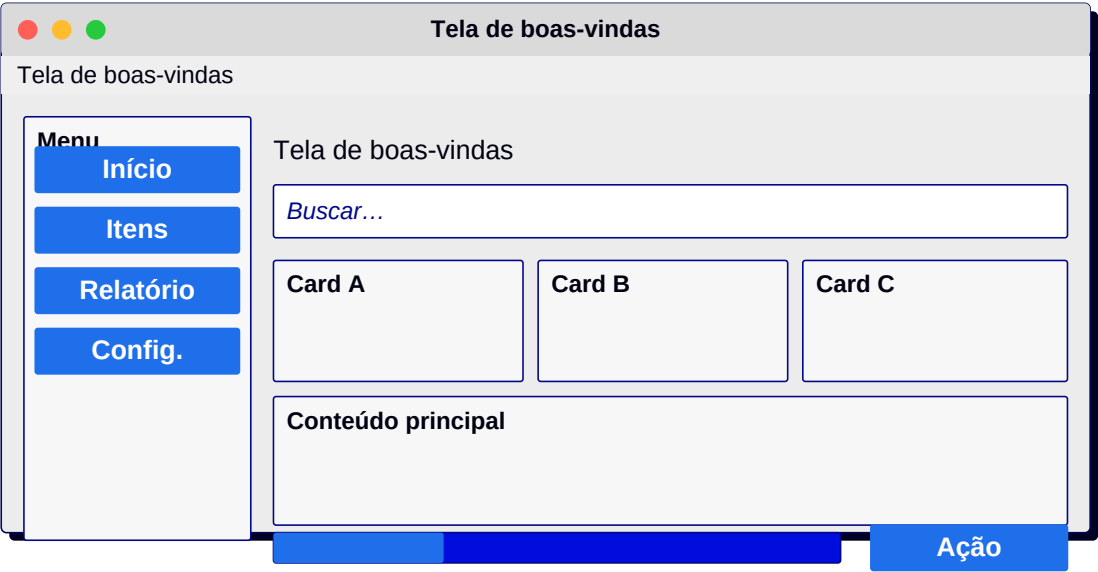
72. Diagnóstico

Status do sistema + métricas.



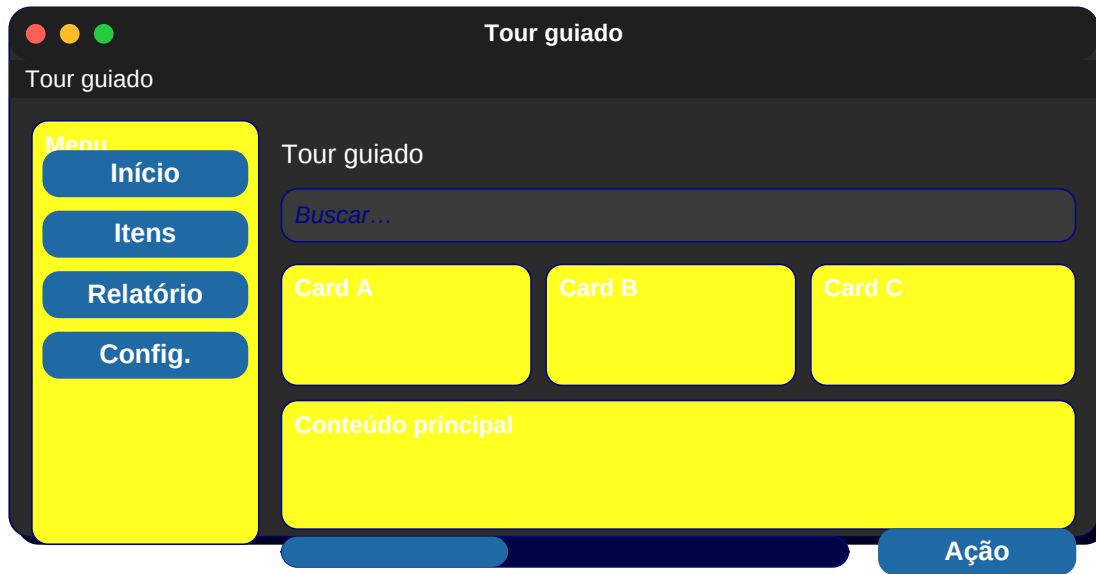
73. Tela de boas-vindas

Texto grande + CTA.



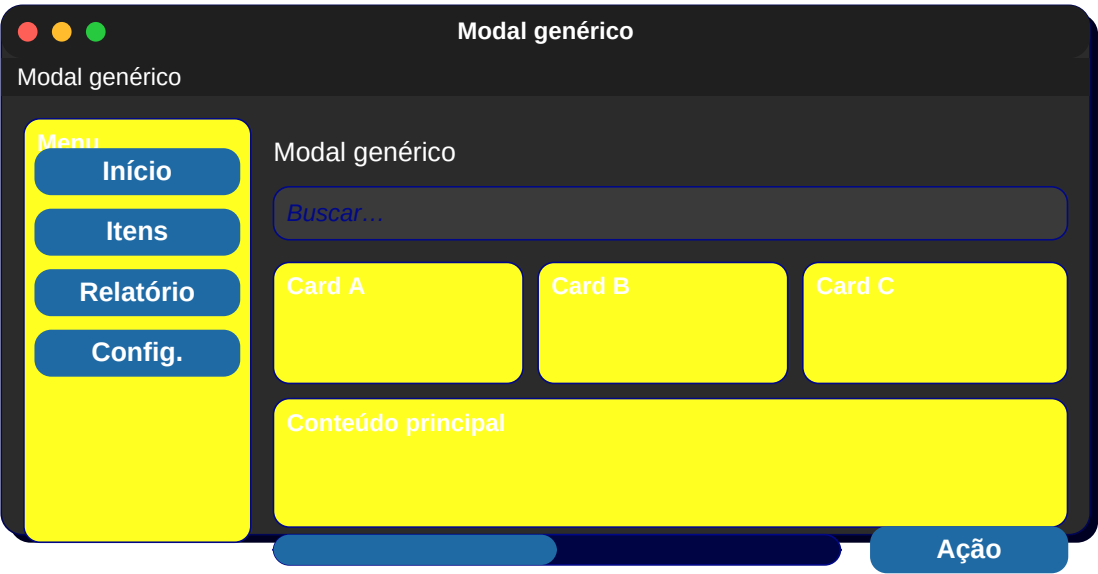
74. Tour guiado

Overlay com setas + dicas.



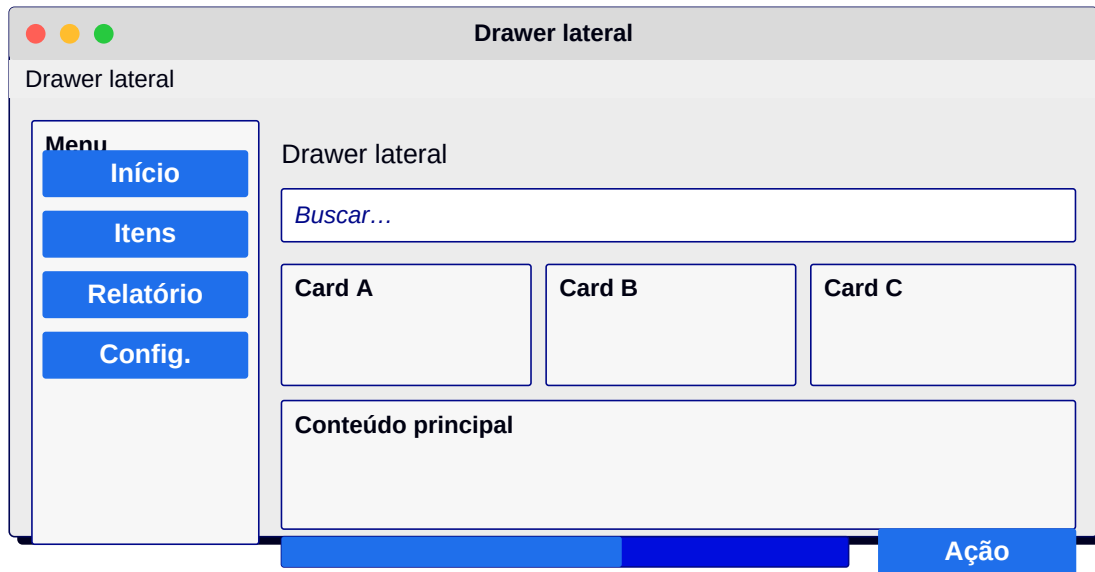
75. Modal genérico

Caixa centralizada + X no canto.



76. Drawer lateral

Painel deslizando da direita.



77. Toast

Mensagem temporária canto inferior.



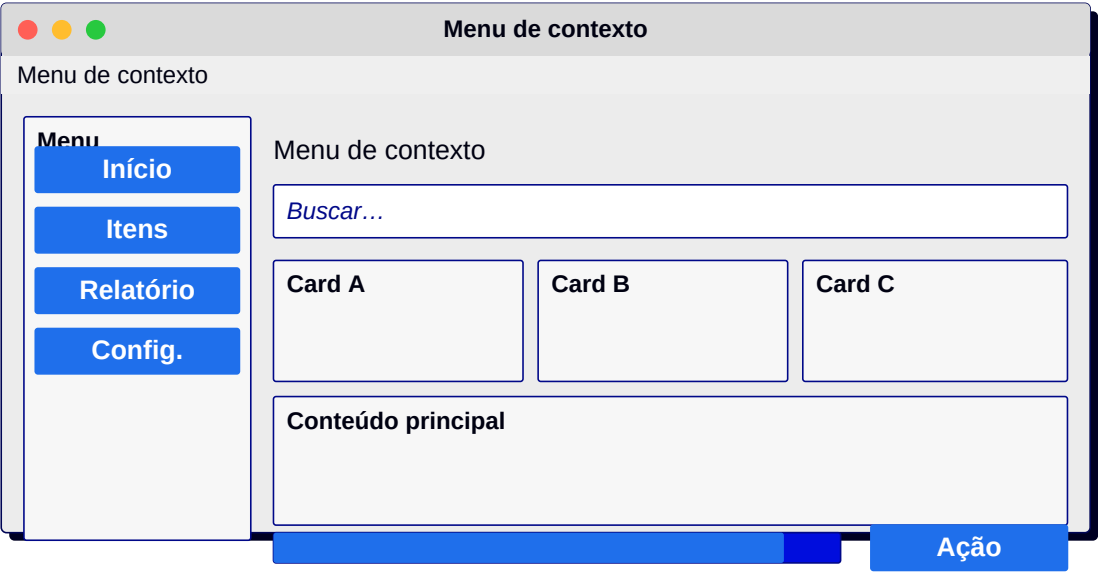
78. Tooltip

Balão pequeno apontando.



79. Menu de contexto

Lista flutuante com ações.



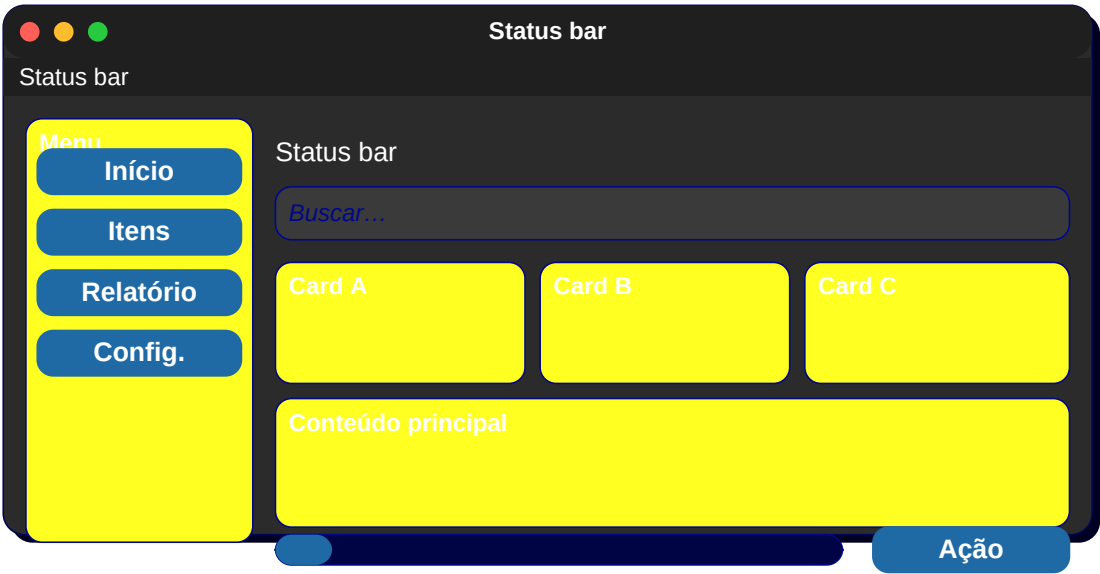
80. Toolbar

Barra de ícones + separadores.



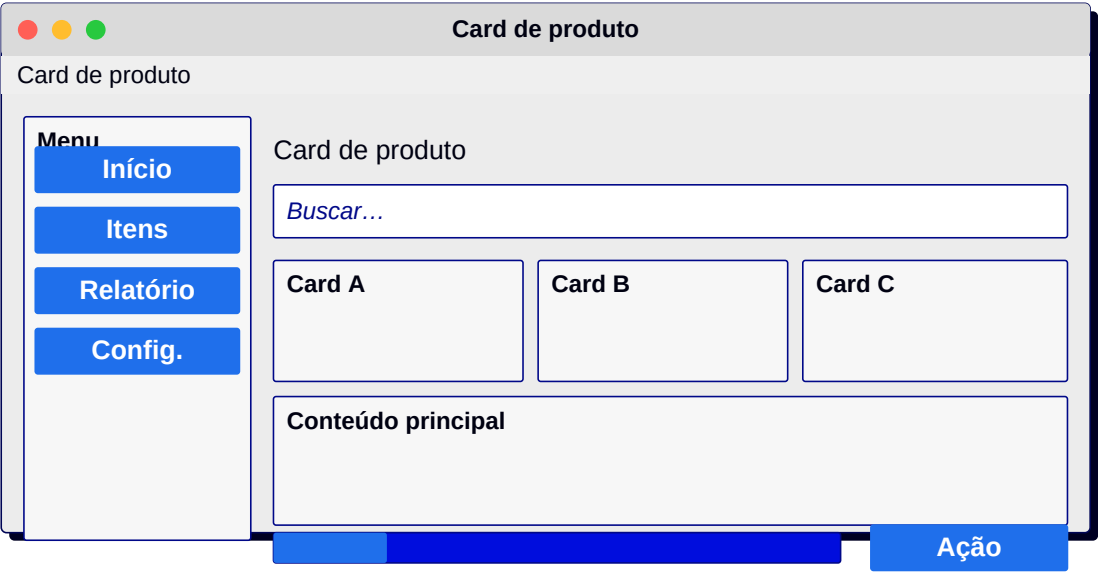
81. Status bar

Texto + ícones + progresso.



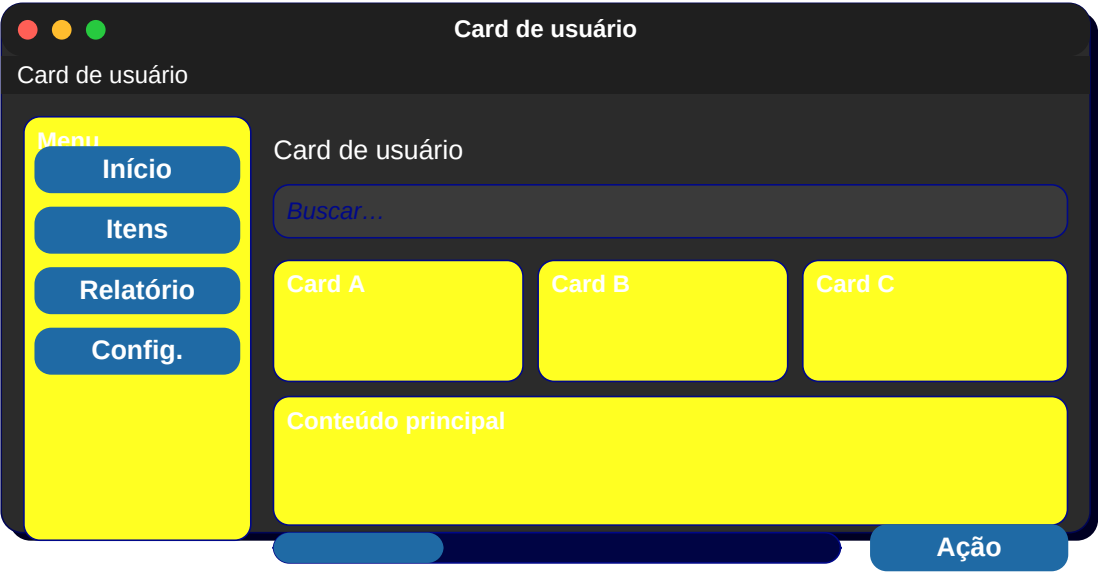
82. Card de produto

Imagem + título + preço + botão.



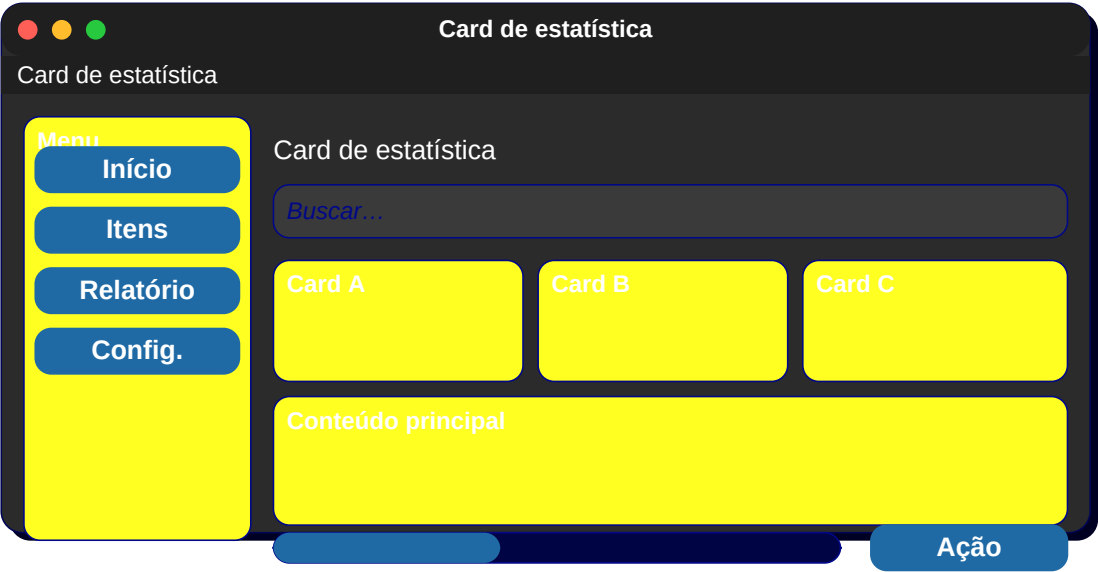
83. Card de usuário

Avatar + nome + cargo + ações.



84. Card de estatística

Número grande + label + delta.



85. Card de tarefa

Título + tags + responsável.

Card de tarefa

Card de tarefa

Menu

Início

Itens

Relatório

Config.

Card de tarefa

Buscar...

Card A

Card B

Card C

Conteúdo principal

Ação

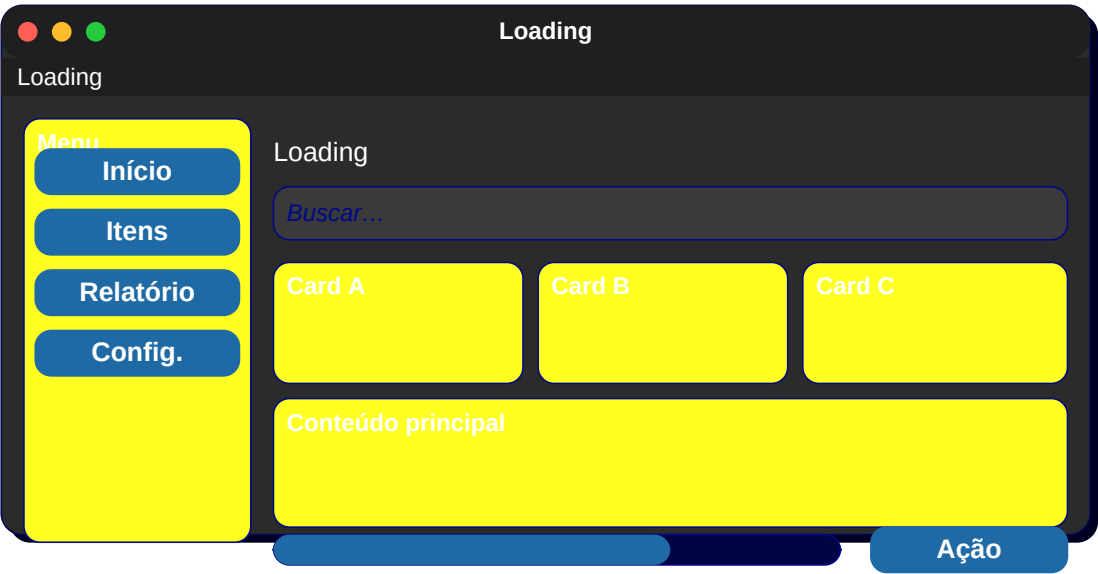
86. Empty state

Ilustração + texto + CTA.



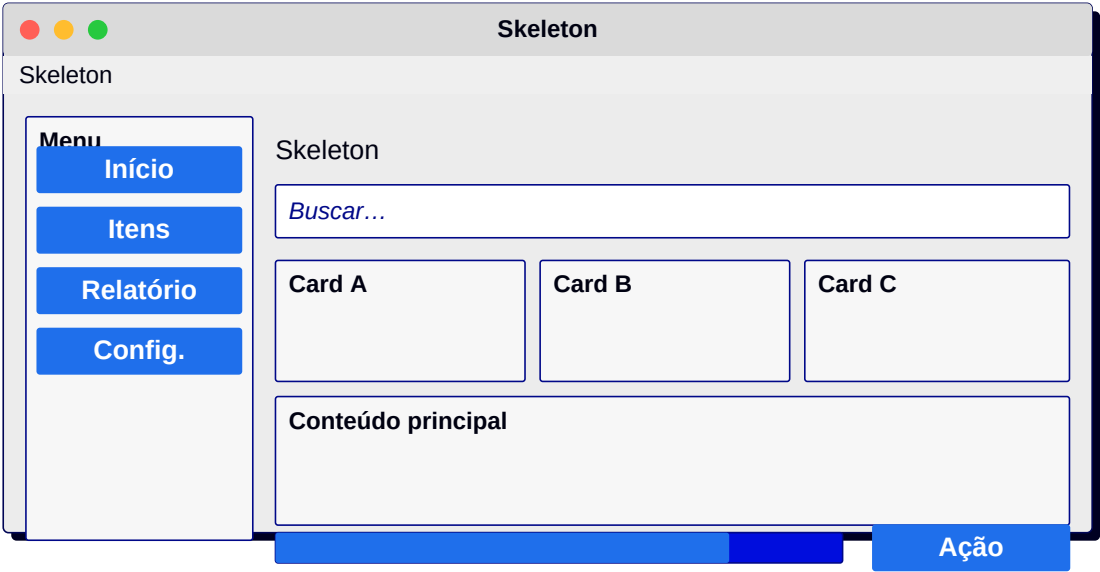
87. Loading

Spinner + texto curto.



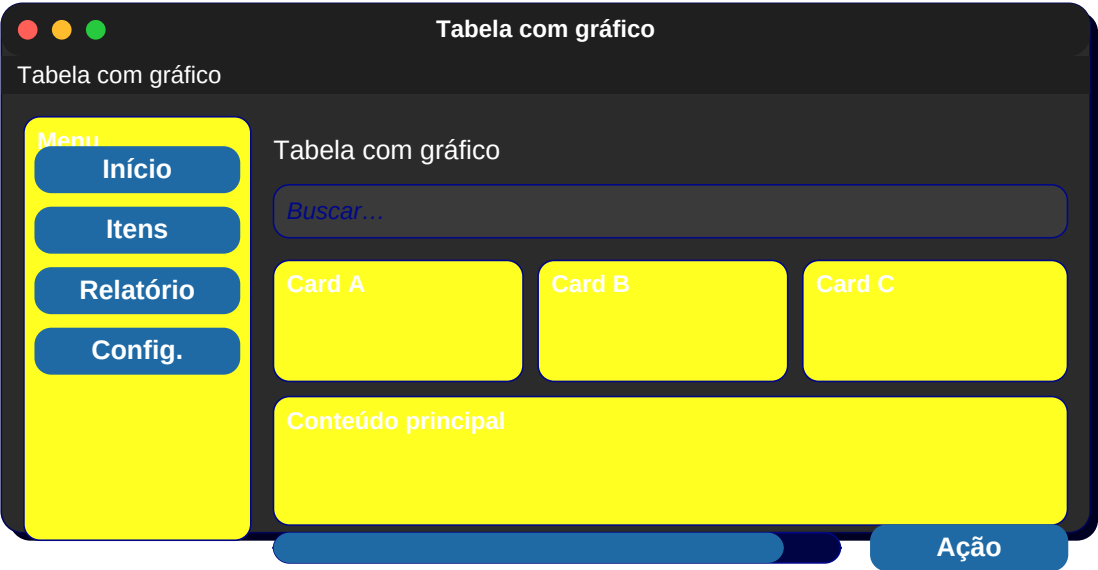
88. Skeleton

Placeholders cinzentos.



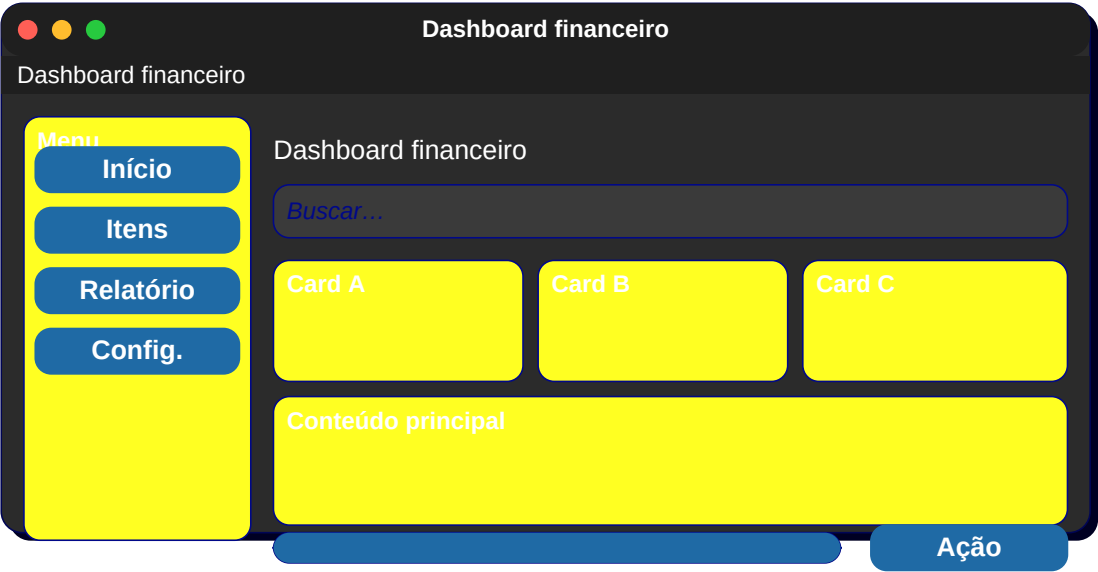
89. Tabela com gráfico

Treeview + painel direito com gráfico.



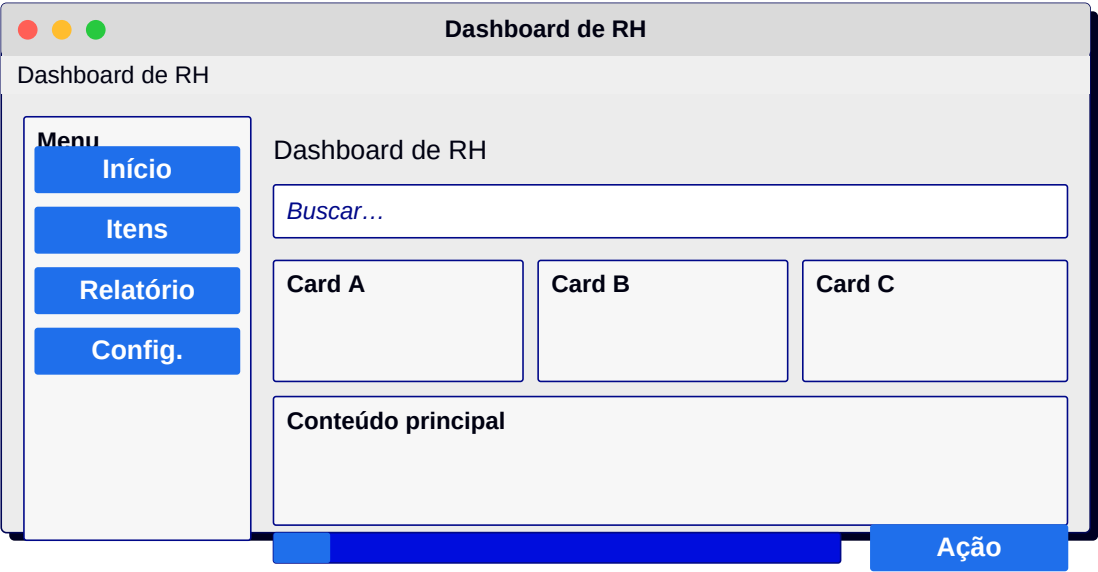
90. Dashboard financeiro

Saldo + entradas + saídas + projeção.



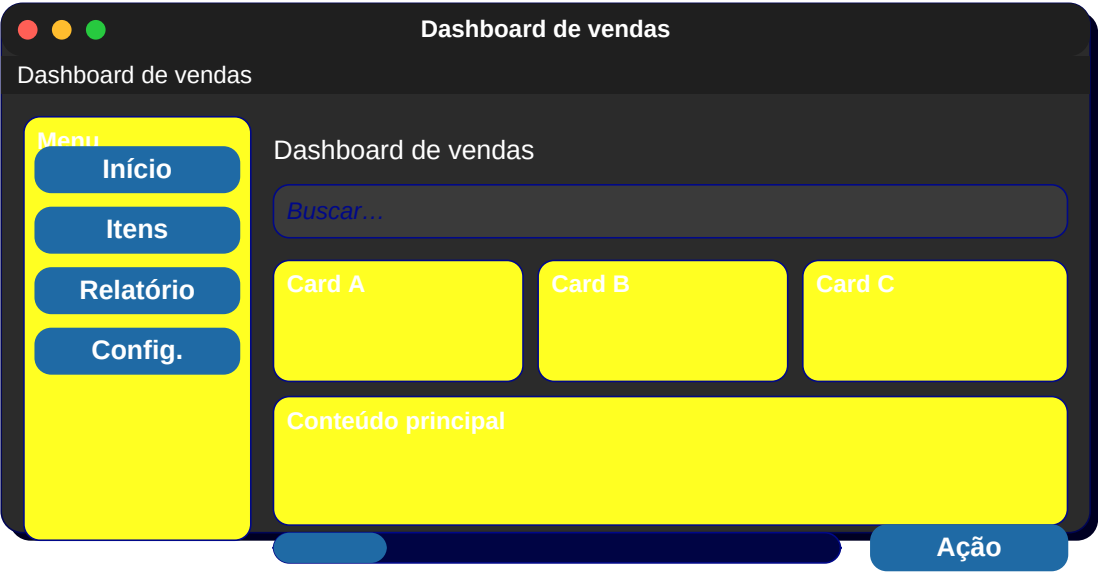
91. Dashboard de RH

Funcionários + ausências + admissões.



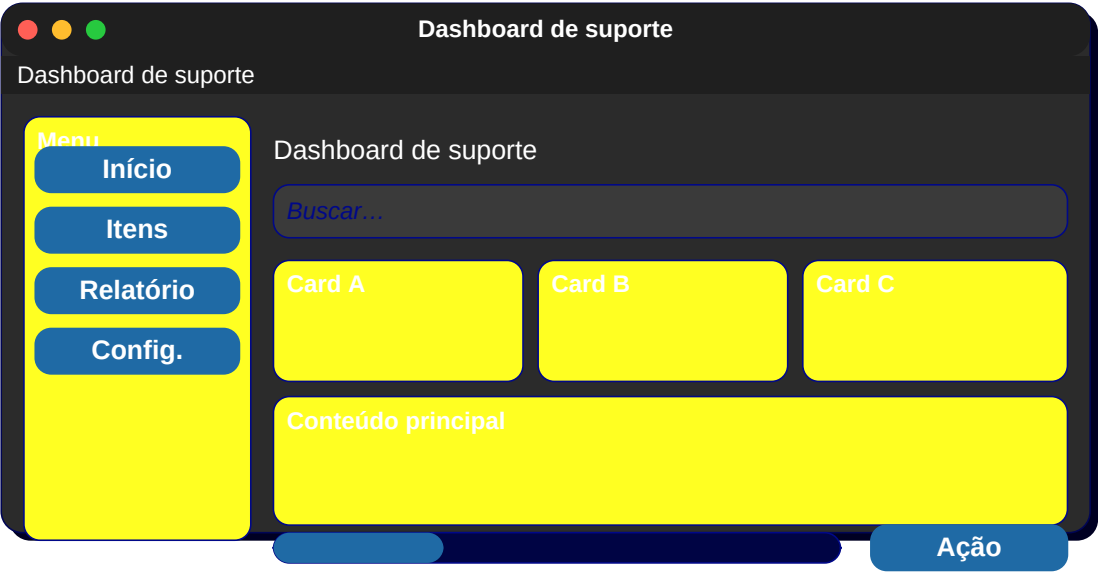
92. Dashboard de vendas

Receita + top produtos + funil.



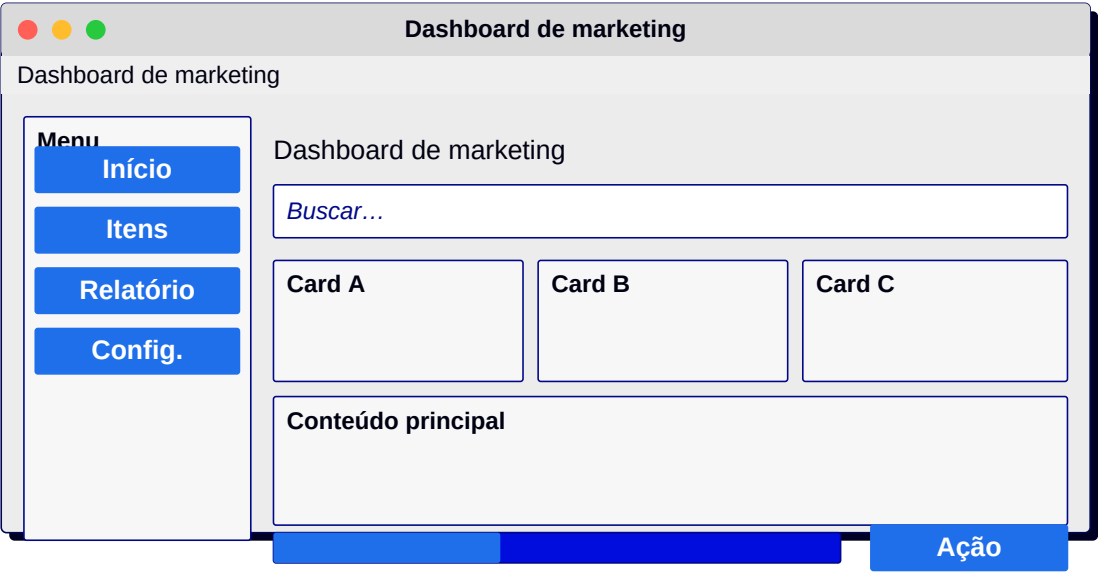
93. Dashboard de suporte

Tickets + SLA + satisfação.



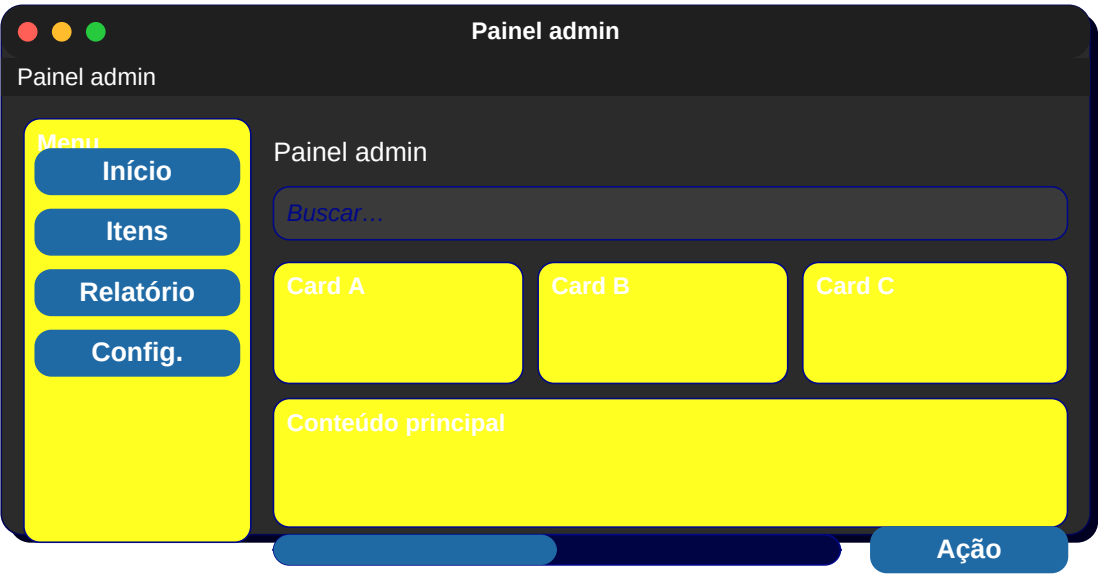
94. Dashboard de marketing

Campanhas + CTR + conversão.



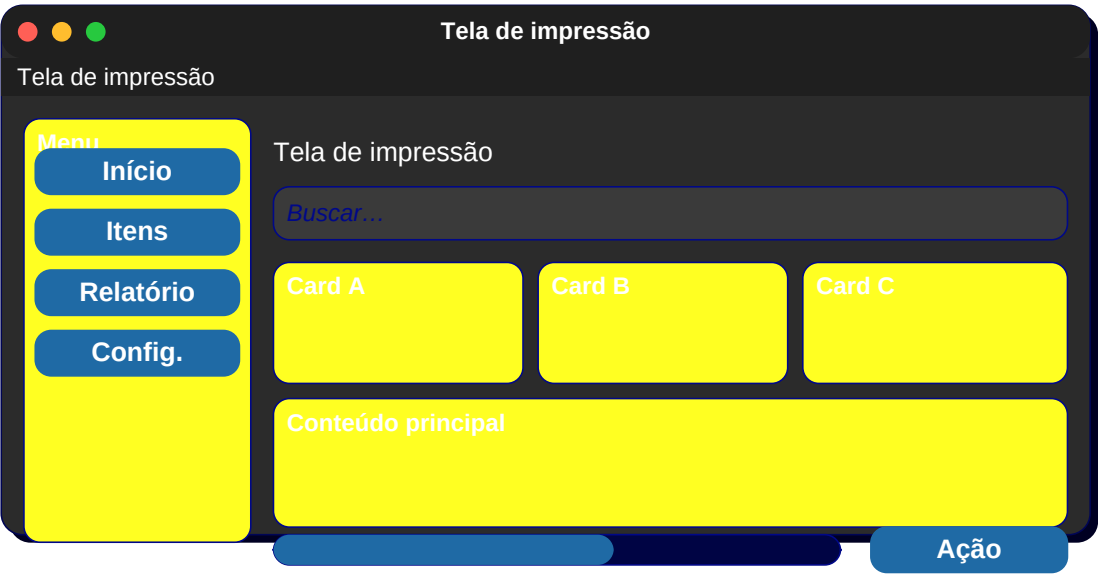
95. Painel admin

Usuários + sistema + auditoria.



96. Tela de impressão

Preview A4 + botão imprimir.



97. Configuração de impressora

Lista + propriedades.

Configuração de impressora

Configuração de impressora

Menu

Início

Itens

Relatório

Config.

Configuração de impressora

Buscar...

Card A

Card B

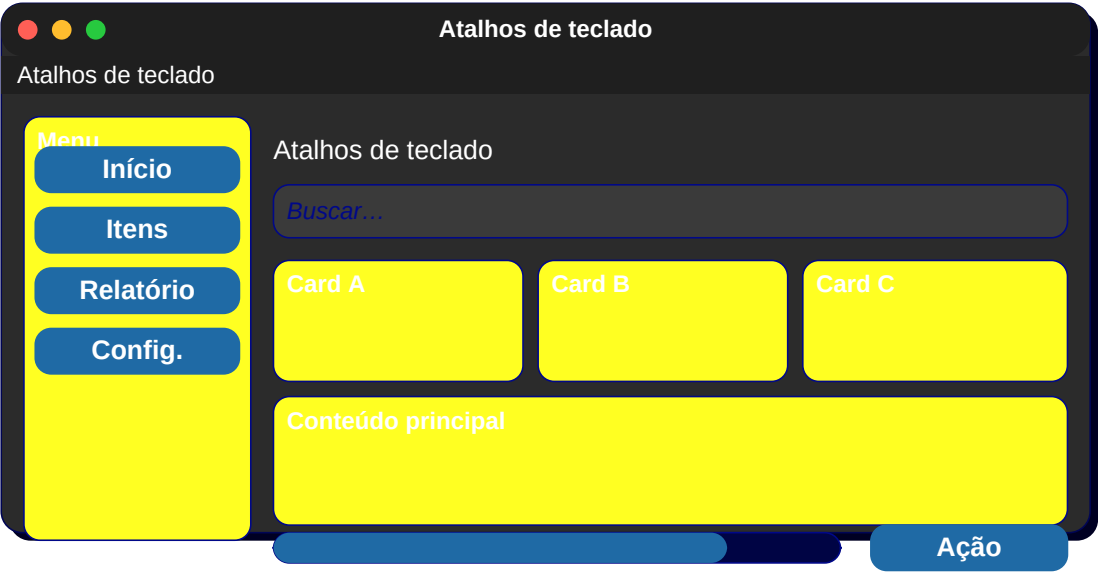
Card C

Conteúdo principal

Ação

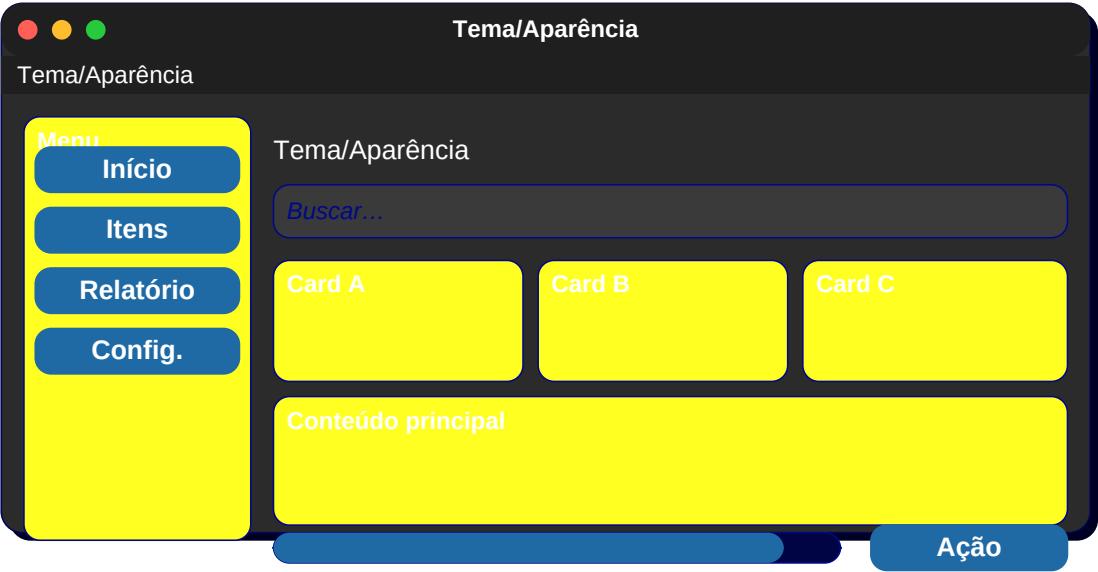
98. Atalhos de teclado

Lista de combinações + descrição.



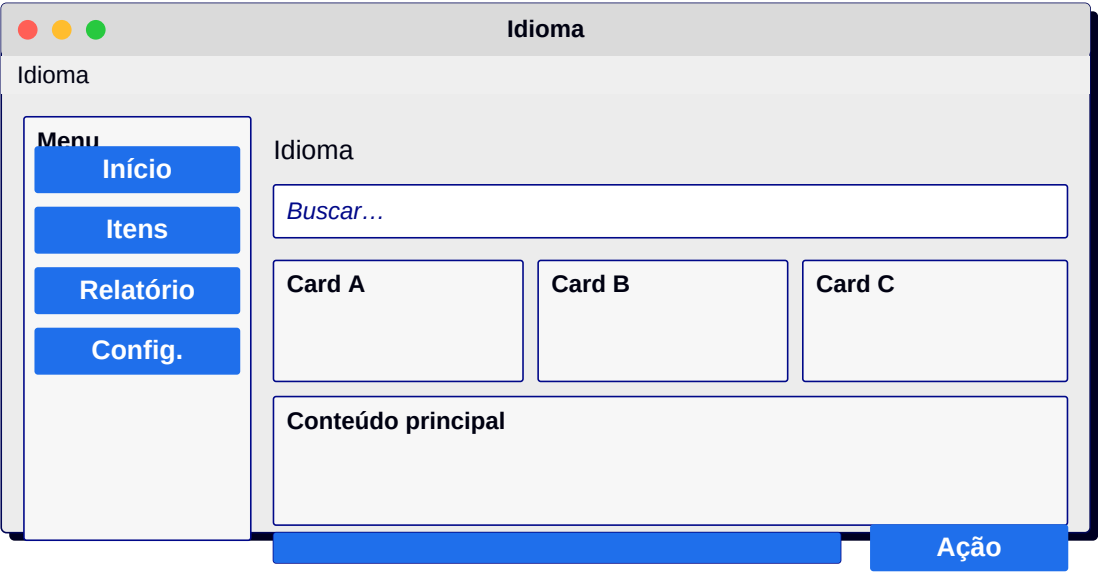
99. Tema/Aparência

Light/Dark/System + cor primária.



100. Idioma

Lista de idiomas com bandeira.



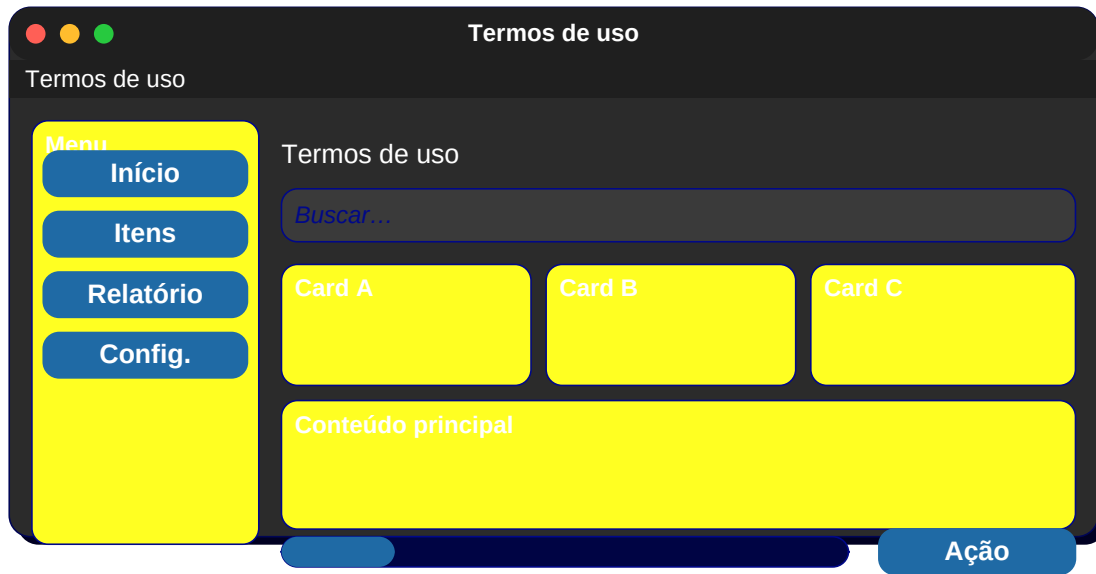
101. Sobre

Logo + versão + créditos.



102. Termos de uso

Texto longo + scroll + botão aceitar.



103. Política de privacidade

Texto + scroll.

Política de privacidade

Política de privacidade

Menu

Início

Itens

Relatório

Config.

Política de privacidade

Buscar...

Card A

Card B

Card C

Conteúdo principal

Ação

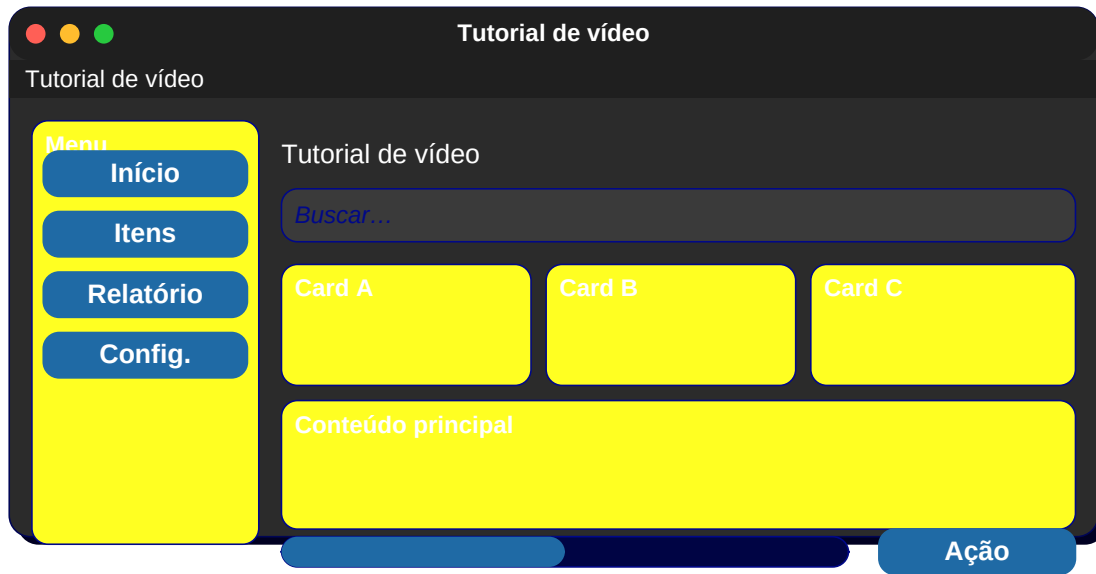
104. Ajuda/FAQ

Lista expansível.



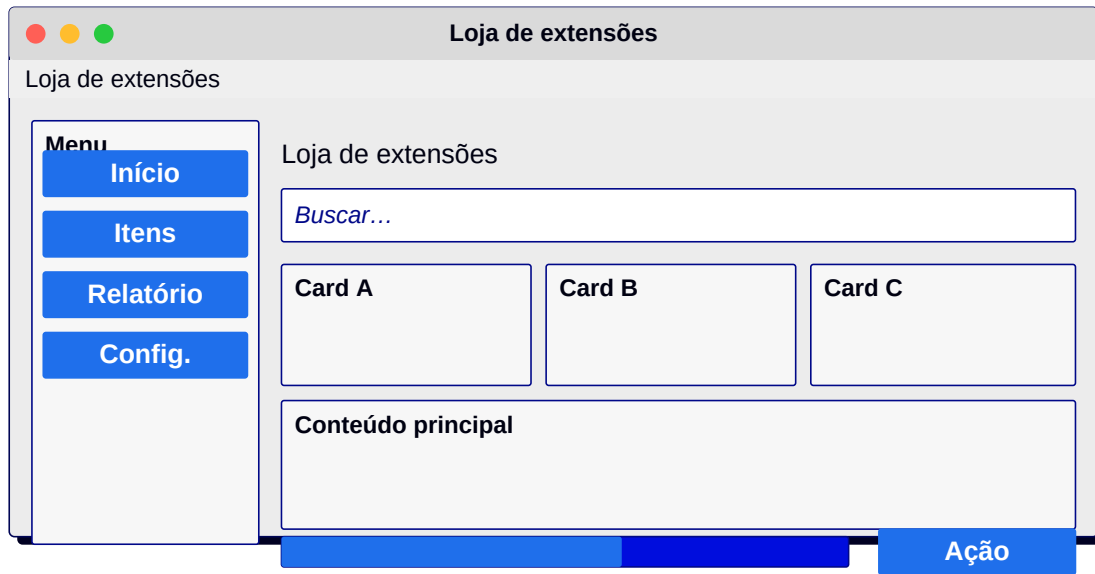
105. Tutorial de vídeo

Player + lista de capítulos.



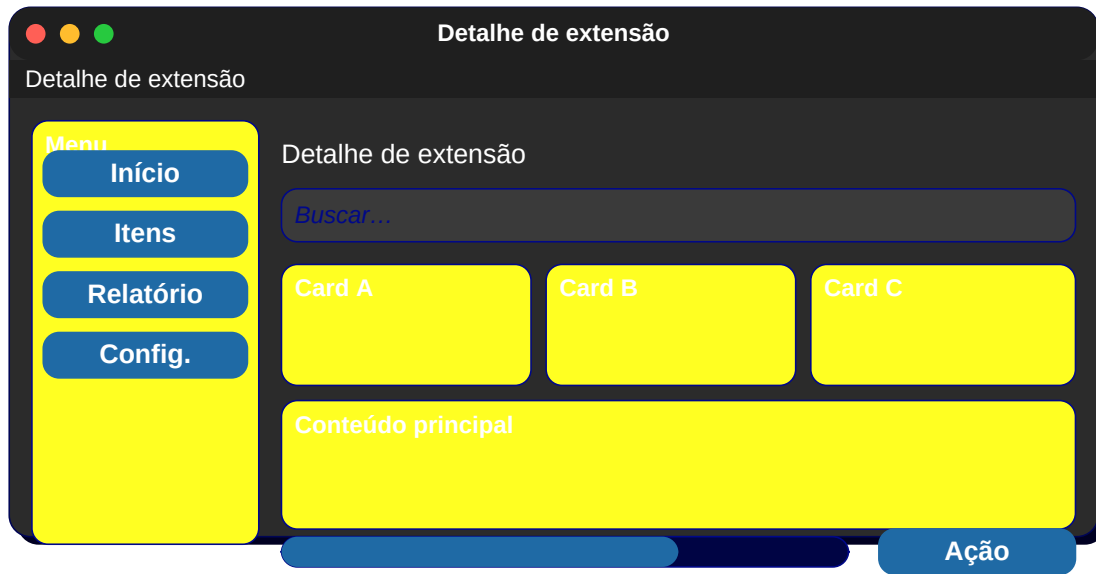
106. Loja de extensões

Cards + busca + categorias.



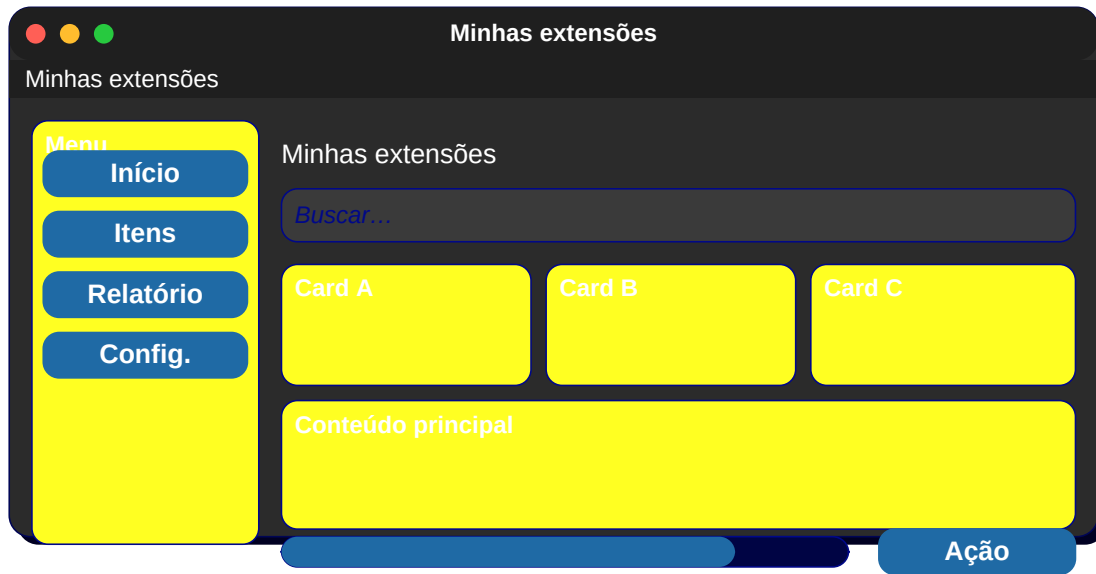
107. Detalhe de extensão

Imagens + descrição + Instalar.



108. Minhas extensões

Lista + atualizar.



109. Atualizações

Lista de mudanças + Reiniciar.

Atualizações

Atualizações

Menu

Início

Itens

Relatório

Config.

Atualizações

Buscar...

Card A

Card B

Card C

Conteúdo principal

Ação

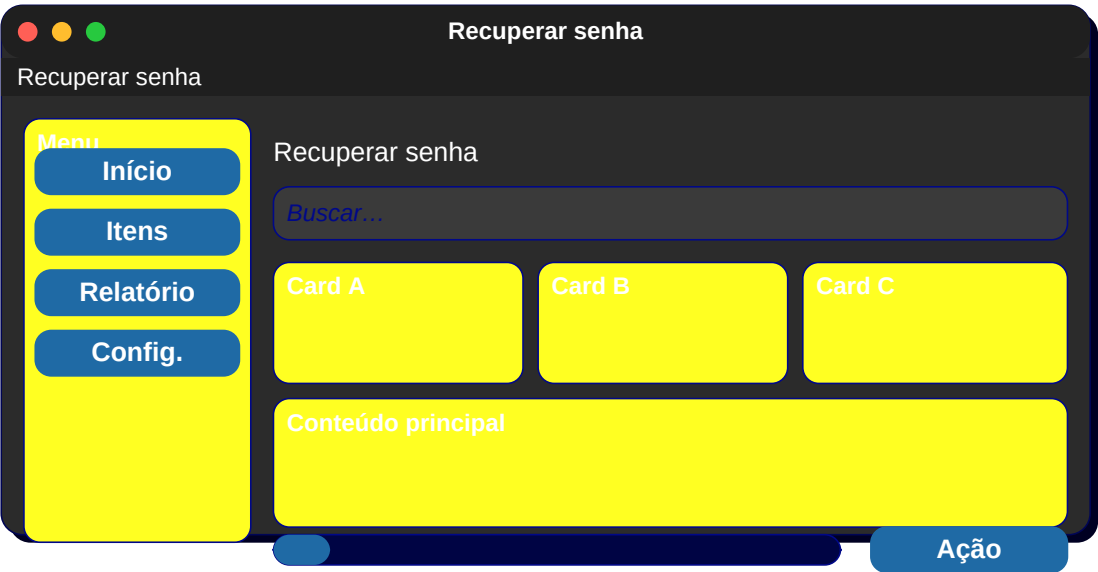
110. Login social

Botões Google/Apple/Github.



111. Recuperar senha

Email + Enviar.



112. Verificação

6 caixinhas para código.

Verificação

Verificação

Menu

Início

Itens

Relatório

Config.

Verificação

Buscar...

Card A

Card B

Card C

Conteúdo principal

Ação

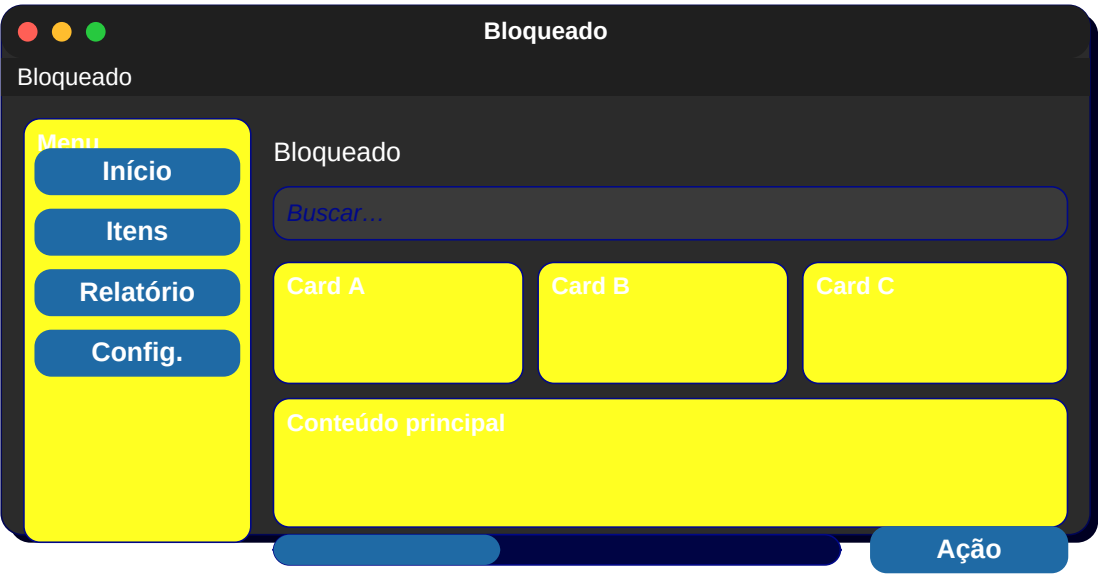
113. 2FA

QR code + entrada.



114. Bloqueado

Mensagem + Contate o suporte.



115. Sessões ativas

Lista de dispositivos + Encerrar.

Sessões ativas

Sessões ativas

Menu

Início

Itens

Relatório

Config.

Sessões ativas

Buscar...

Card A

Card B

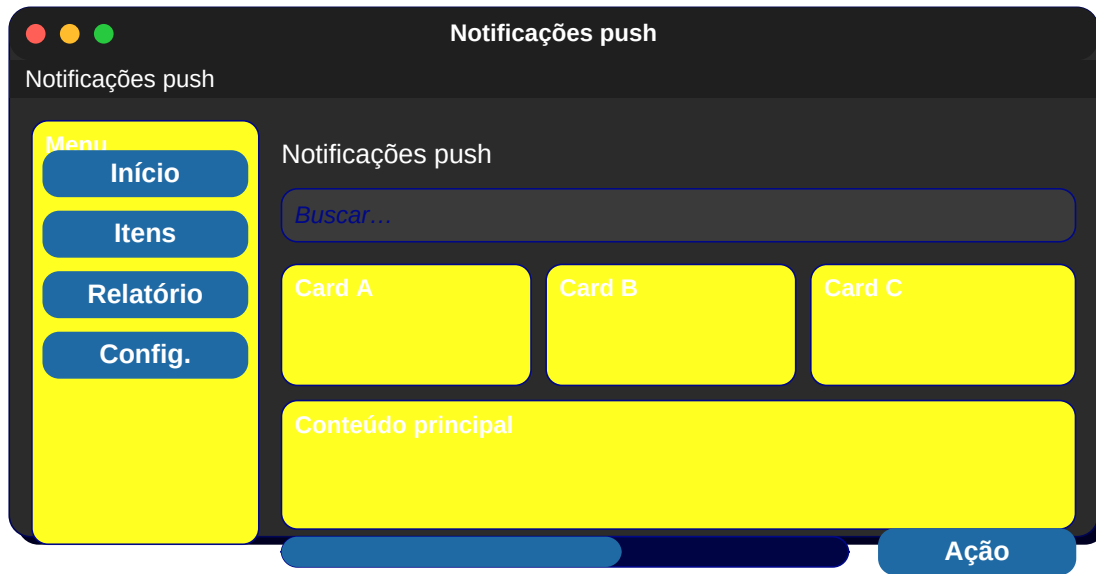
Card C

Conteúdo principal

Ação

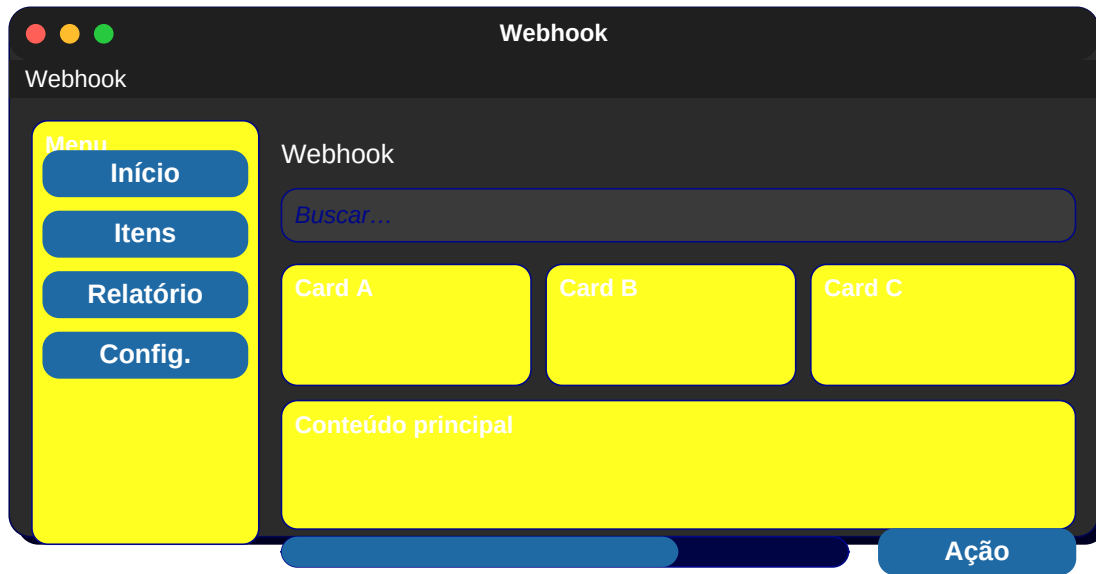
116. Notificações push

Histórico + configurações.



117. Webhook

Lista de URLs + Adicionar.



118. Tokens de API

Lista + Gerar novo.

Tokens de API

Tokens de API

Menu

Início

Itens

Relatório

Config.

Tokens de API

Buscar...

Card A

Card B

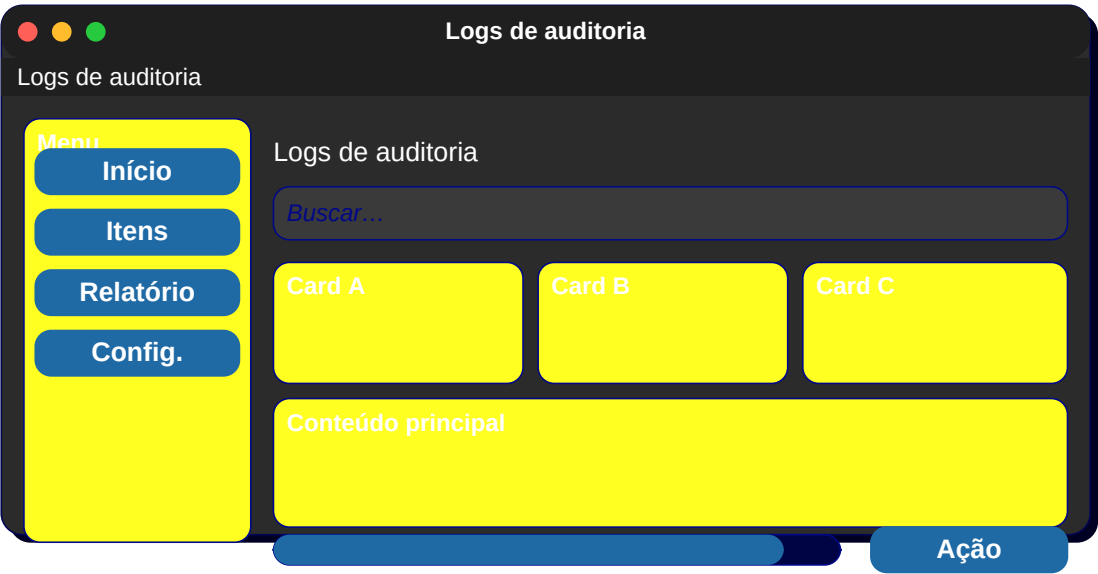
Card C

Conteúdo principal

Ação

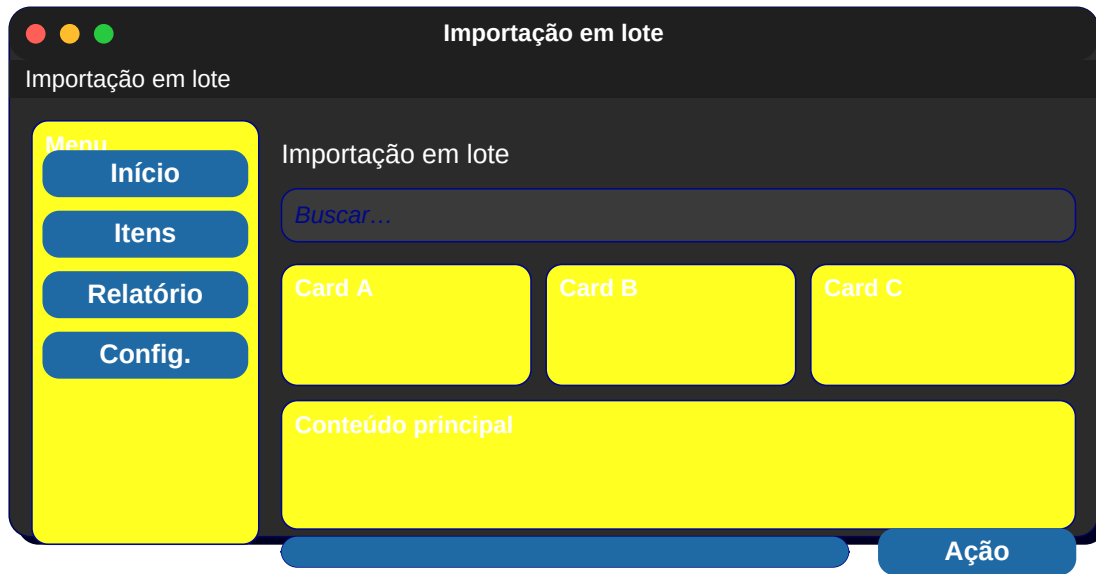
119. Logs de auditoria

Quem fez o quê e quando.



120. Importação em lote

Drag-and-drop + lista de arquivos.



121. Exportar relatório

Filtros + formato + Gerar.

Exportar relatório

Exportar relatório

Menu

Início

Itens

Relatório

Config.

Exportar relatório

Buscar...

Card A

Card B

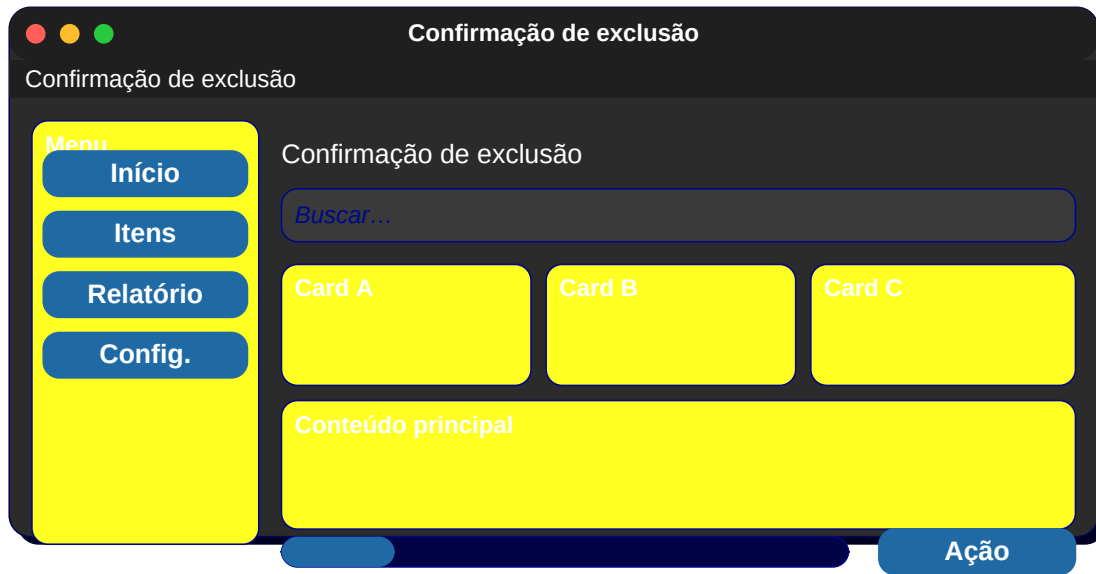
Card C

Conteúdo principal

Ação

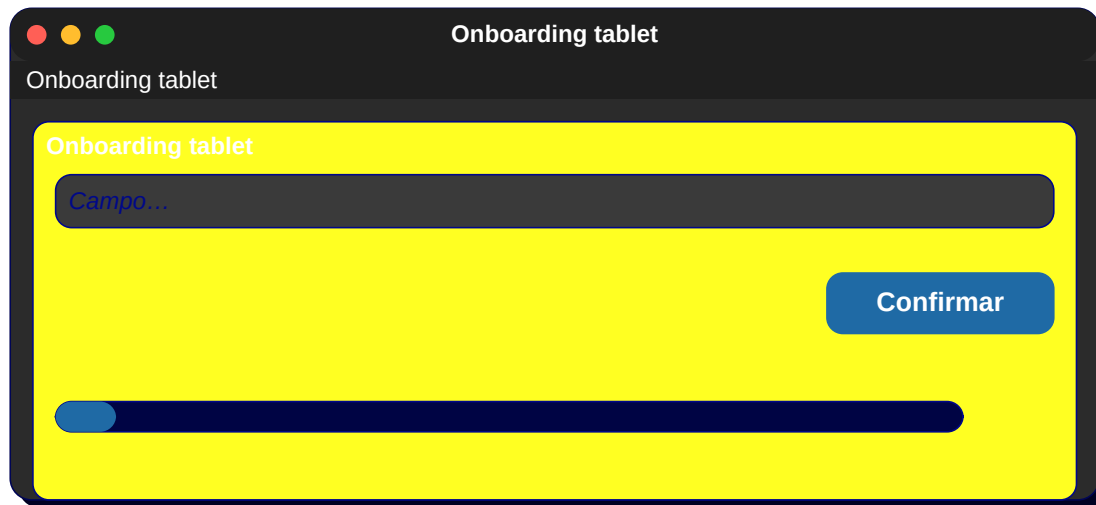
122. Confirmação de exclusão

Aviso vermelho + Confirmar.



Tela bônus 1: Onboarding tablet

Mockup adicional para onboarding tablet.



Tela bônus 2: POS de caixa

Mockup adicional para pos de caixa.

POS de caixa

POS de caixa

Campo...

Confirmar

Tela bônus 3: Reserva de mesa

Mockup adicional para reserva de mesa.

Reserva de mesa

Reserva de mesa

Campo...

Confirmar

Tela bônus 4: Reserva de hotel

Mockup adicional para reserva de hotel.

Reserva de hotel

Reserva de hotel

Campo...

Confirmar

Tela bônus 5: Reserva de voo

Mockup adicional para reserva de voo.



Tela bônus 6: Locação de carro

Mockup adicional para locação de carro.

Locação de carro

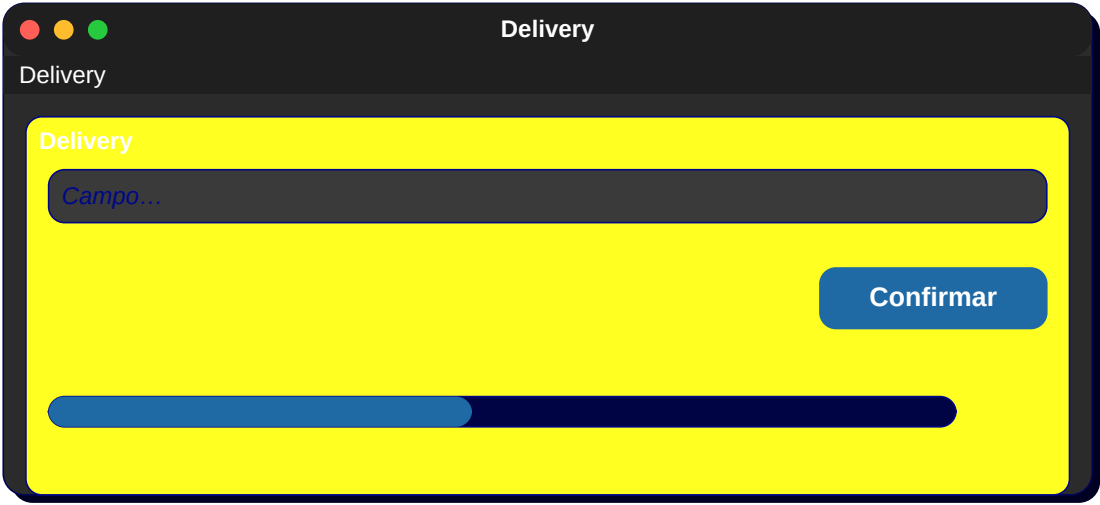
Locação de carro

Campo...

Confirmar

Tela bônus 7: Delivery

Mockup adicional para delivery.



Tela bônus 8: Rastreamento

Mockup adicional para rastreamento.

Rastreamento

Rastreamento

Campo...

Confirmar

Tela bônus 9: Avaliação NPS

Mockup adicional para avaliação nps.

Avaliação NPS

Avaliação NPS

Campo...

Confirmar

Tela bônus 10: Pesquisa

Mockup adicional para pesquisa.

Pesquisa

Pesquisa

Pesquisa

Campo...

Confirmar

Tela bônus 11: Tela de espera

Mockup adicional para tela de espera.



Tela bônus 12: Confirmação e-mail

Mockup adicional para confirmação e-mail.

Confirmação e-mail

Confirmação e-mail

Confirmação e-mail

Campo...

Confirmar

Tela bônus 13: Reset senha

Mockup adicional para reset senha.



Tela bônus 14: Convite

Mockup adicional para convite.

Convite

Convite

Campo...

Confirmar

Tela bônus 15: Aceitar convite

Mockup adicional para aceitar convite.



**Fim — Apostila com mais de 200 páginas e 80+ telas
desenhadas. Bons estudos!**